

CSCI 598H

Applied Computer Vision for Engineering Systems

Lecture 6-1: Object Detection & Segmentation

Dr. Minghan Li

Assistant Professor

Computer Science Department

Colorado School of Mines

Sept/29/2026



Administration

- #3 Code Section



[https://
www.when2me
et.com/?
38932314-5P7
wk](https://www.when2meet.com/?38932314-5P7wk)

CSCI 598H Course Overview



Weeks 2-4

Weeks 5-10

Weeks 11-16

Module 1

Deep Learning Basics

- Linear Classifier and Optimization
- CNNs and BackPropagation
- RNNs, Attention, Transformers

Module 2

Modern Visual Intelligence

- Perceiving and understanding the visual world
- Generative models and foundation models
- Geometry, 3D vision and spatial understanding

Module 3

Engineering Systems

- Robotics
- Earth science
- Medical imaging
- Energy

Recap:

Computer Vision Tasks

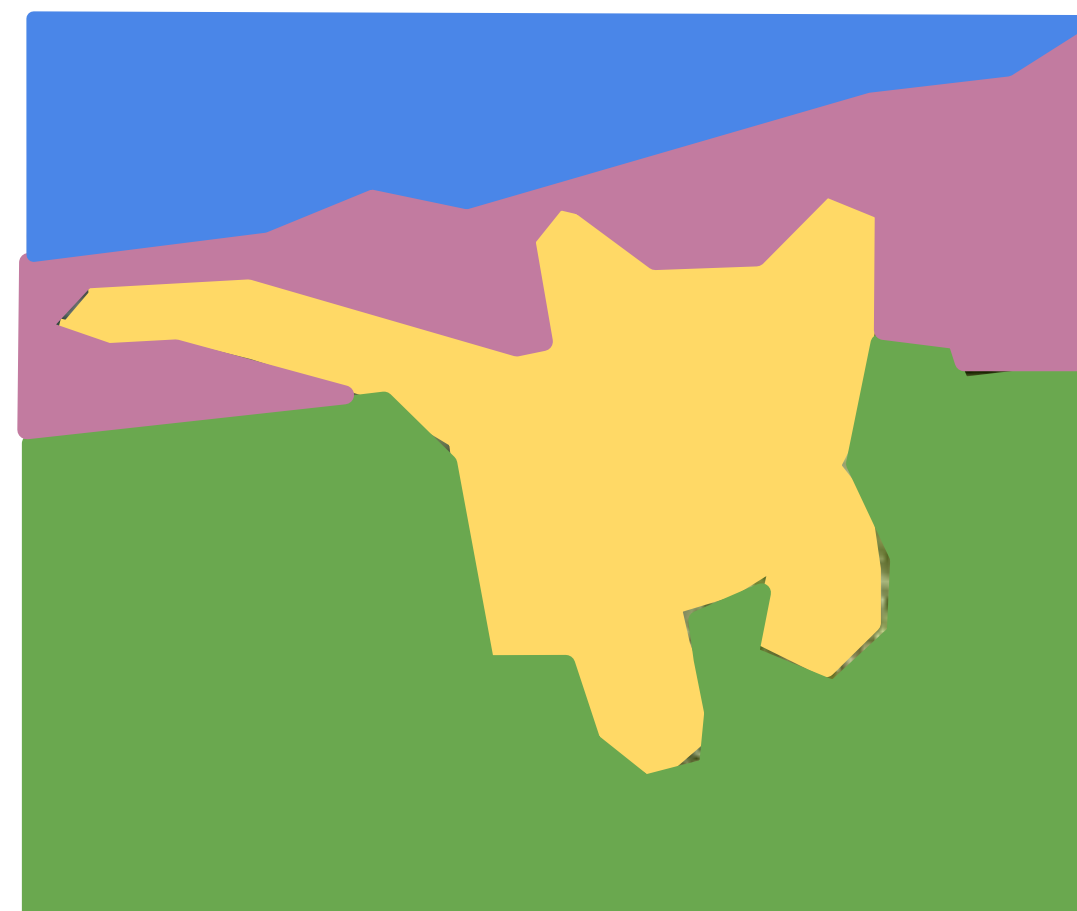
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation

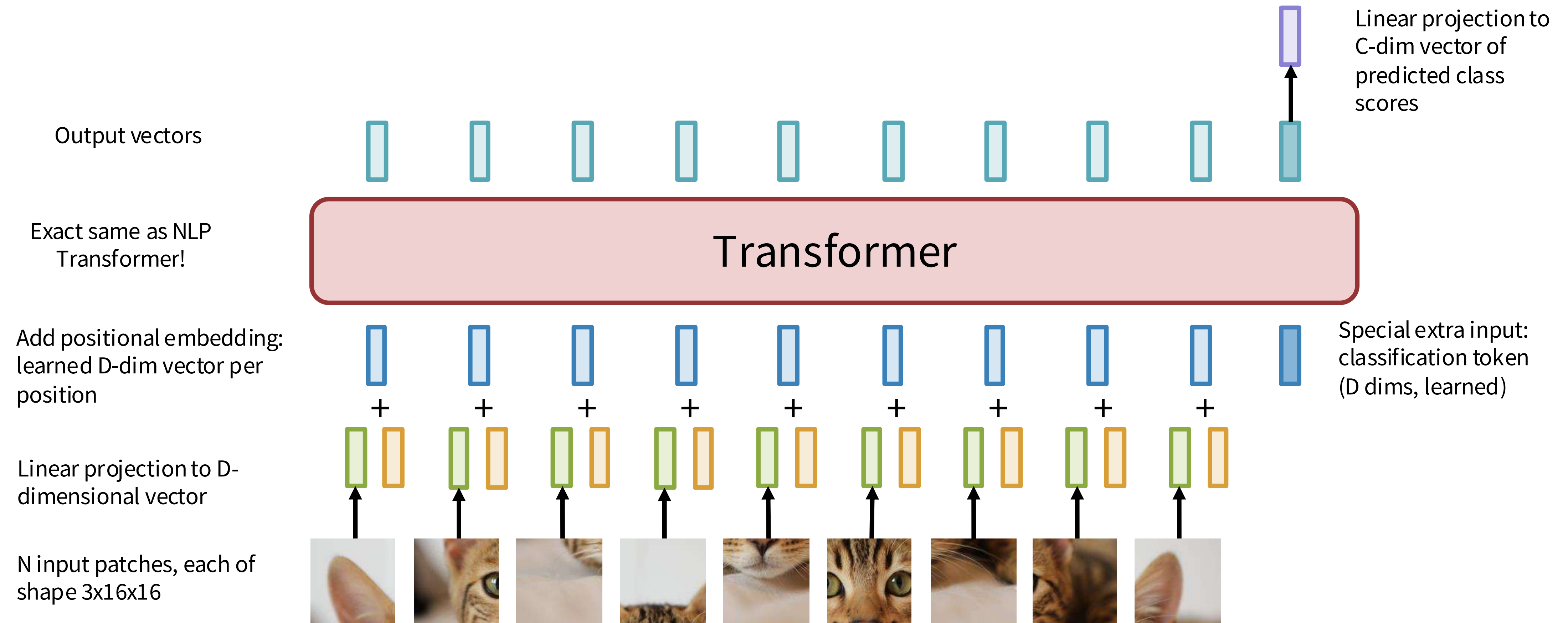


DOG, DOG, CAT

[This image is CC0 public domain](#)

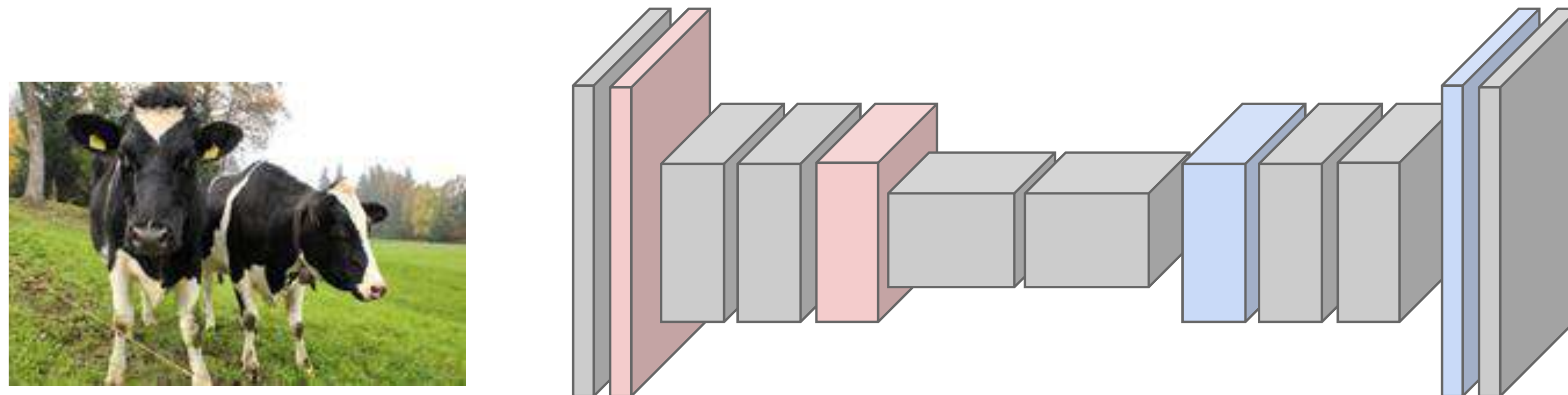
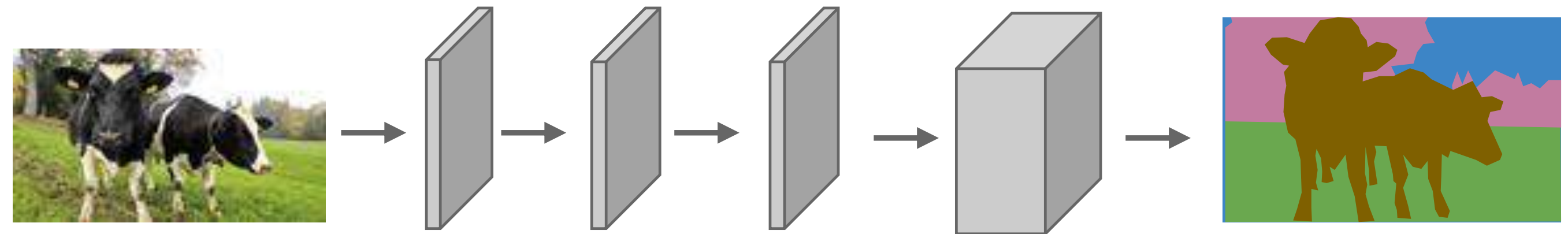
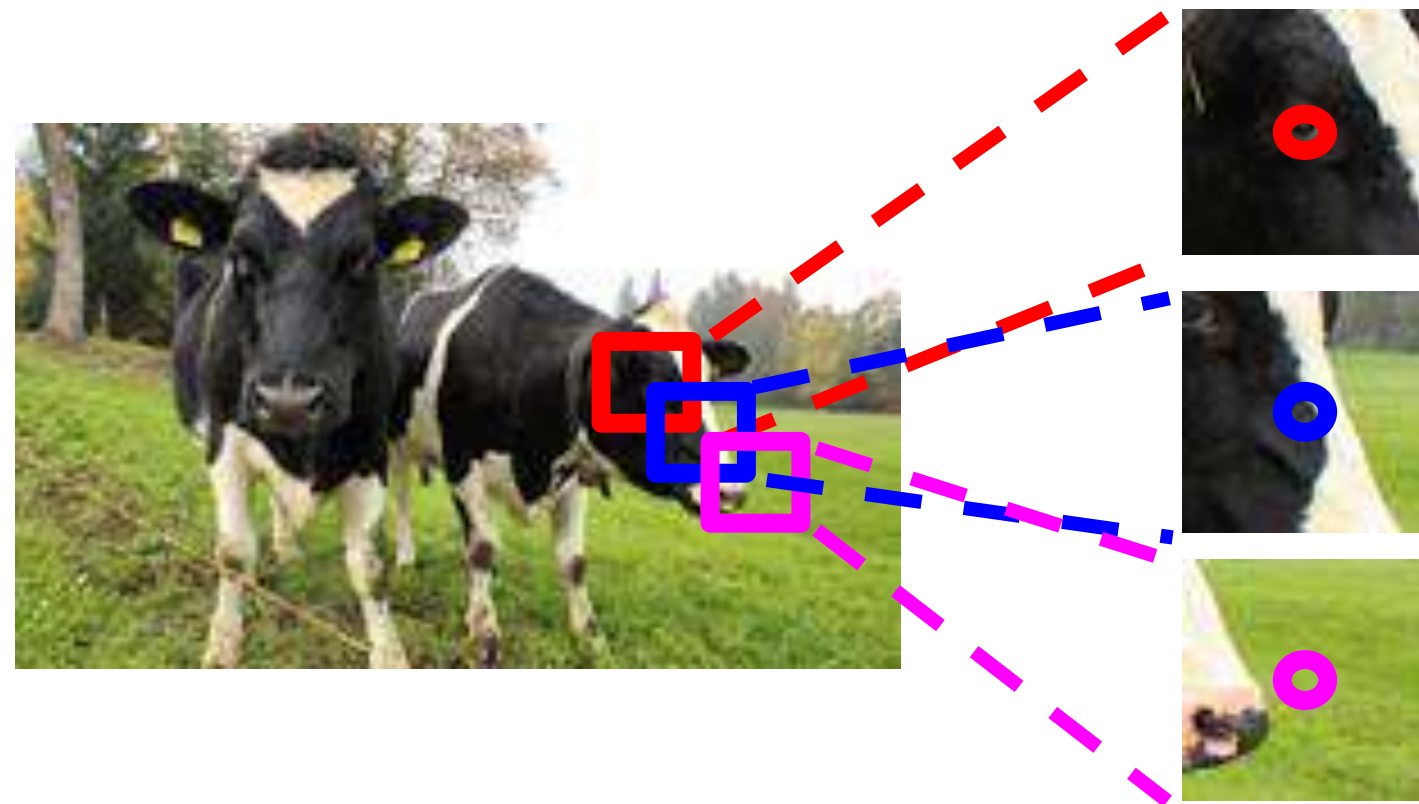
Recap:

Vision Transformers (ViT)



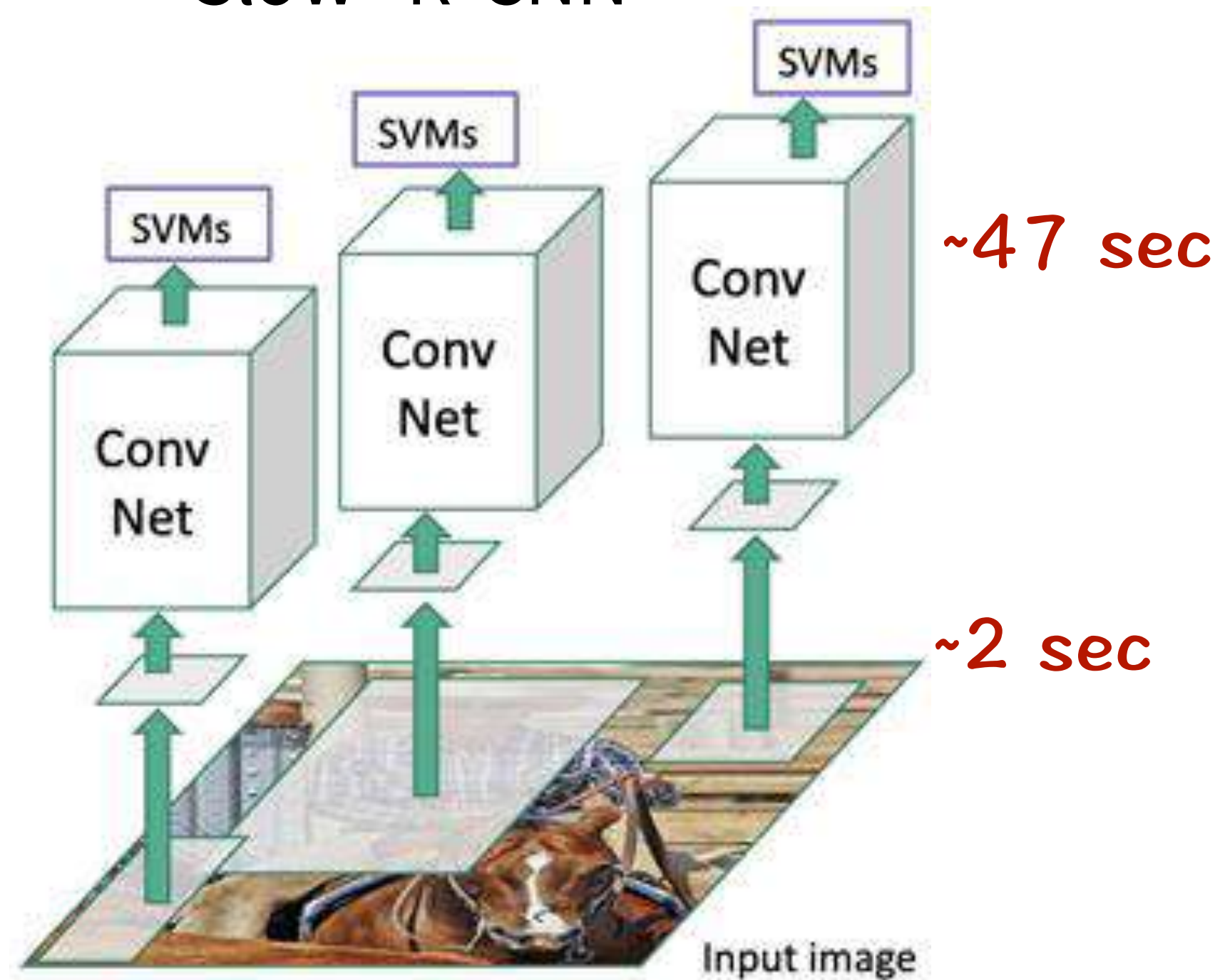
Recap:

Semantic Segmentation: Summary

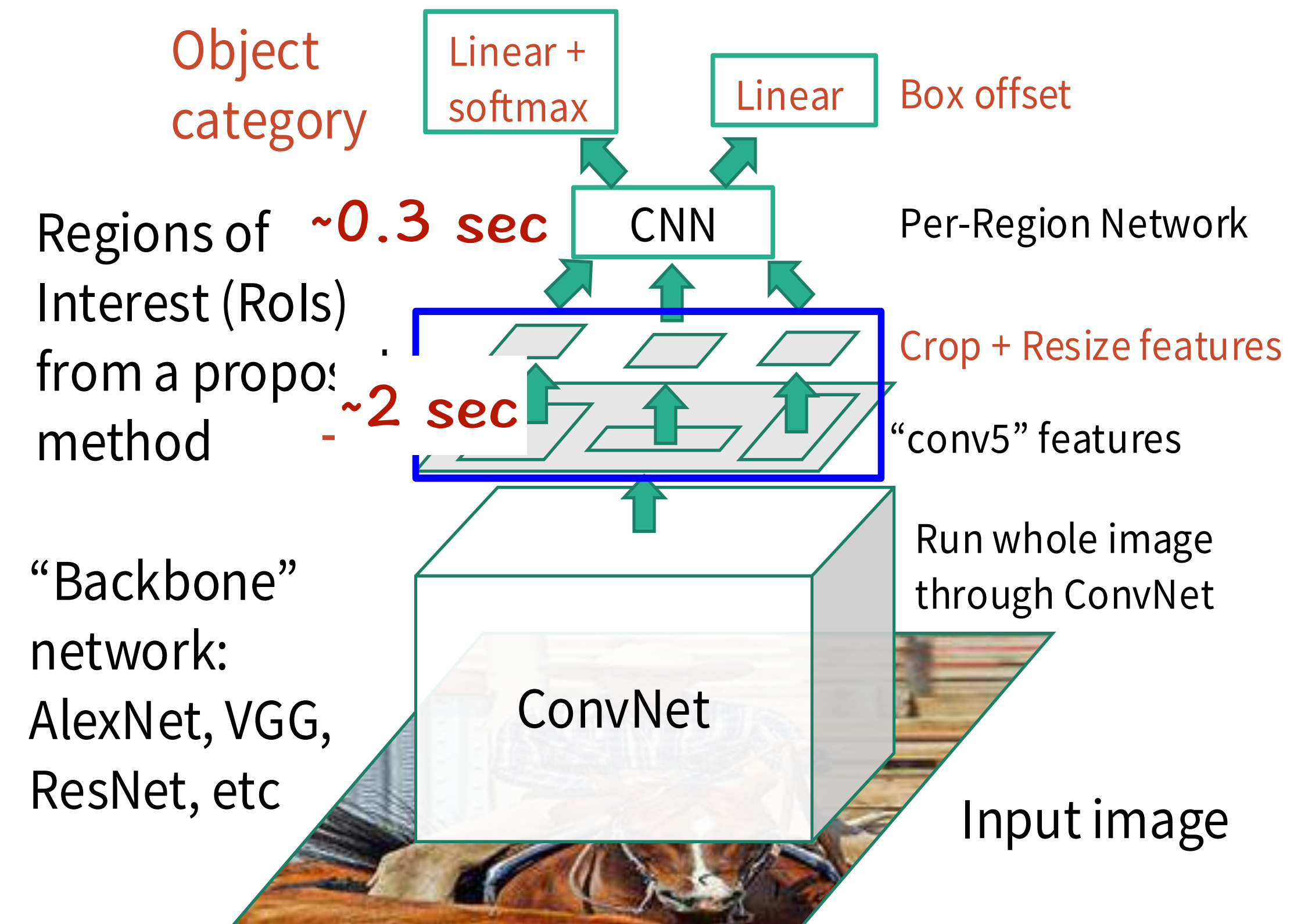


Recap: Object Detection Summary

“Slow” R-CNN

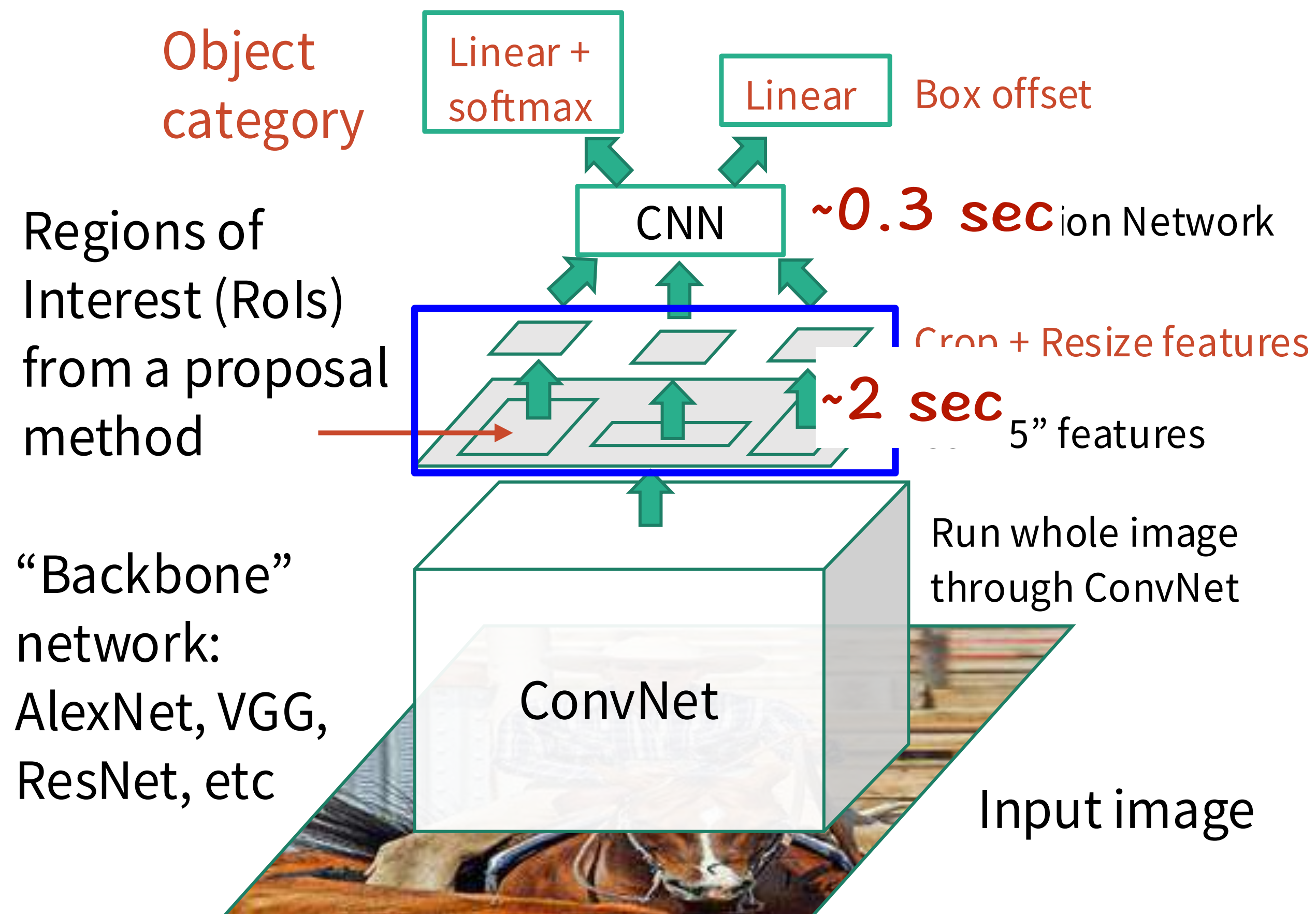


Fast R-CNN



Recap: Object Detection Summary

Fast R-CNN

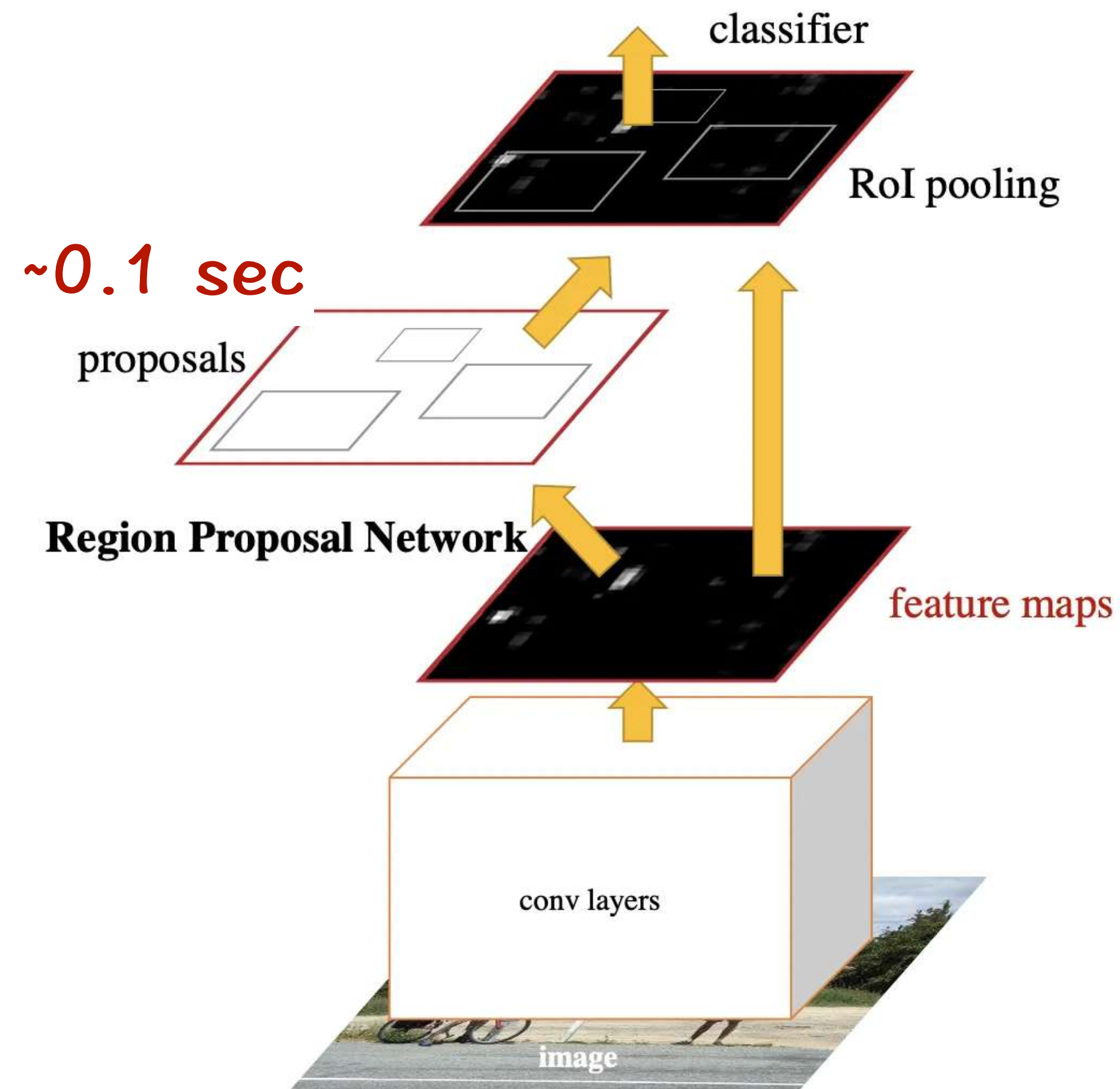


Selective Search ~2 sec

- start with small image regions and repeatedly merges neighboring regions based on color, texture, size, and how well they fit together.
- convert regions at different stages into candidate bounding boxes, which Fast R-CNN then classifies and refines.

Recap: Object Detection Summary

Faster R-CNN



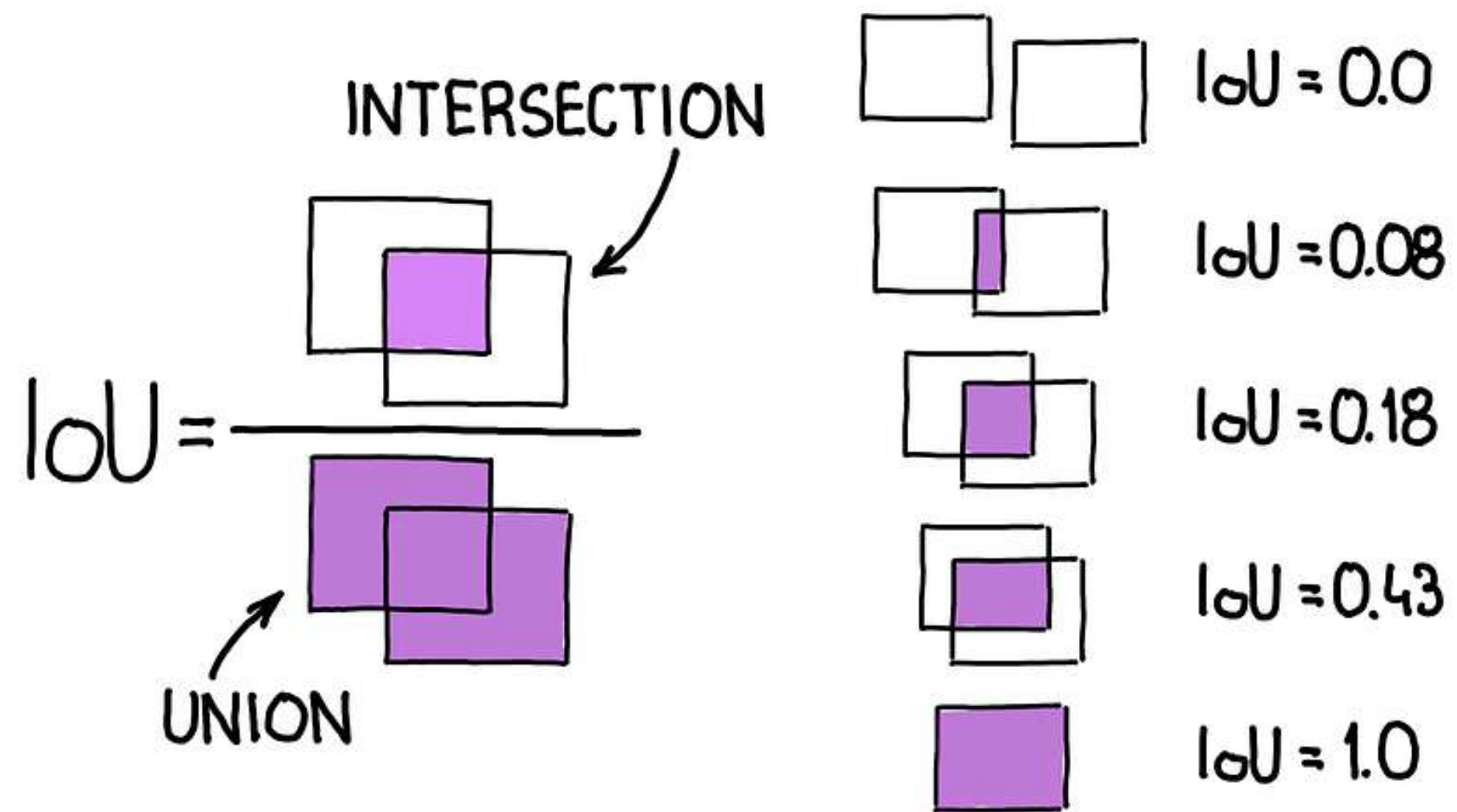
Region Proposal Network ~0.1 sec

- scans a shared convolutional feature map and predicts which predefined anchor boxes are likely to contain objects
- adjusts their coordinates, producing candidate regions for the detector to classify and refine

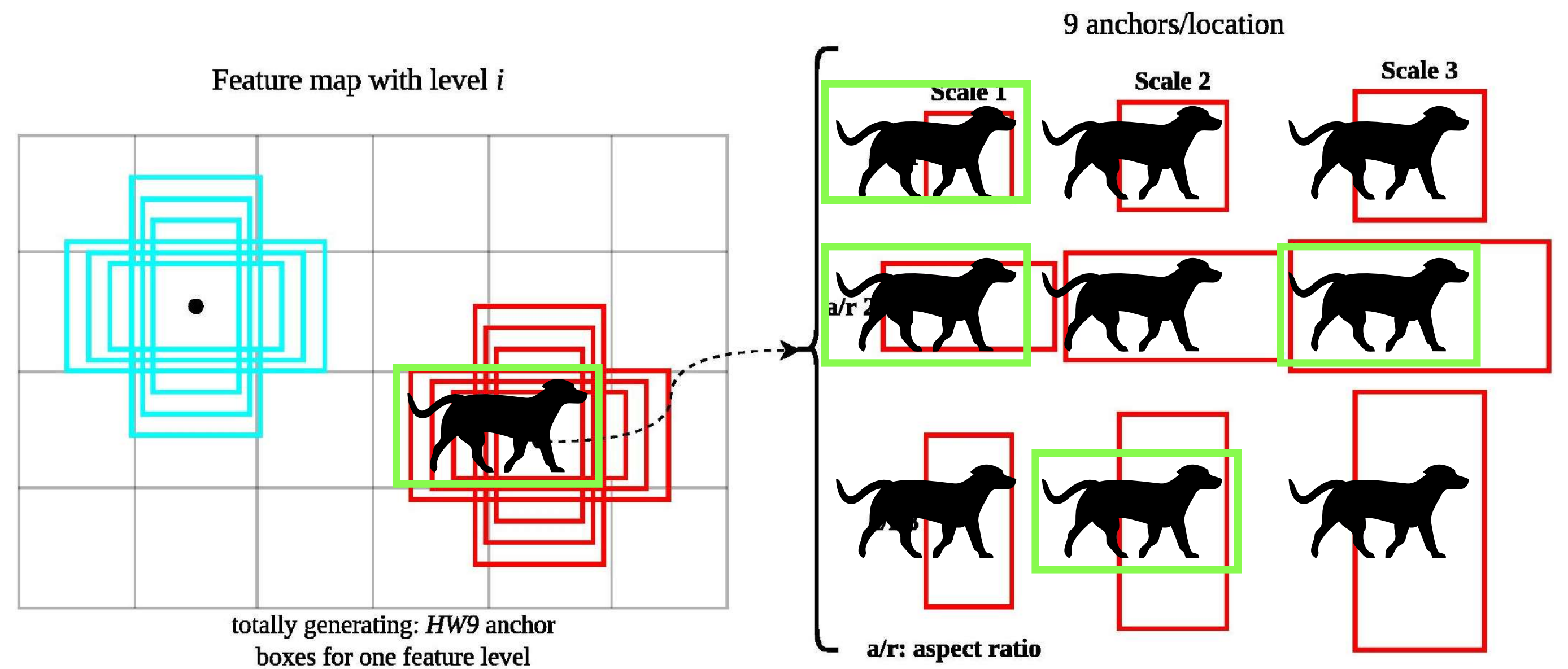
Today's Agenda

- Two-stage Object Detection & Instance Segmentation
- One-stage Object Detection & Instance Segmentation
- Visualization & Understanding

IoU (Intersection of Union)



Anchor & Bounding box



- Different scales
- Various aspect ratios

Bounding Box Regression: Multi-Scale Anchors as References

- Anchor central position, width & height:

$$(x_c^a, y_c^a, w^a, h^a)$$

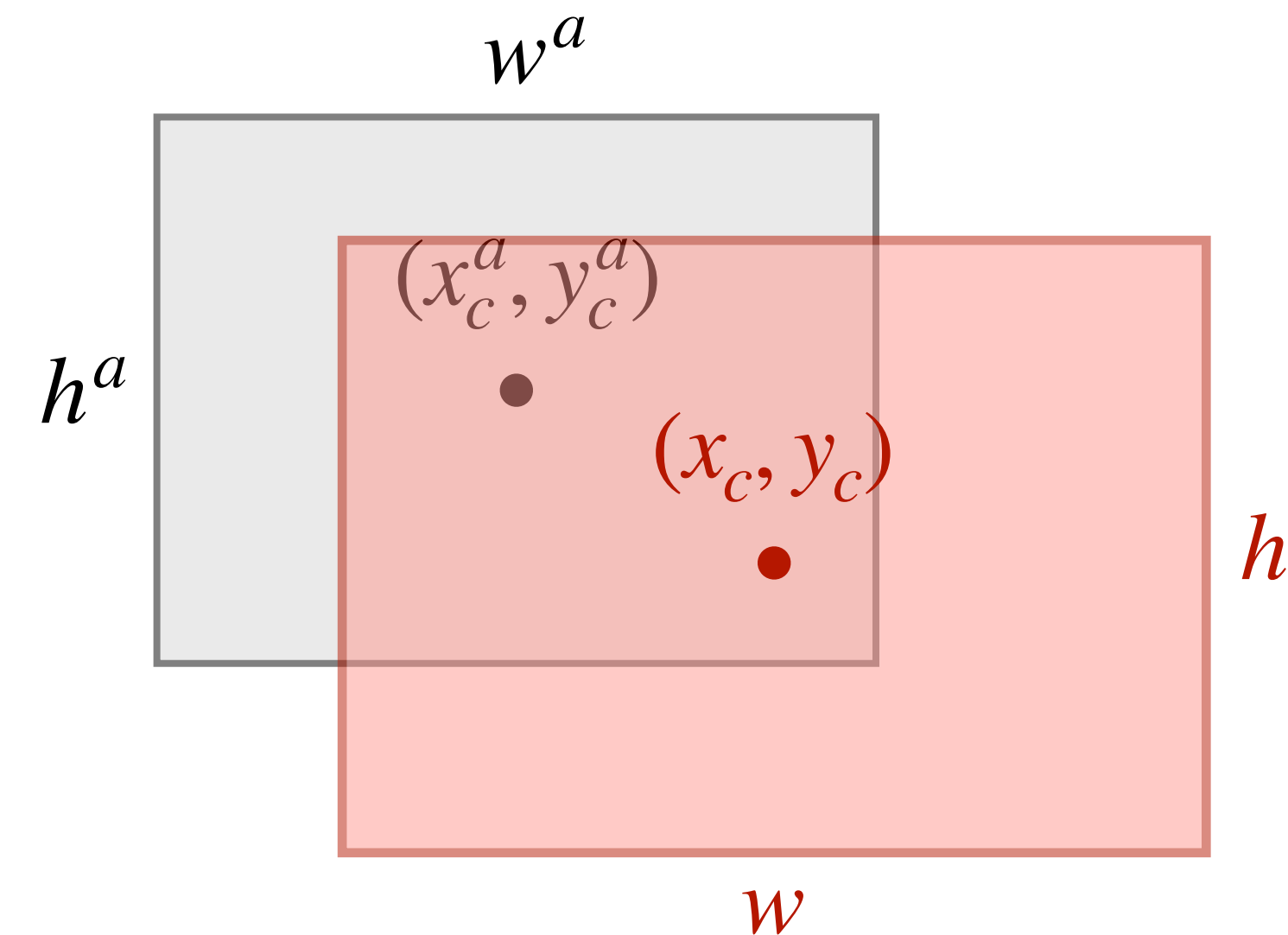
- Ground-truth bounding box:

$$(x_c, y_c, w, h)$$

- Predicted bounding box offsets:

$$(\Delta x, \Delta y, \Delta w, \Delta h)$$

Q: How can we calculate offsets?



$$\Delta w = \log\left(\frac{w}{w^a}\right) \quad \Delta x = \frac{(x_c - x_c^a)}{w^a}$$

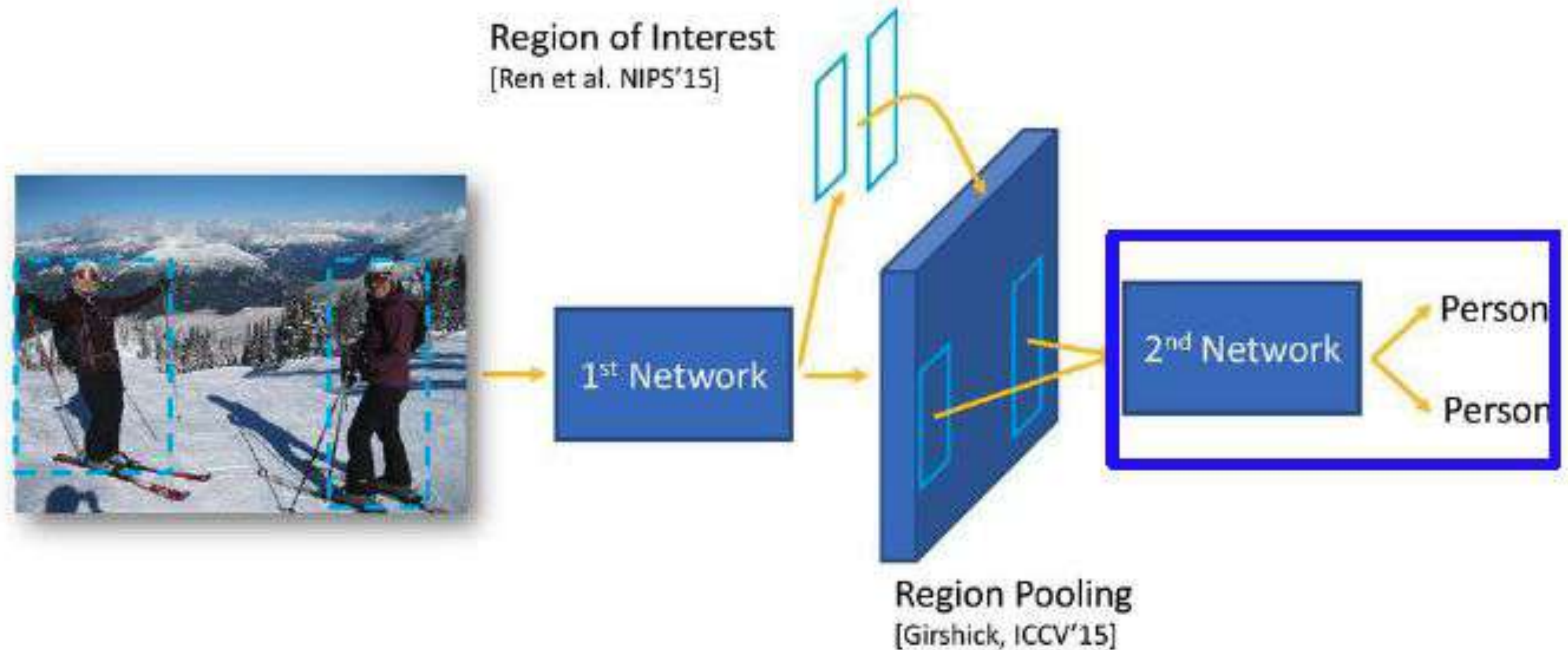
$$\Delta h = \log\left(\frac{h}{h^a}\right) \quad \Delta y = \frac{(y_c - y_c^a)}{h^a}$$

Two-stage Object Detection & Segmentation

Faster R-CNN v.s. Mask R-CNN

Details for Two-Stage Object Detectors

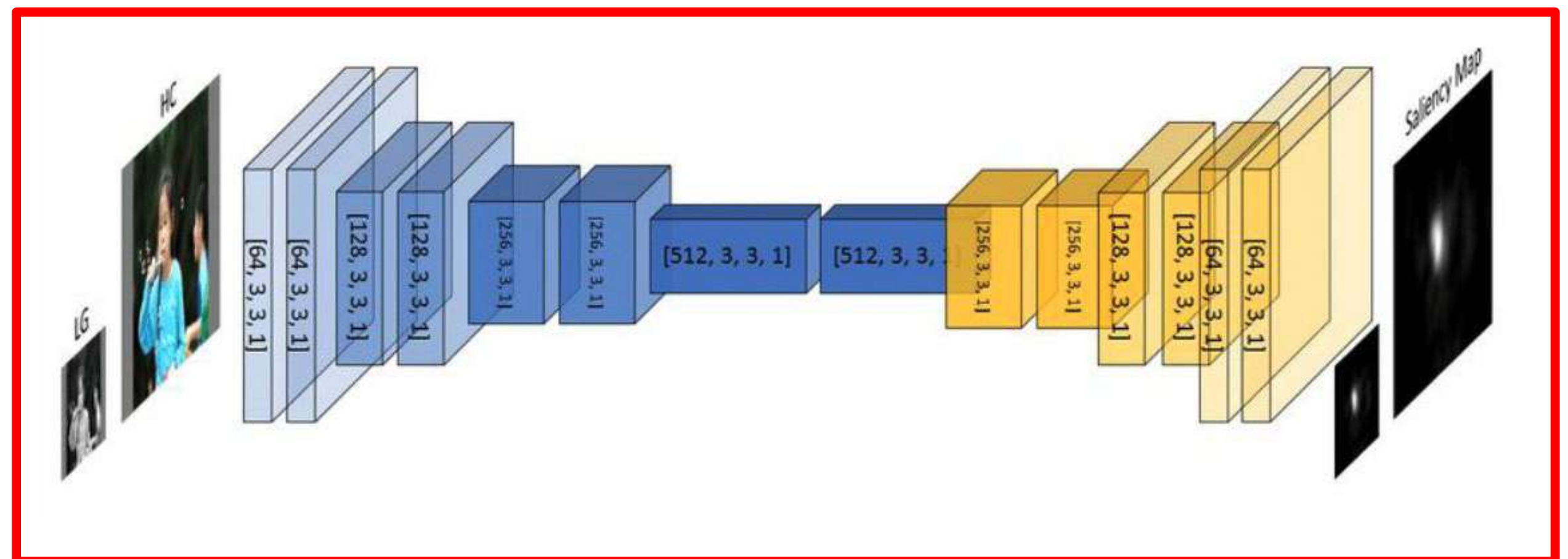
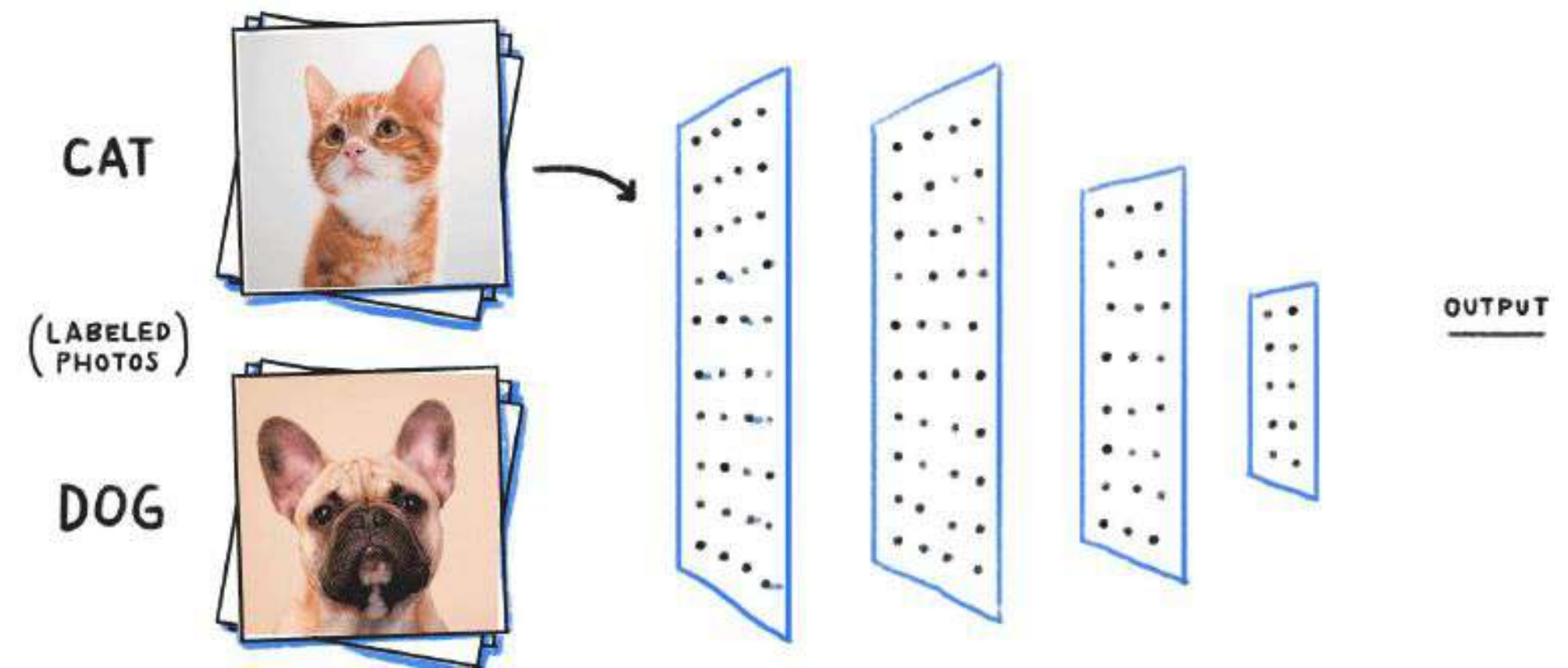
Stage 1: Region Proposal Network (RPN) (~0.1 sec)



Stage 2: RoI Pooling (~0.3 sec)

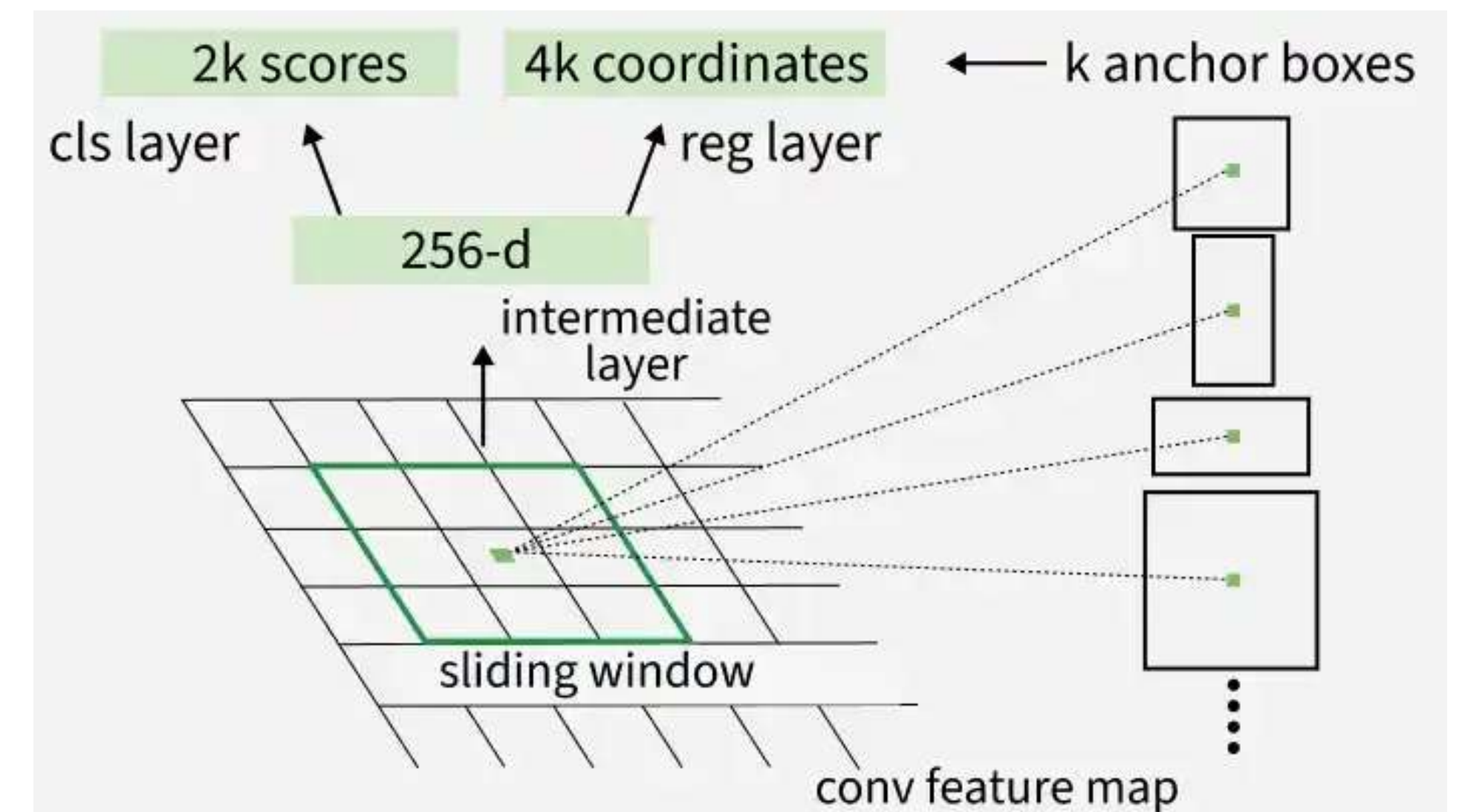
Feature extractor

- Every pixel makes prediction
- Image classification: single output



Extract Anchor Boxes

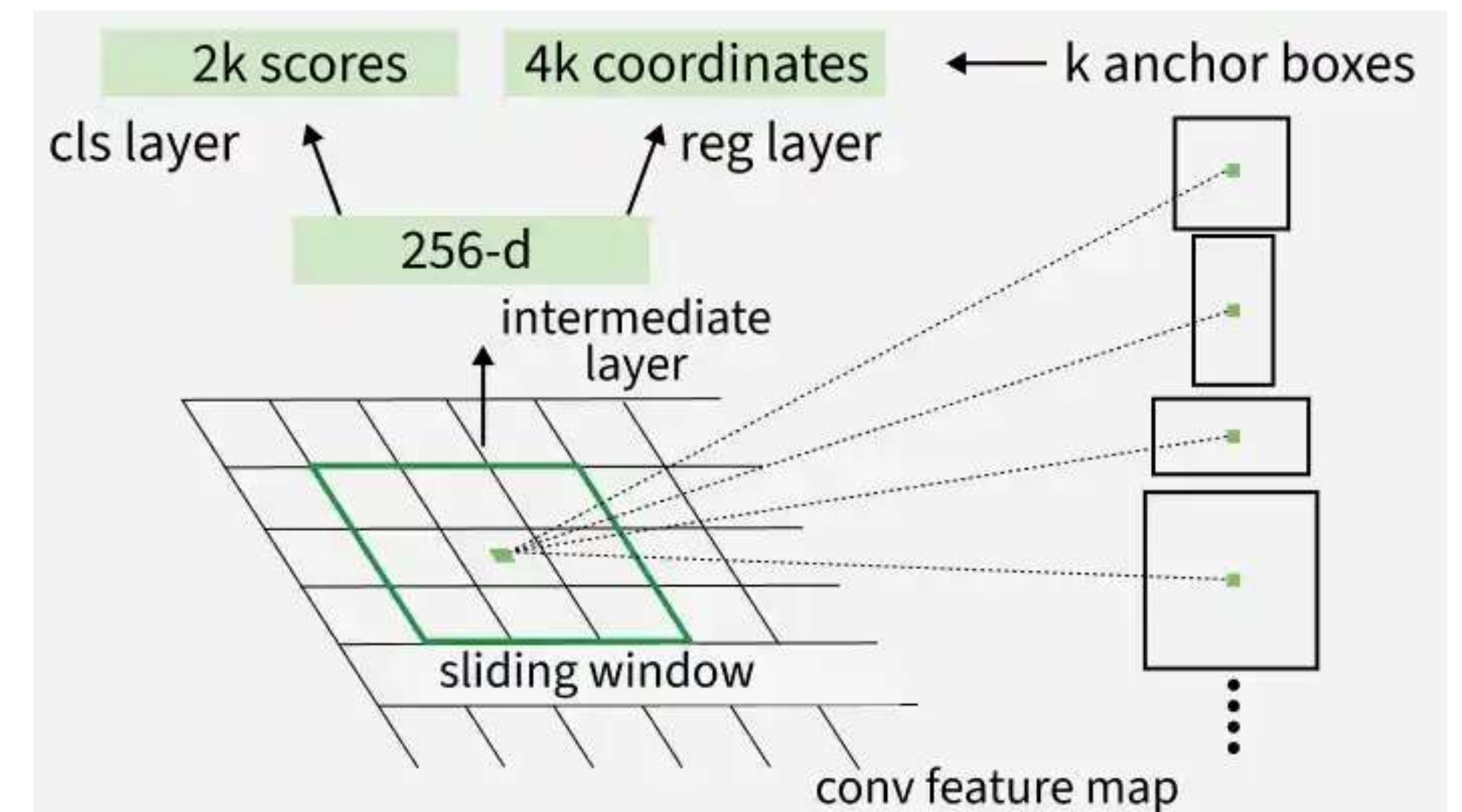
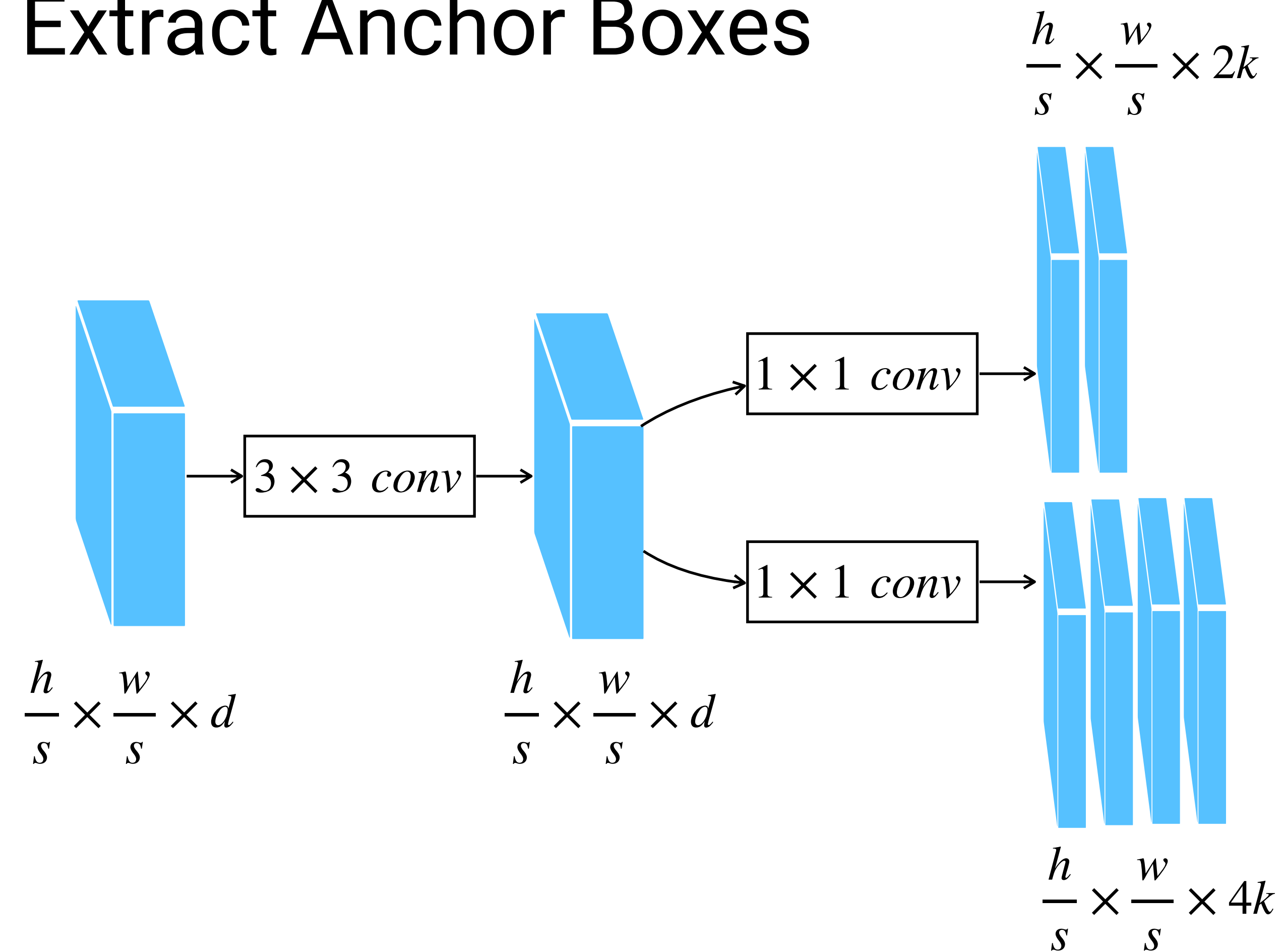
- For each output pixel
 - "Objectness" classification
 - Regression
- Often thousands of anchors for an image
- Pass anchors that correspond to ground-truth locations to the next stage, plus negative anchors



https://github.com/facebookresearch/Detectron/blob/main/detectron/modeling/generate_anchors.py

https://github.com/facebookresearch/Detectron/blob/main/detectron/modeling/rpn_heads.py

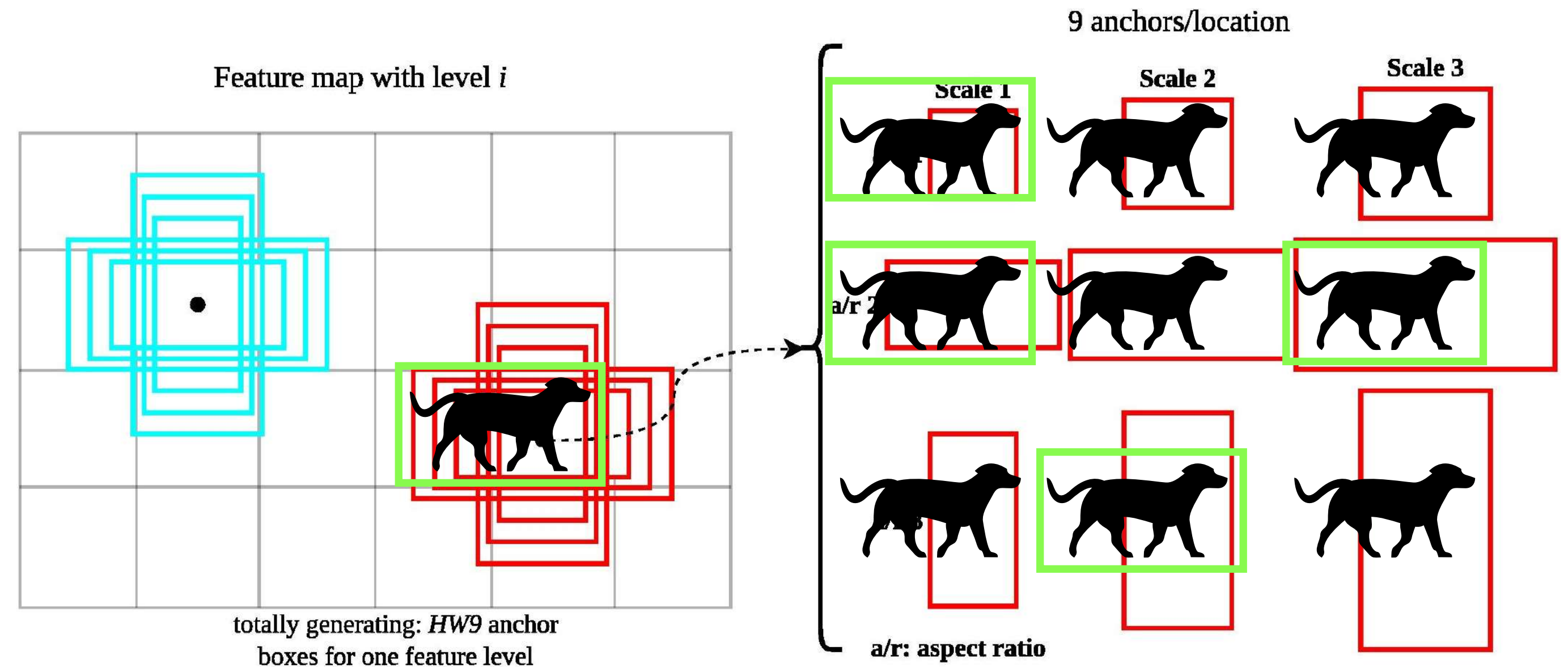
Extract Anchor Boxes



https://github.com/facebookresearch/Detectron/blob/main/detectron/modeling/generate_anchors.py

https://github.com/facebookresearch/Detectron/blob/main/detectron/modeling/rpn_heads.py

RPN Loss Function



Sign a positive label to two kinds of anchors:

- The anchor with the highest IoU overlap with a GT box
- An anchor that has an IoU overlap higher than 0.7 with any GT box

Note that: 1 GT box may assign positive labels to multiple anchors.

Sign a negative label:

- IoU is lower than 0.3 for all GT boxes.

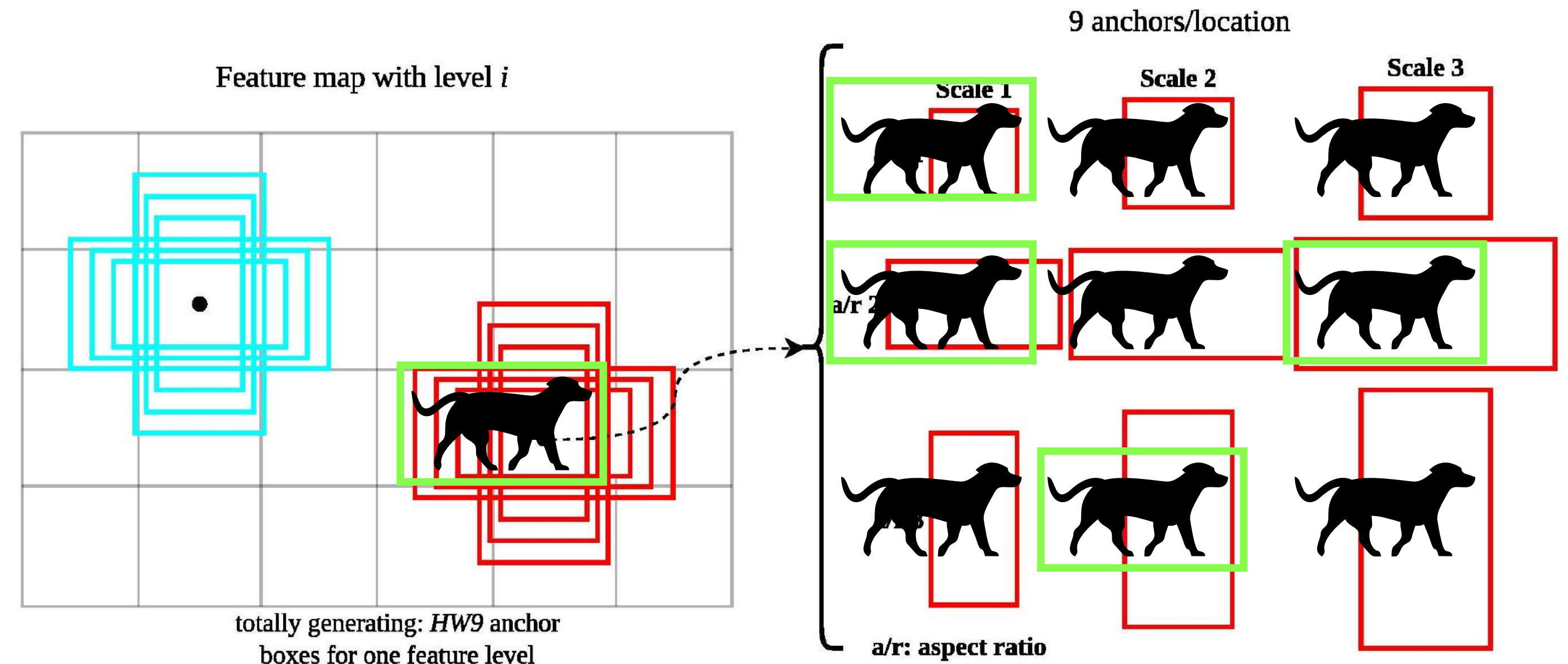
RPN Loss Function

Binary Cross-Entropy (BCE) Loss

$$L = \frac{1}{N_{cls}} \sum_i L_{cls}(\hat{p}_i, p_i)$$

$$+ \lambda \frac{1}{N_{reg}} \sum_i L_{reg}(\Delta \hat{b}_i, \Delta b_i)$$

L1, Smooth L1, L2, ...



Sign a positive label to two kinds of anchors:

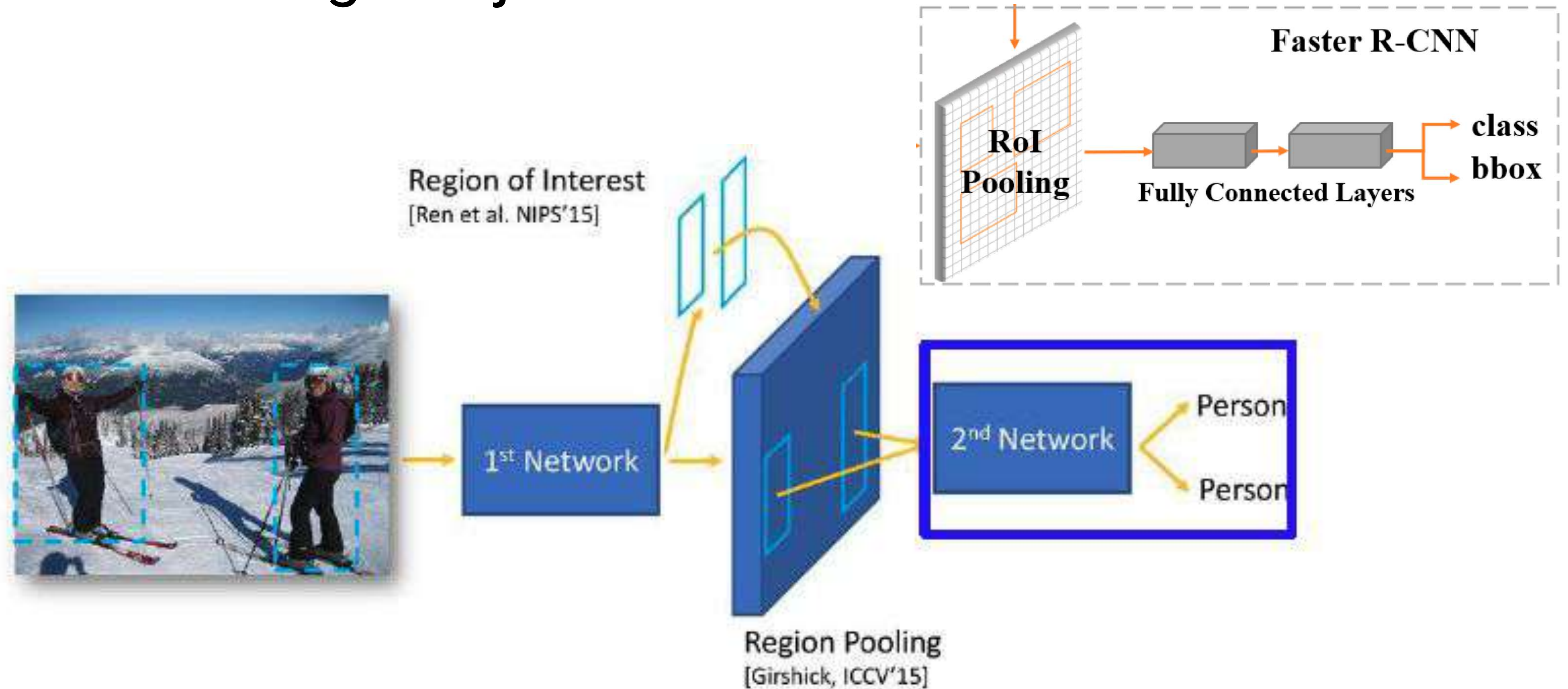
- The anchor with the highest IoU overlap with a GT box
- An anchor that has an IoU overlap higher than 0.7 with any GT box

Note that: 1 GT box may assign positive labels to multiple anchors.

Sign a negative label:

- IoU is lower than 0.3 for all GT boxes.

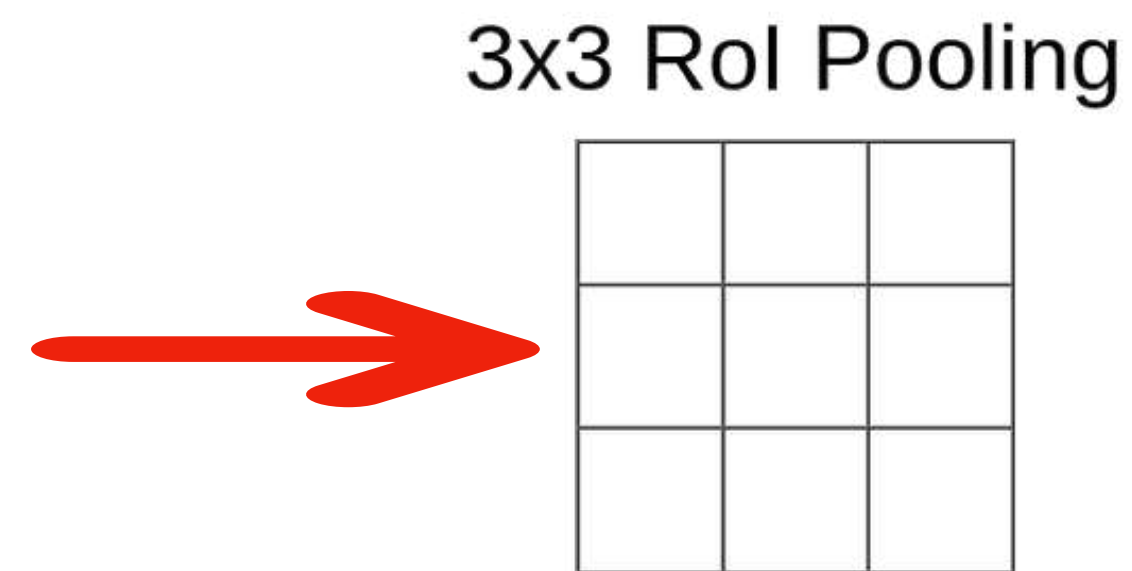
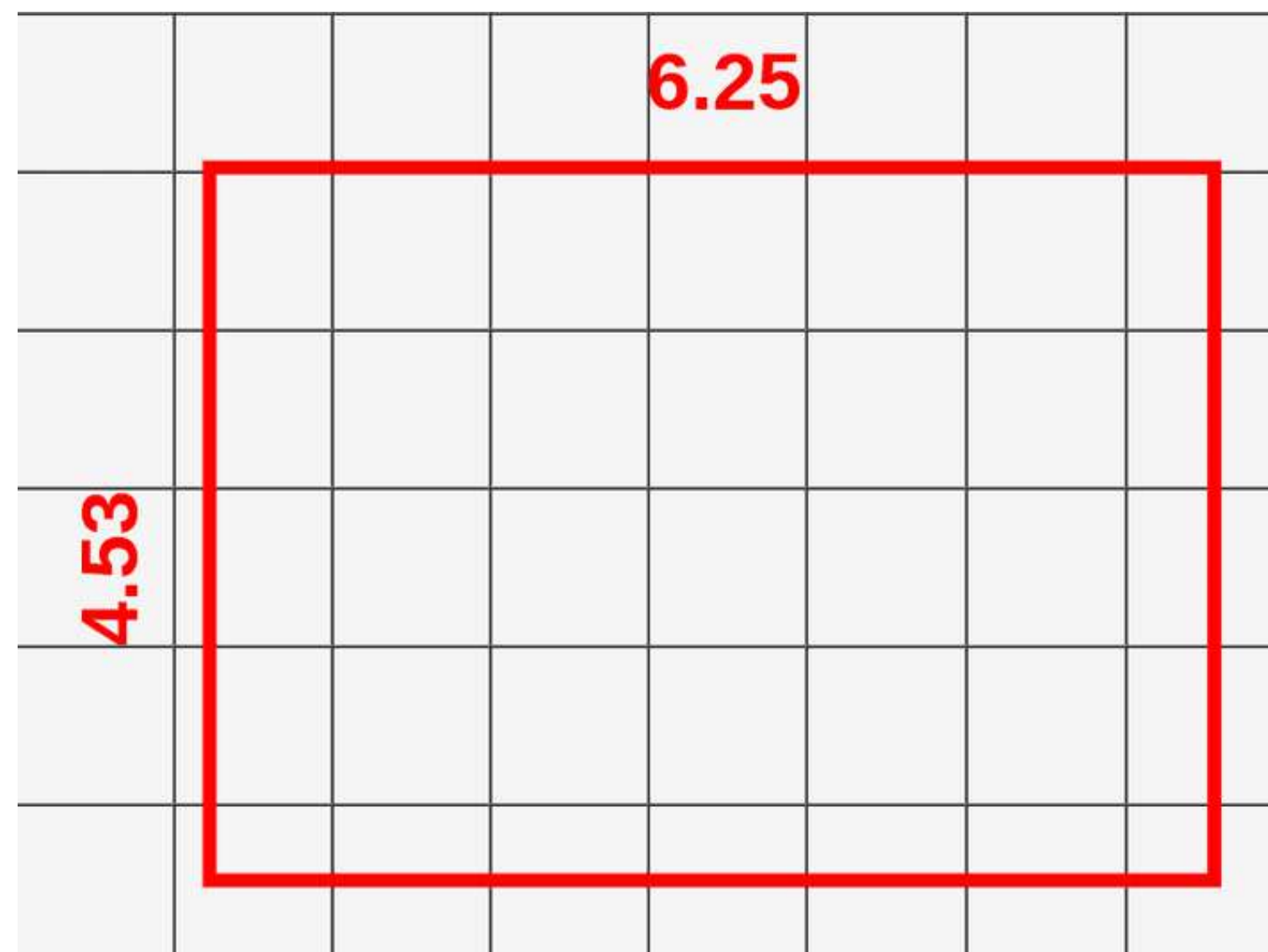
Details for Two-Stage Object Detectors



turn a variable-sized region of interest
(RoI) into a fixed-size feature map

RoI Pooling v.s. RoI Align

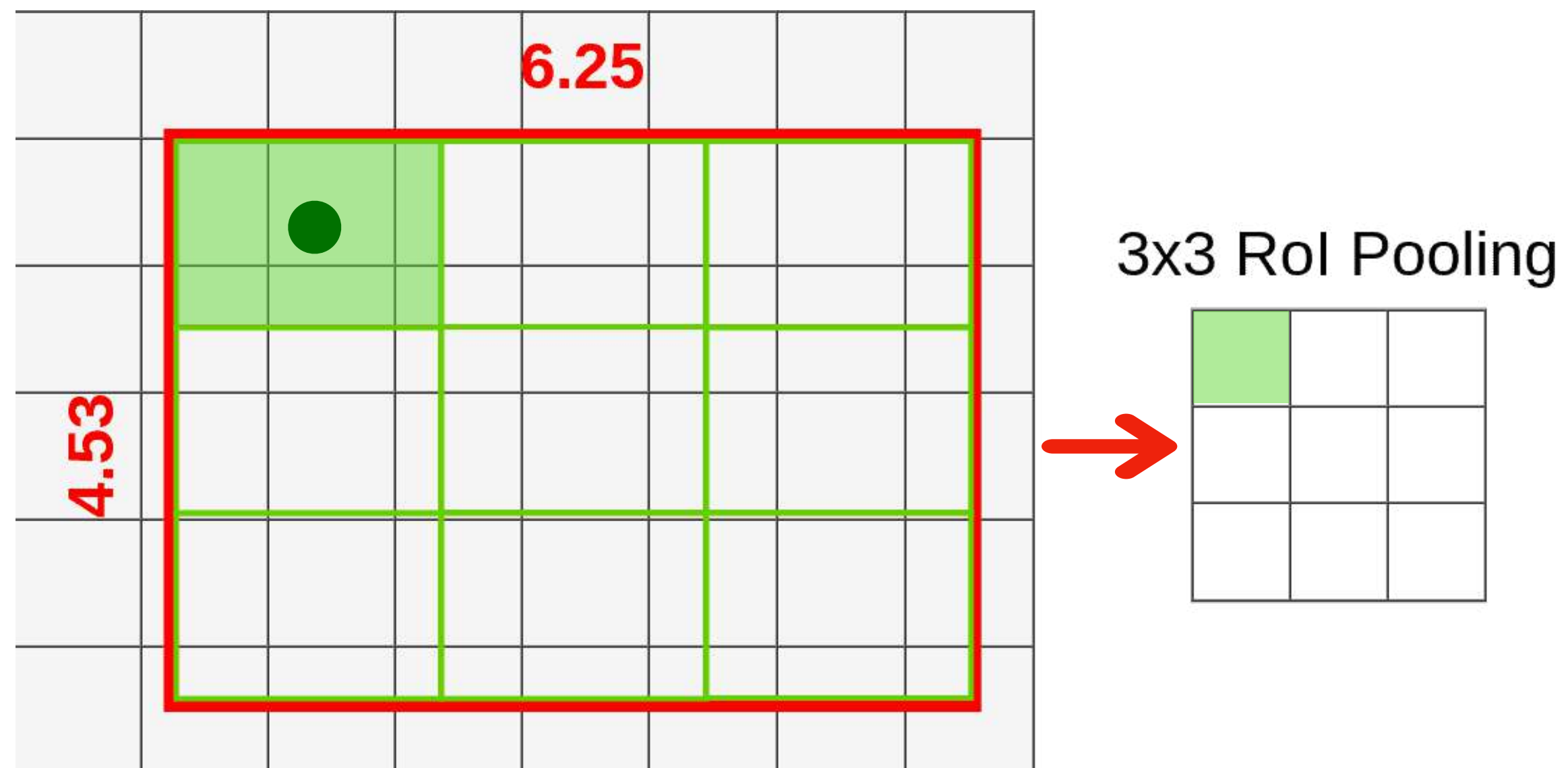
- Given region-of-interests (Rois), we want to pool from the backbone features



turn a variable-sized region of interest (RoI) into a fixed-size feature map

Rol Pooling v.s. Rol Align

- Given region-of-interests (Rols), we want to pool from the backbone features



- dividing original Rol into 9 equal size boxes

RoI Align

- Keeps fractional coordinates and uses bilinear interpolation to sample each bin



$$P \approx \frac{7.5 - 6.5}{7.5 - 6.5} \left(\frac{10.5 - 9.94}{10.5 - 9.5} 0.1 + \frac{9.94 - 9.5}{10.5 - 9.5} 0.2 \right) + \frac{6.5 - 6.5}{7.5 - 6.5} \left(\frac{10.5 - 9.94}{10.5 - 9.5} 1 + \frac{9.94 - 9.5}{10.5 - 9.5} 0.7 \right)$$

6.25

4.53



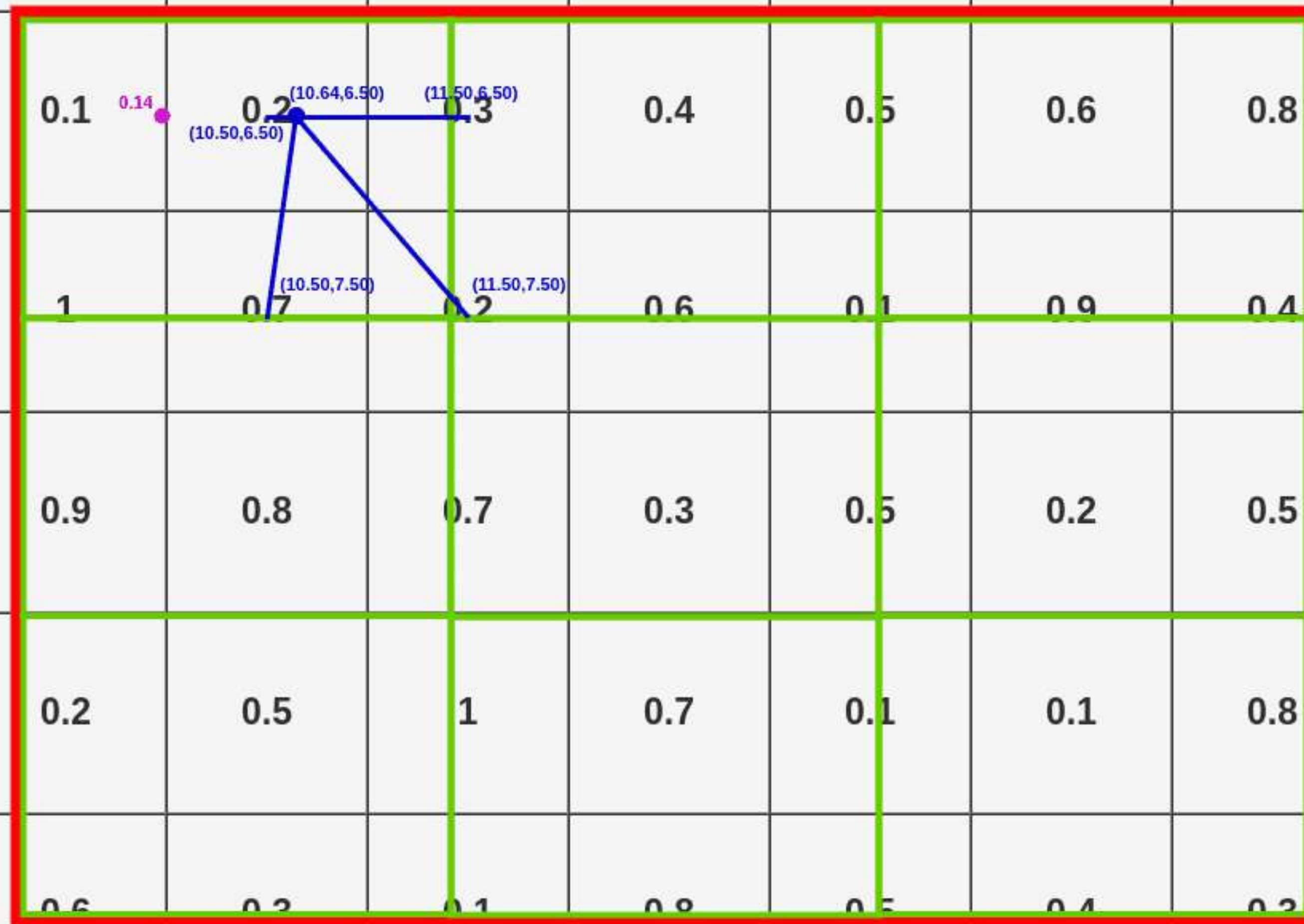
RoI Align

- Keeps fractional coordinates and uses bilinear interpolation to sample each bin

$$P \approx \frac{7.5 - 6.5}{7.5 - 6.5} \left(\frac{11.5 - 10.64}{11.5 - 10.5} 0.2 + \frac{10.64 - 10.5}{11.5 - 10.5} 0.3 \right) + \frac{6.5 - 6.5}{7.5 - 6.5} \left(\frac{11.5 - 10.64}{11.5 - 10.5} 0.7 + \frac{10.64 - 10.5}{11.5 - 10.5} 0.2 \right)$$

6.25

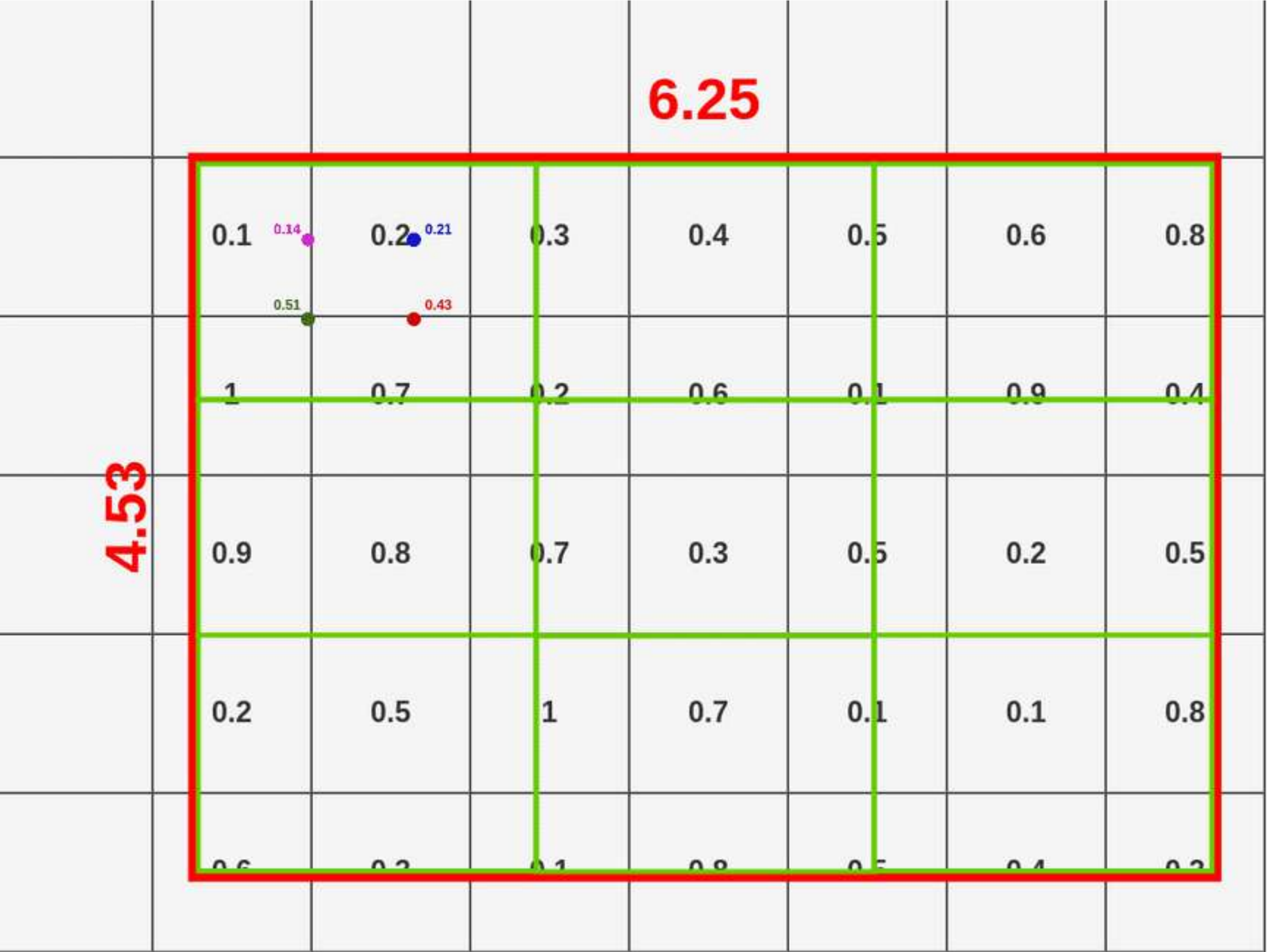
4.53



RoI Align

- Keeps fractional coordinates and uses bilinear interpolation to sample each bin

RoI Align



- Keeps fractional coordinates and uses bilinear interpolation to sample each bin

$1 \times 1 = \text{MAX}(0.14, 0.21, 0.51, 0.43) = 0.51$

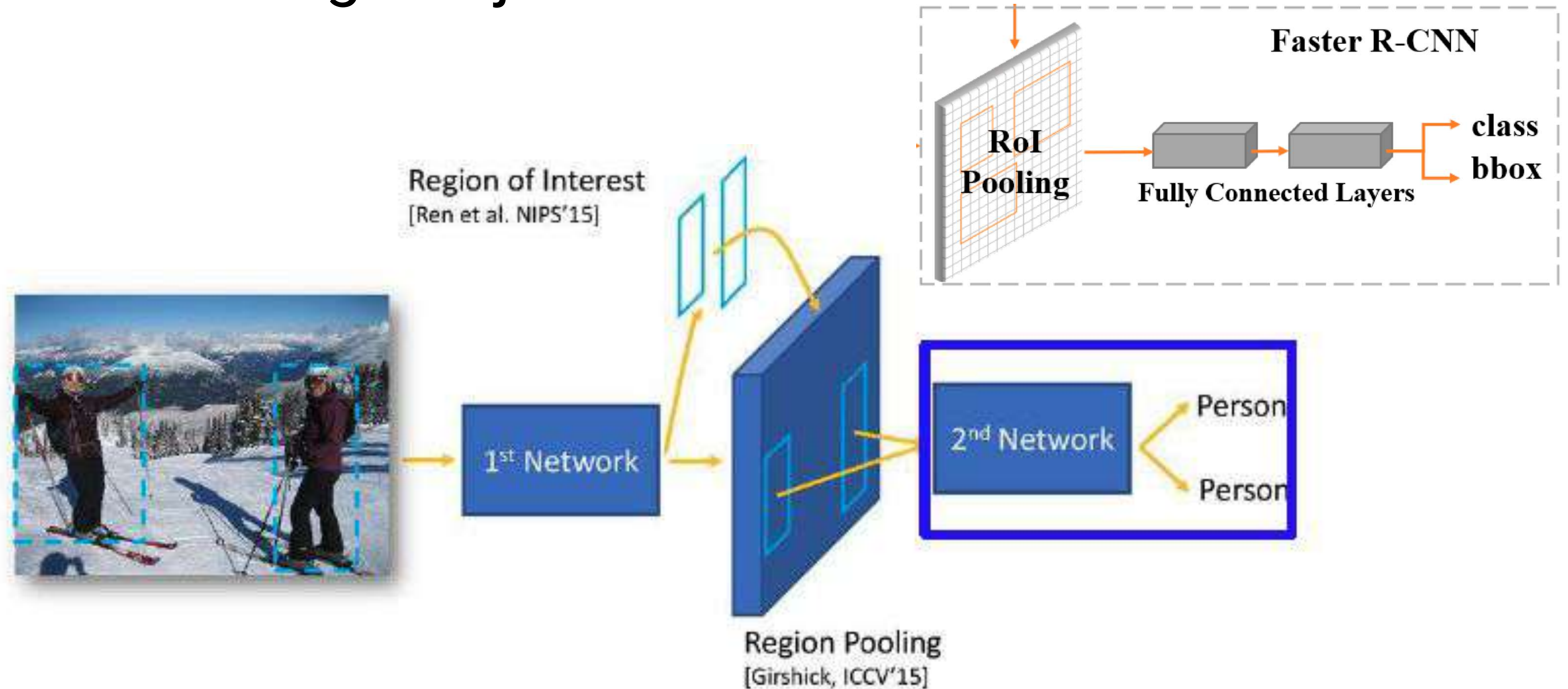
3x3 RoIAlign

0.51		

RoI Align



Details for Two-Stage Object Detectors



turn a variable-sized region of interest (RoI) into a fixed-size feature map

Instance Segmentation

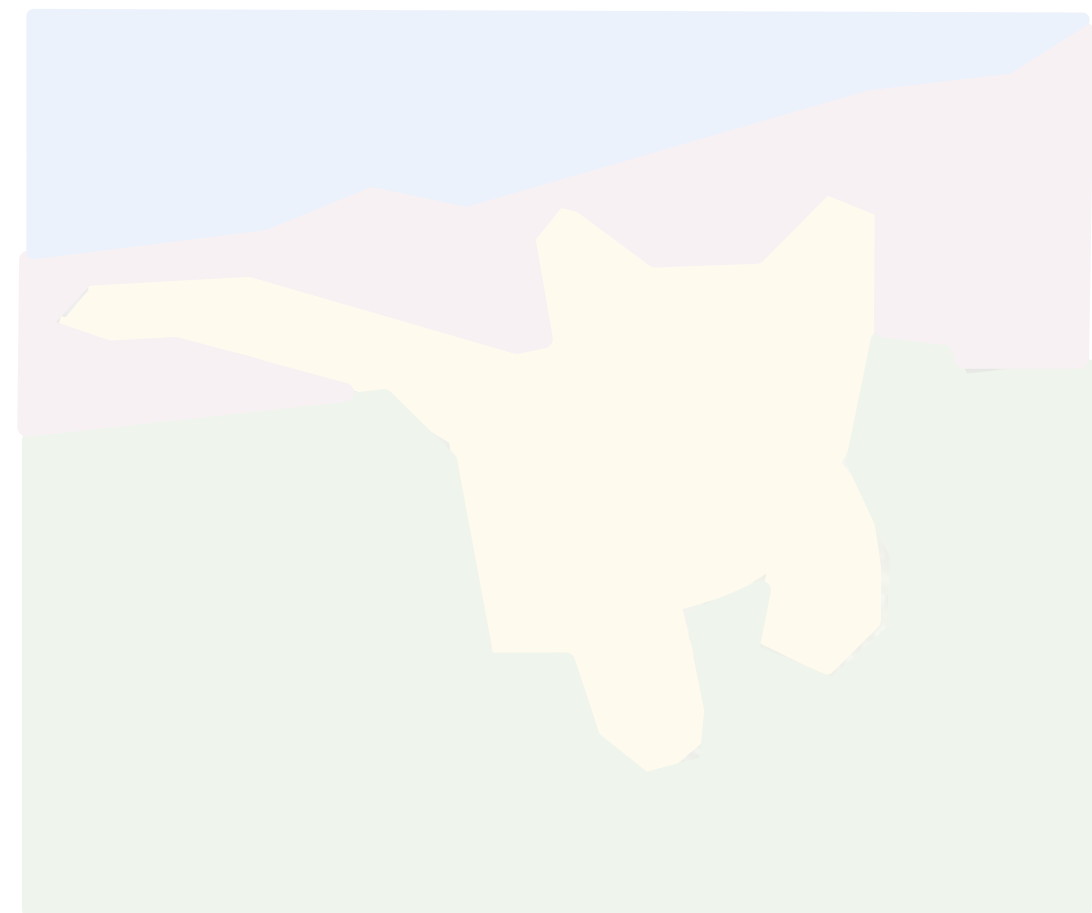
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

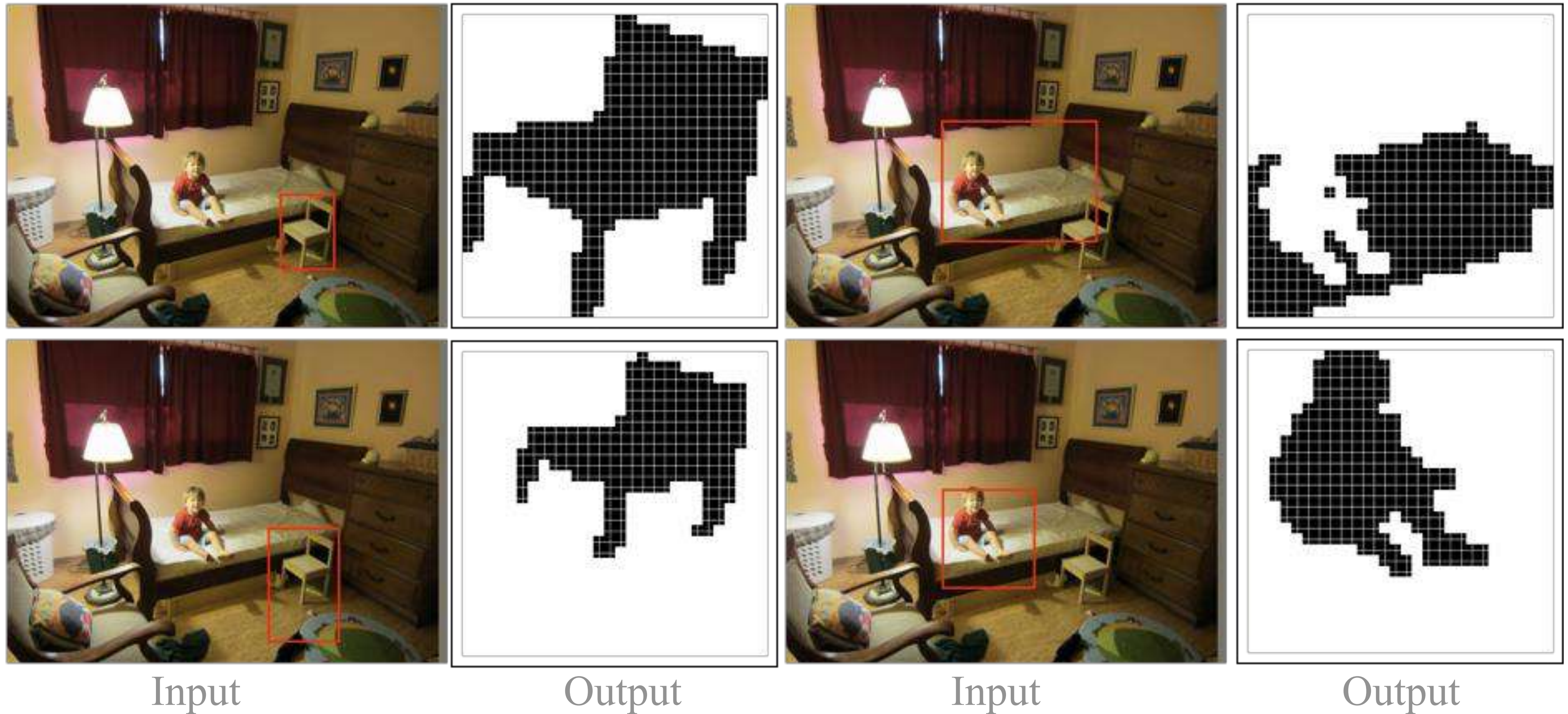
Multiple Object

Instance Segmentation

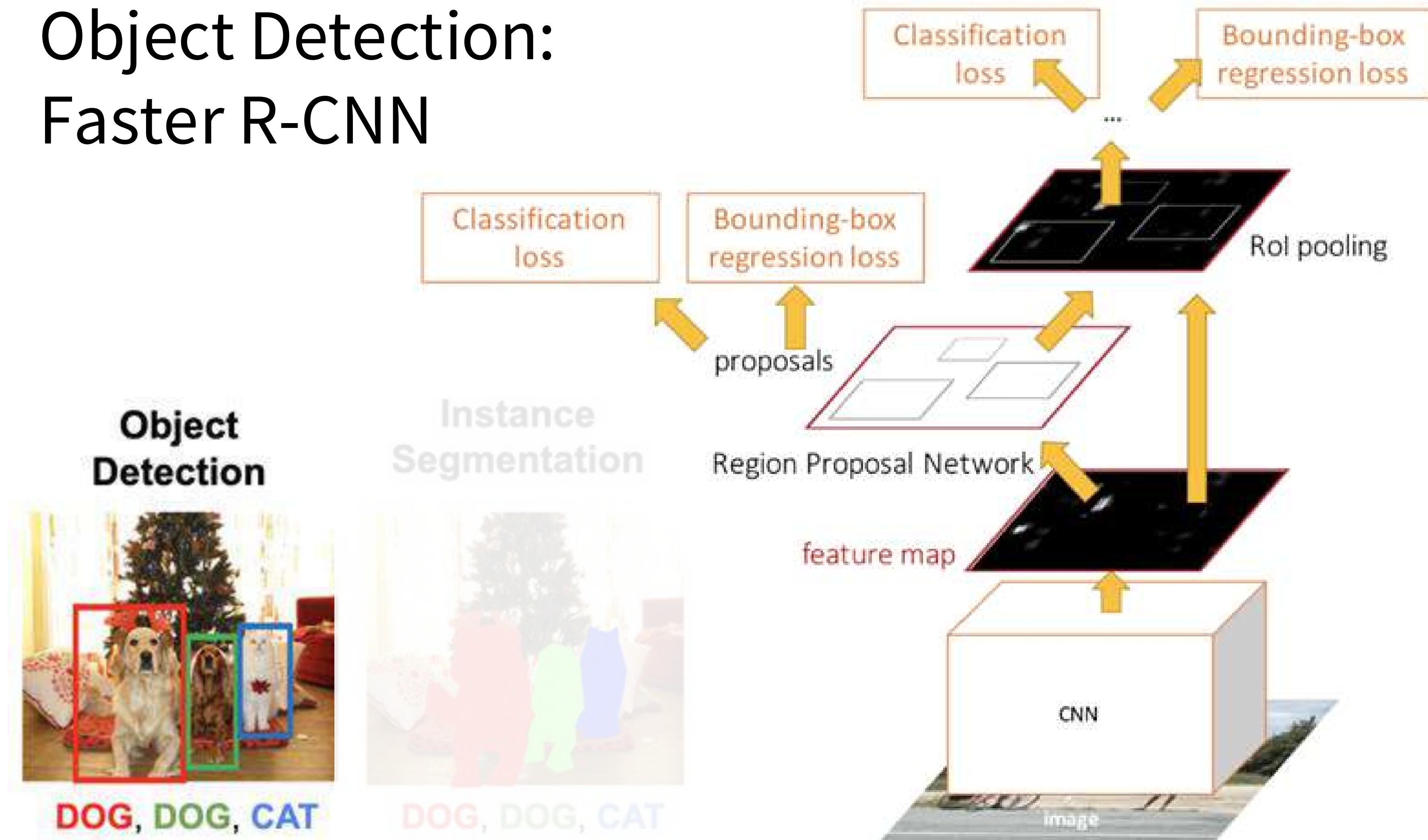


DOG, DOG, CAT

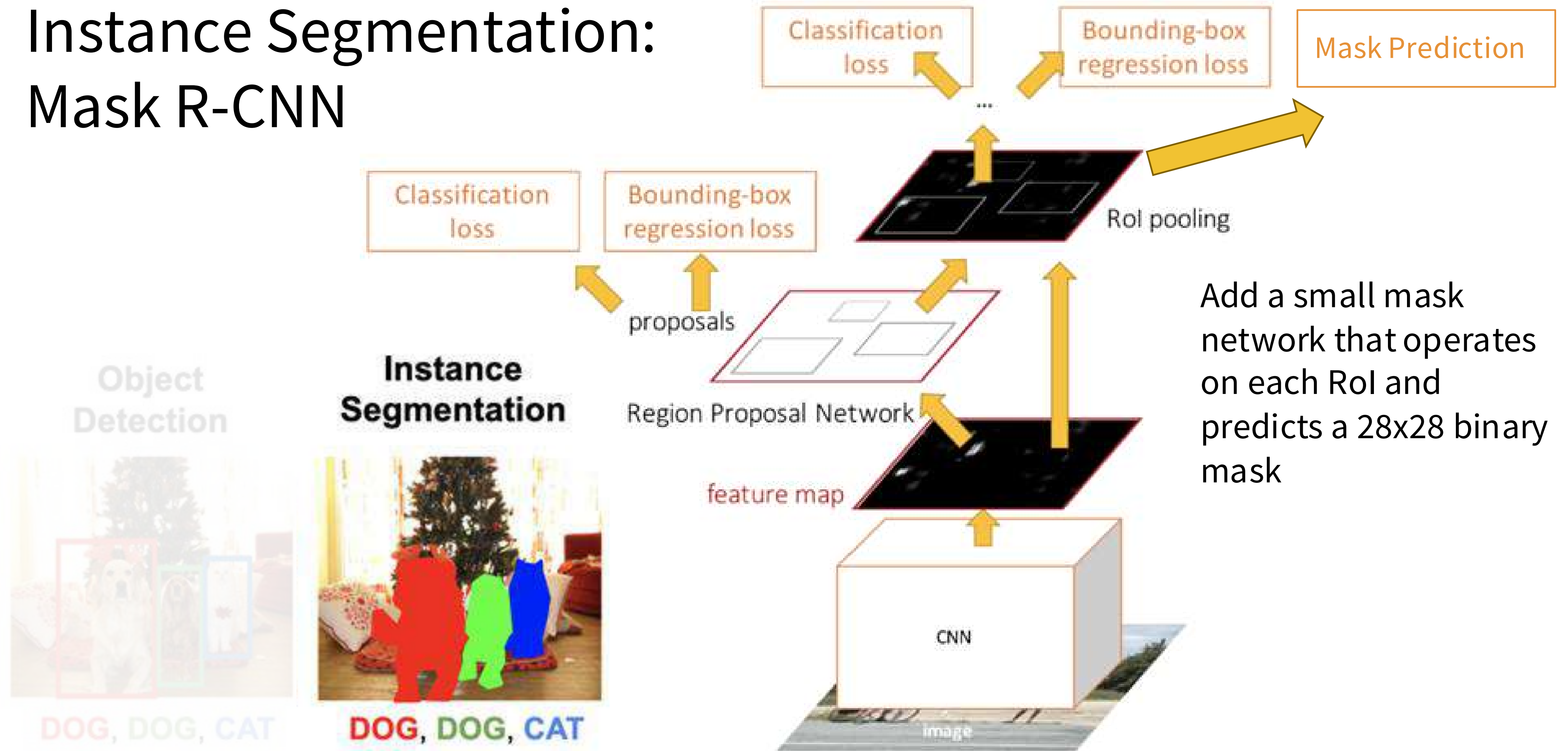
Mask R-CNN: Example Mask Training Targets



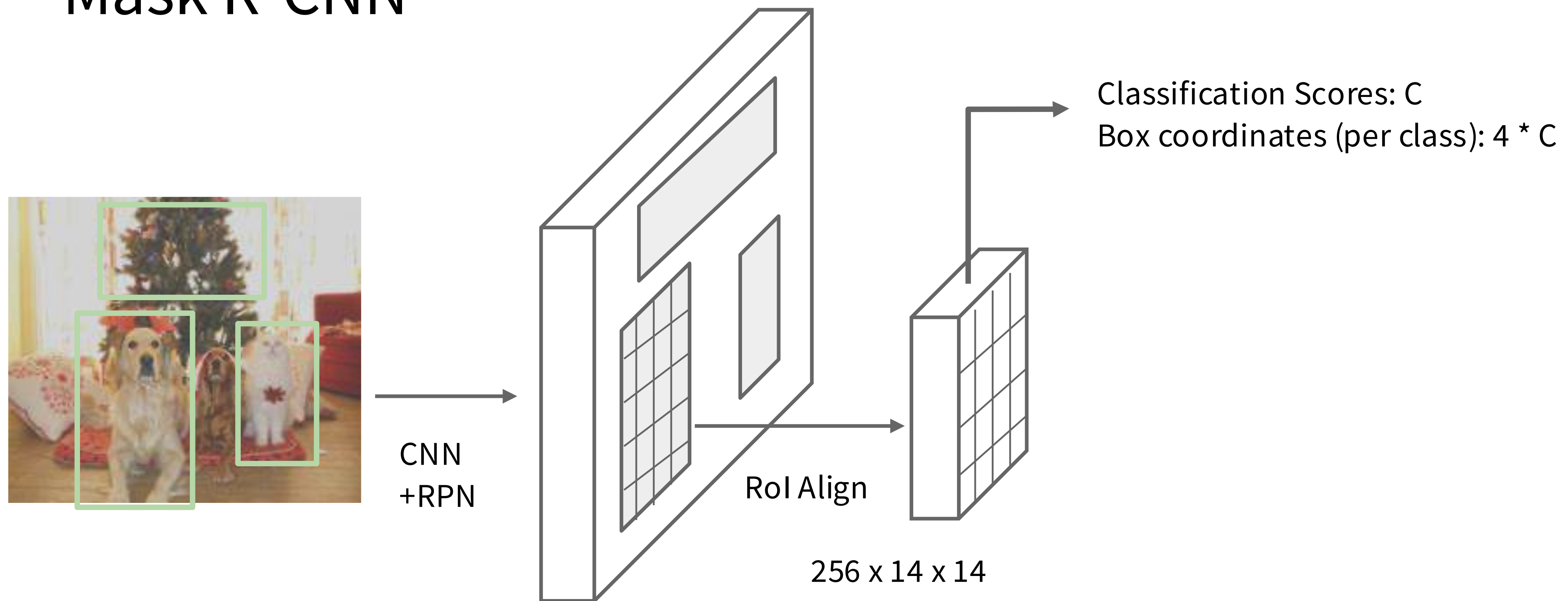
Object Detection: Faster R-CNN



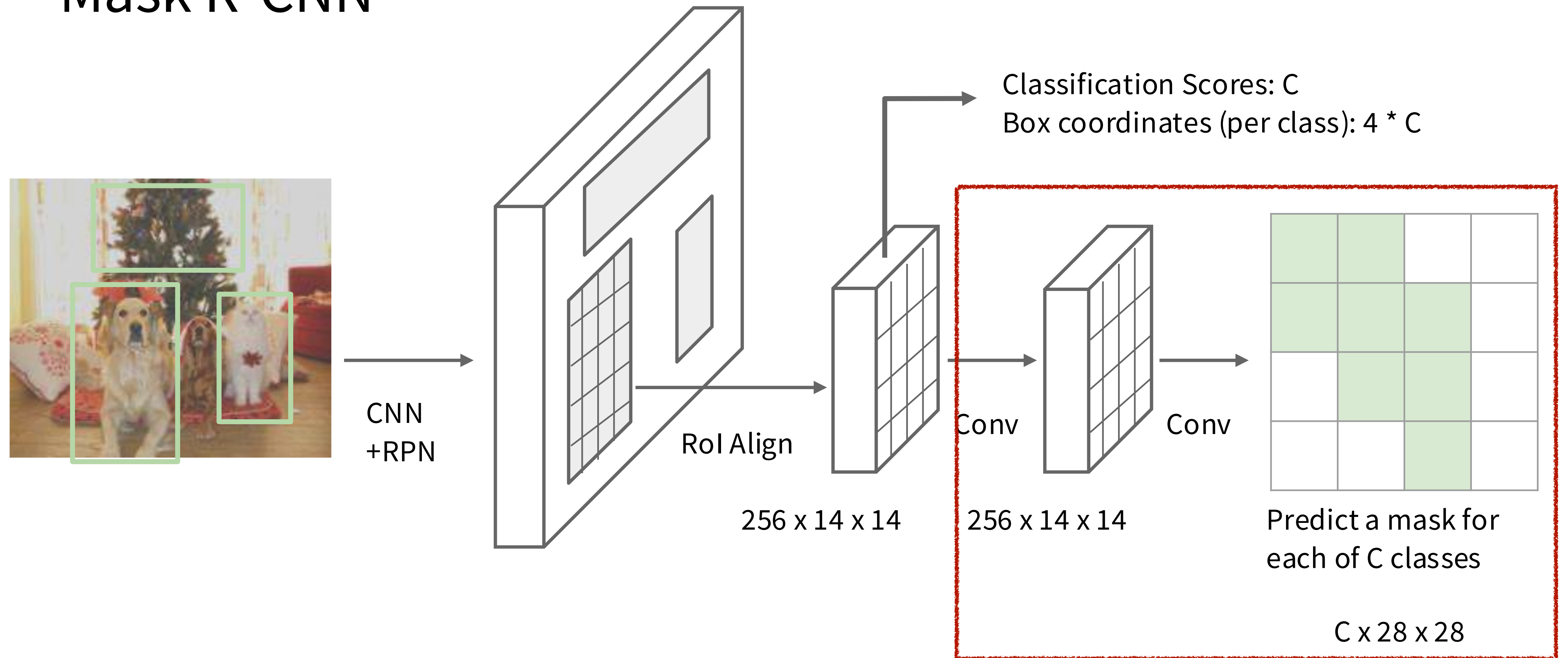
Instance Segmentation: Mask R-CNN



Mask R-CNN



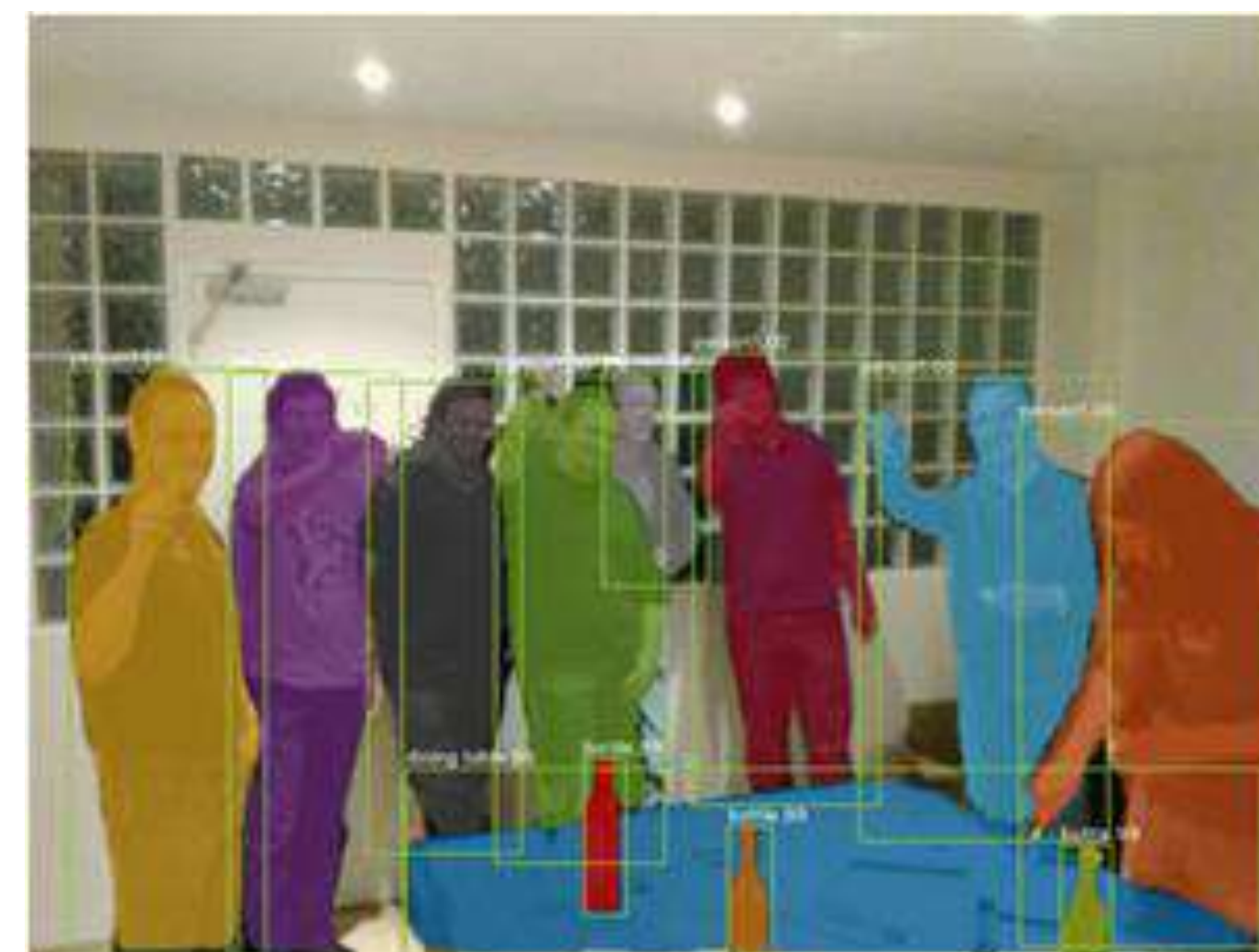
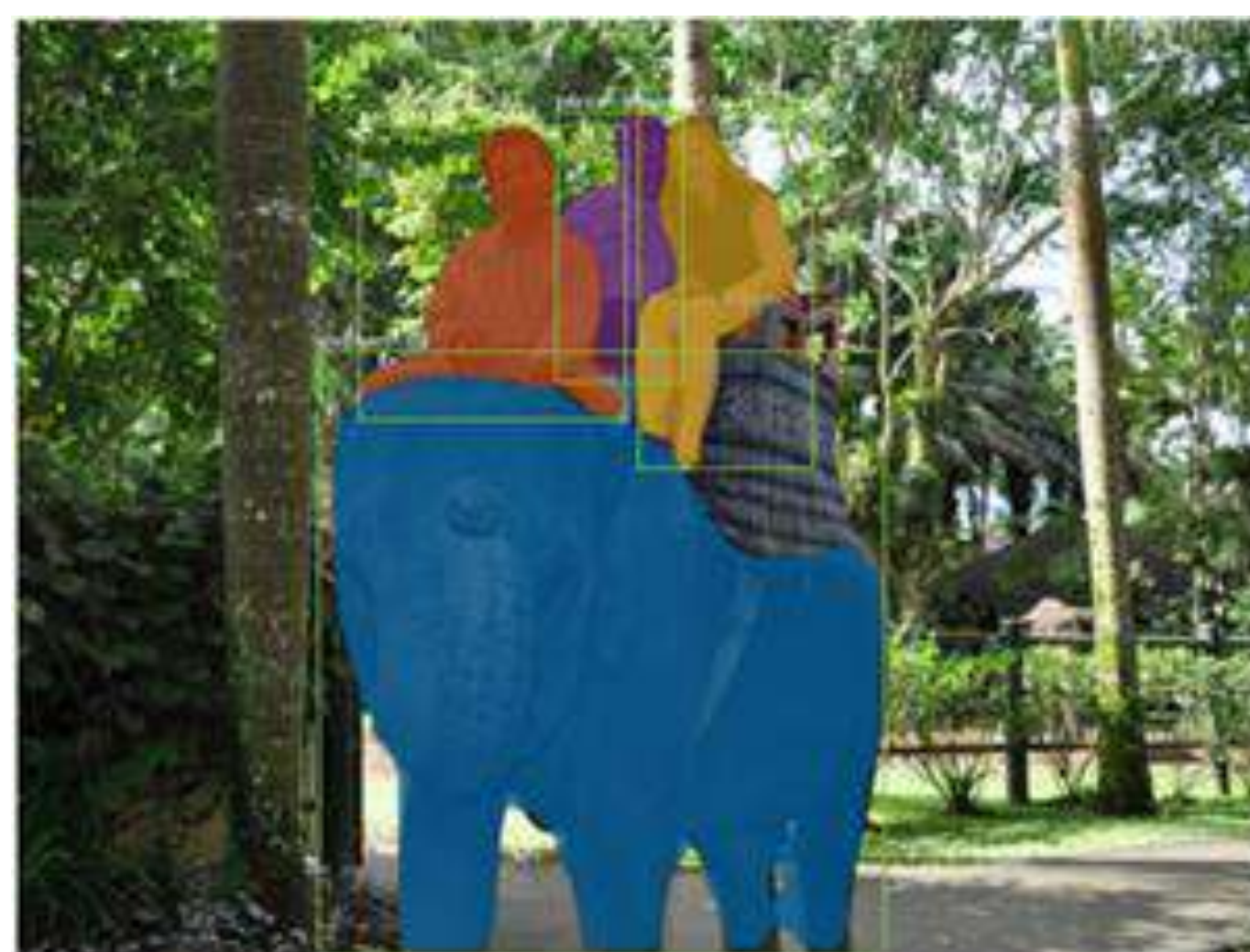
Mask R-CNN



He et al, "Mask R-CNN", arXiv 2017

https://github.com/matterport/mask_RCNN

Mask R-CNN: Very Good Results!



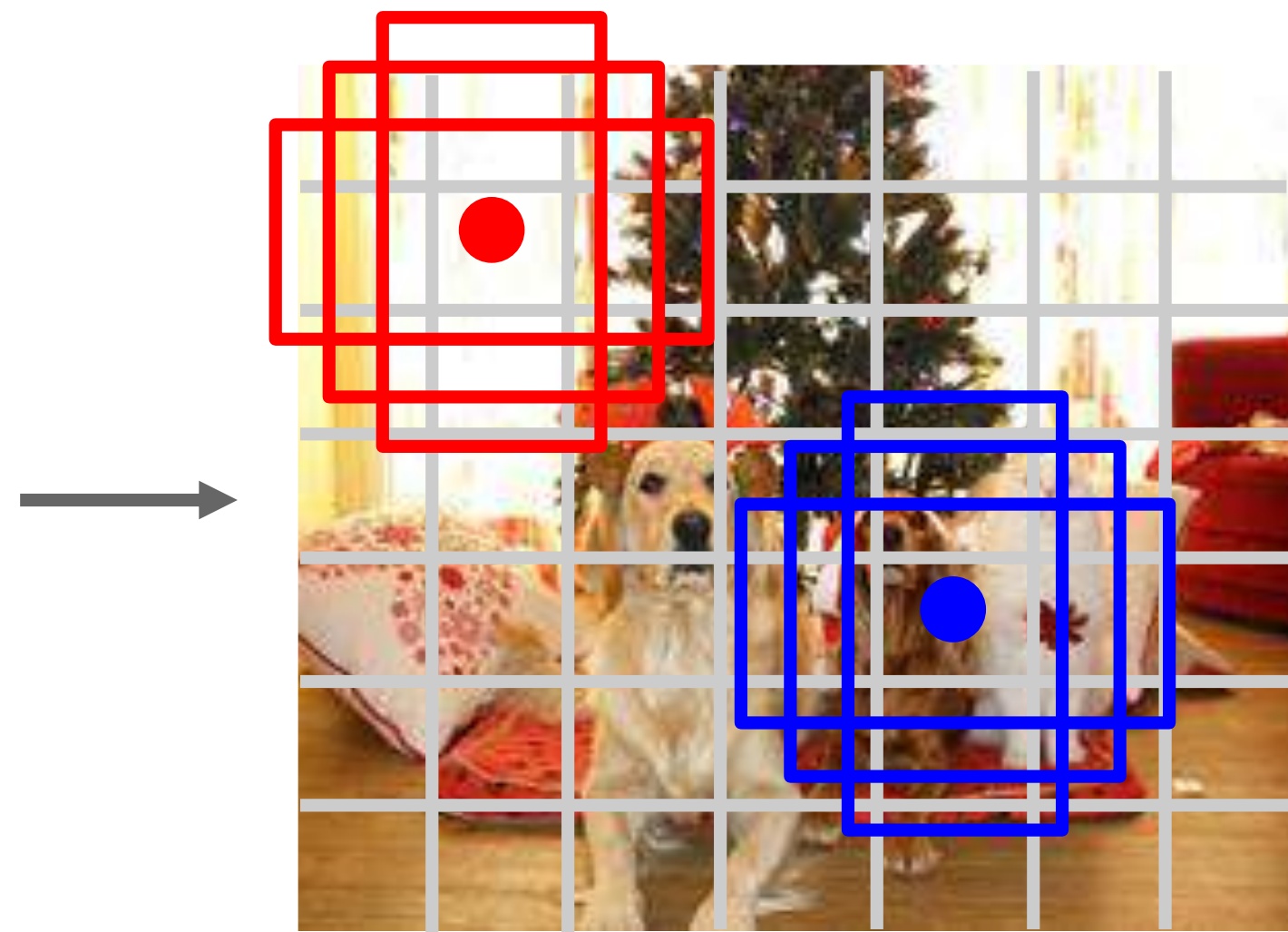
One-stage Object Detection & Segmentation

YOLO v.s. FPN

Single-Stage Object Detectors: YOLO / SSD / RetinaNet



Input image
 $3 \times H \times W$



Divide image into grid
 7×7

Image a set of base boxes
centered at each grid cell
Here $B = 3$

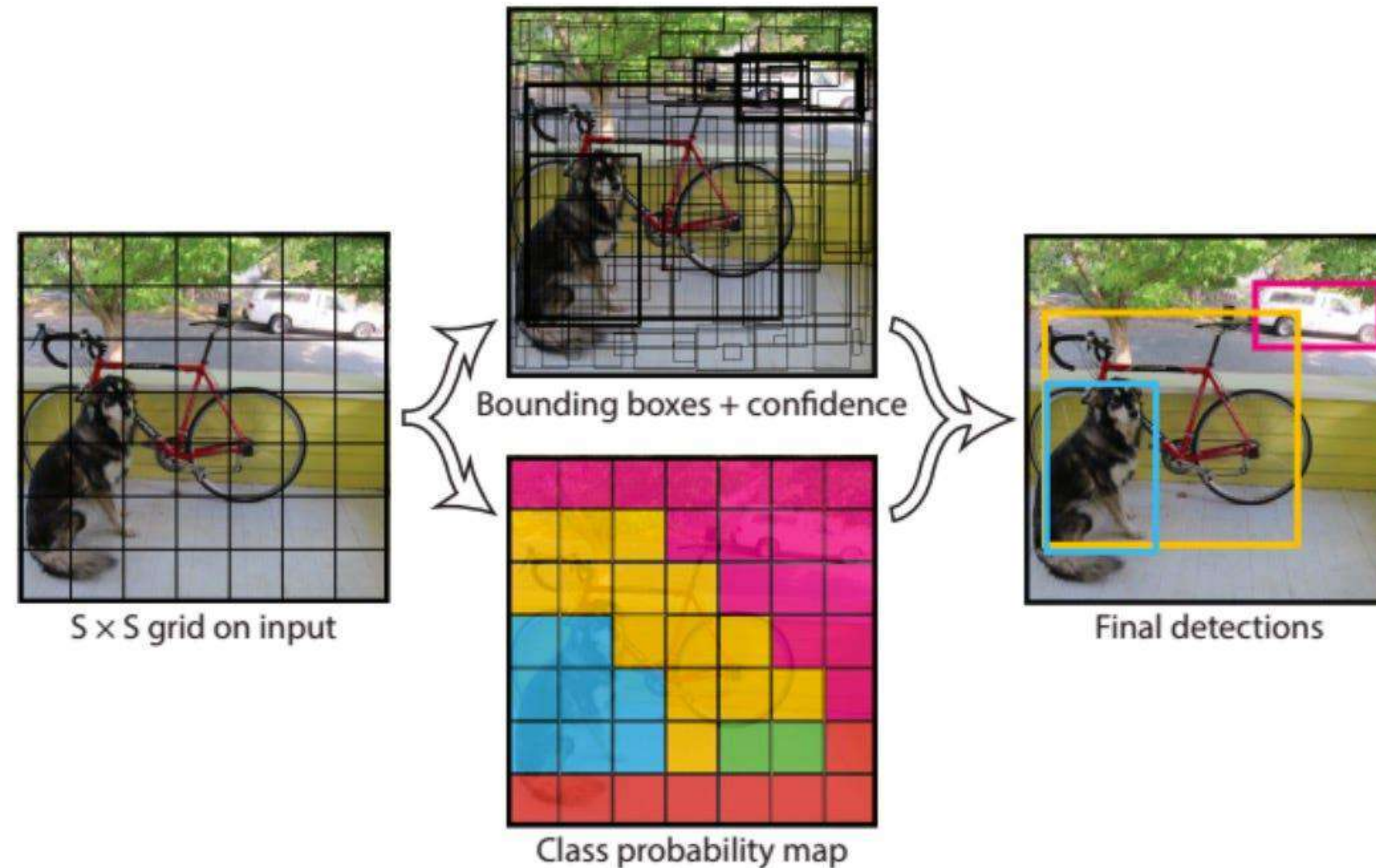
Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers:
(dx, dy, dh, dw, confidence)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

Output:
 $7 \times 7 \times (5 * B + C)$

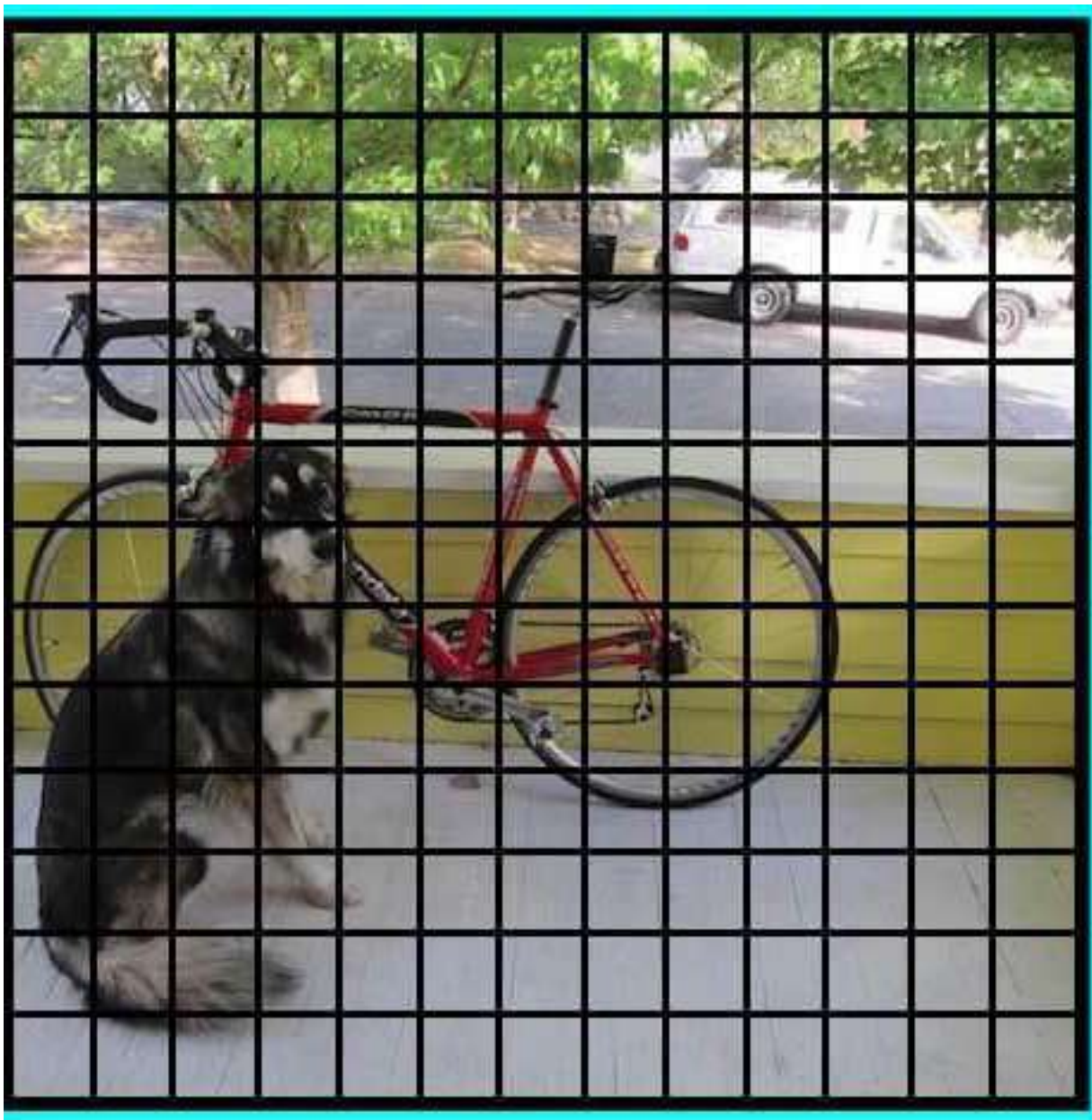
Redmon et al, "You Only Look Once:
Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

YOLO (You Only Look Once) real-time object detection



Redmon et al. "You only look once: unified, real-time object detection (2015)."

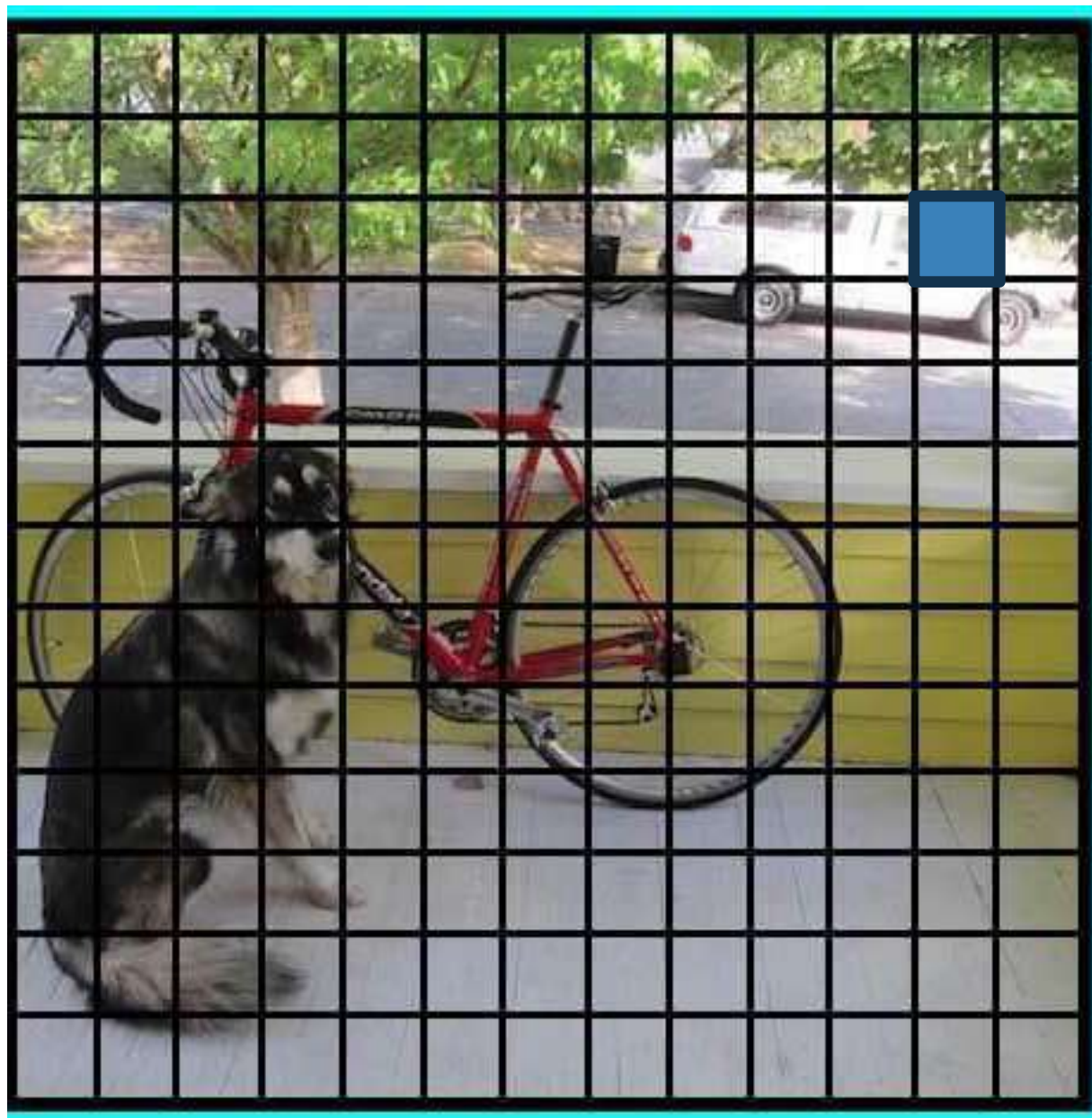
YOLO



SxS Grid

Redmon et al. "You only look once: unified, real-time object detection (2015)."

YOLO

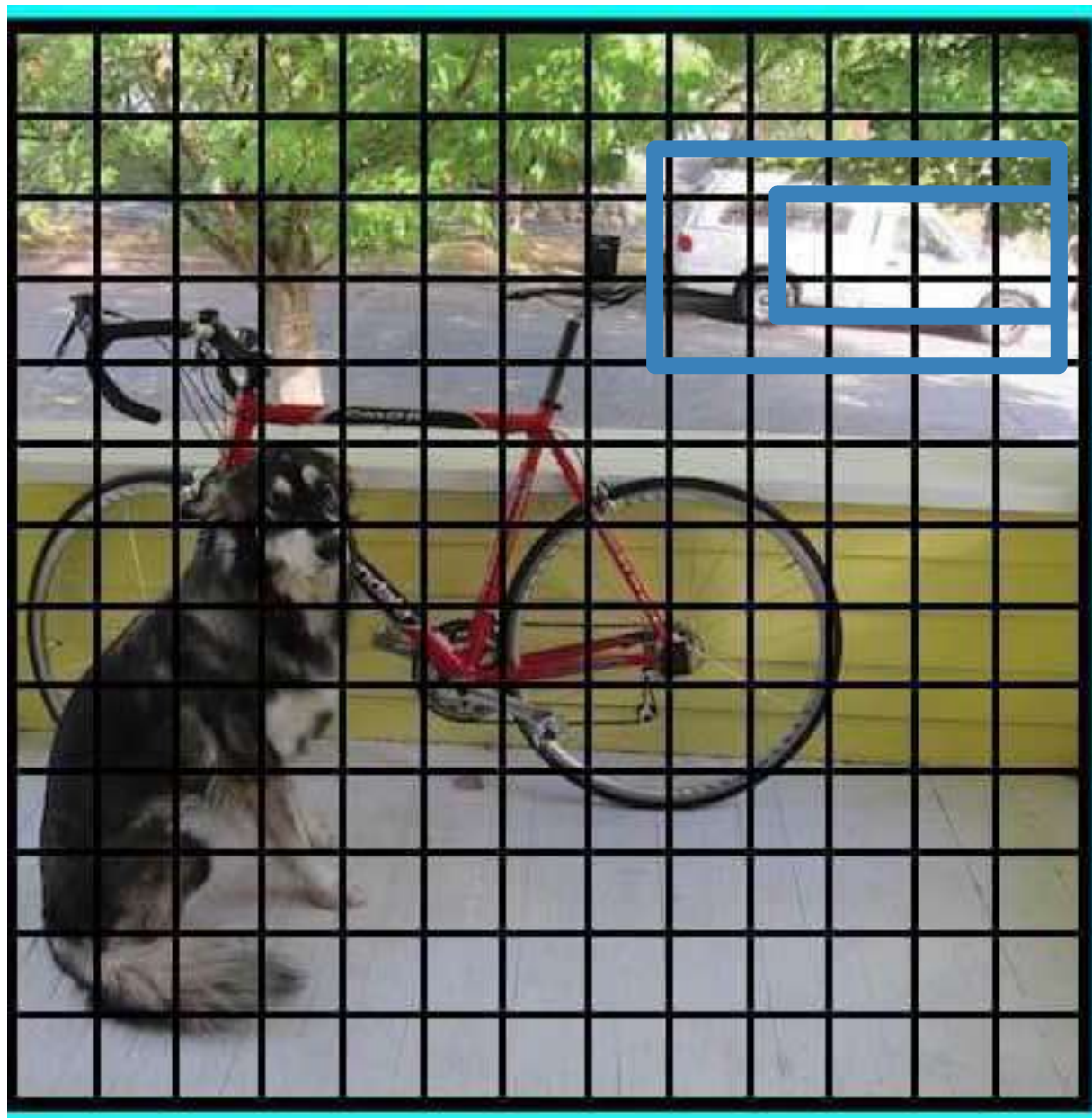


SxS Grid

For each box output:

- $P(\text{object})$: probability that the box contains an object
- B bounding boxes (x, y, h, w)
- $P(\text{class})$: probability of belonging to a class

YOLO



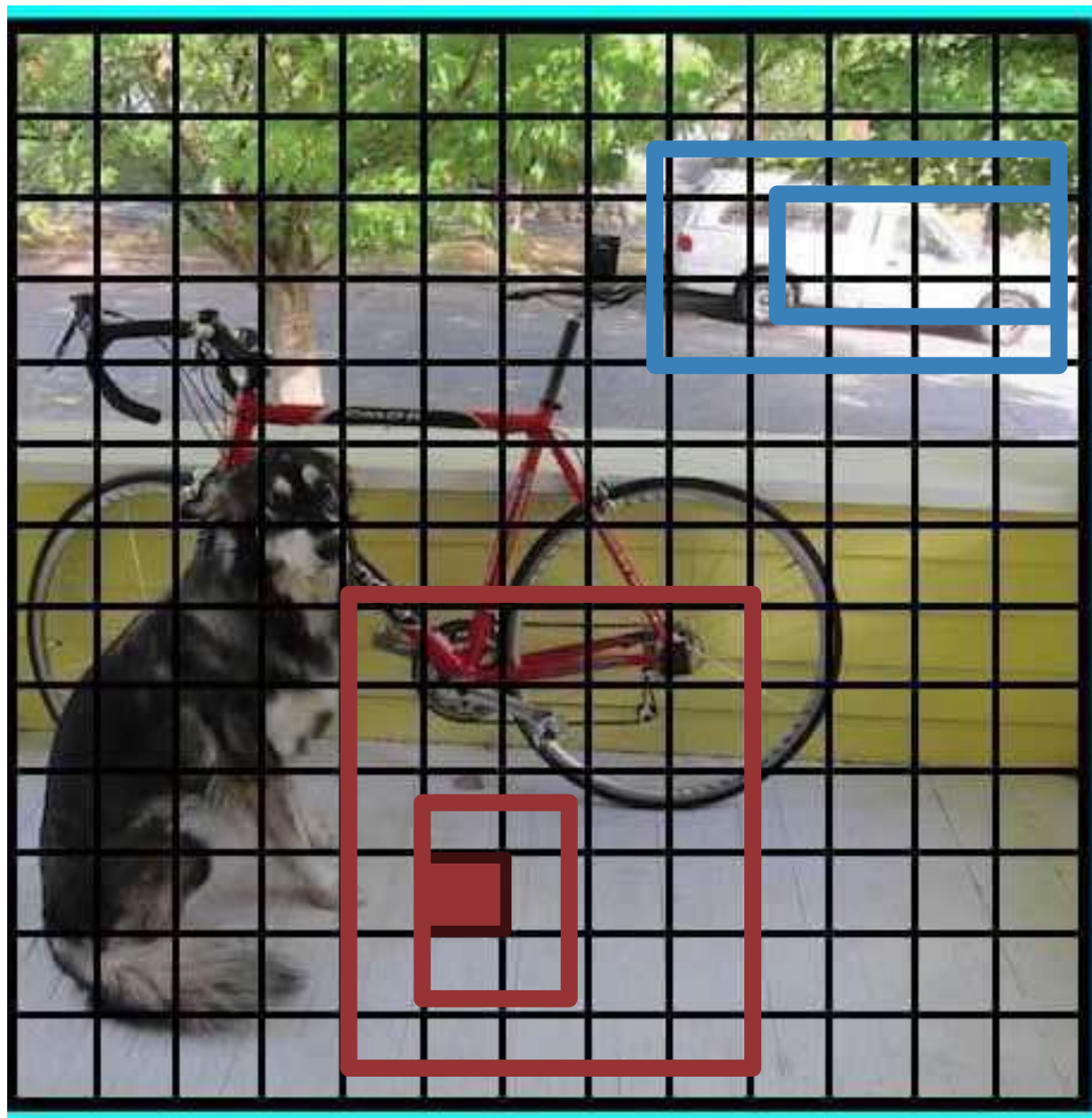
SxS Grid

For each box output:

- $P(\text{object})$: probability that the box contains an object
- B bounding boxes (x, y, h, w)
- $P(\text{class})$: probability of belonging to a class

$B=2$

YOLO



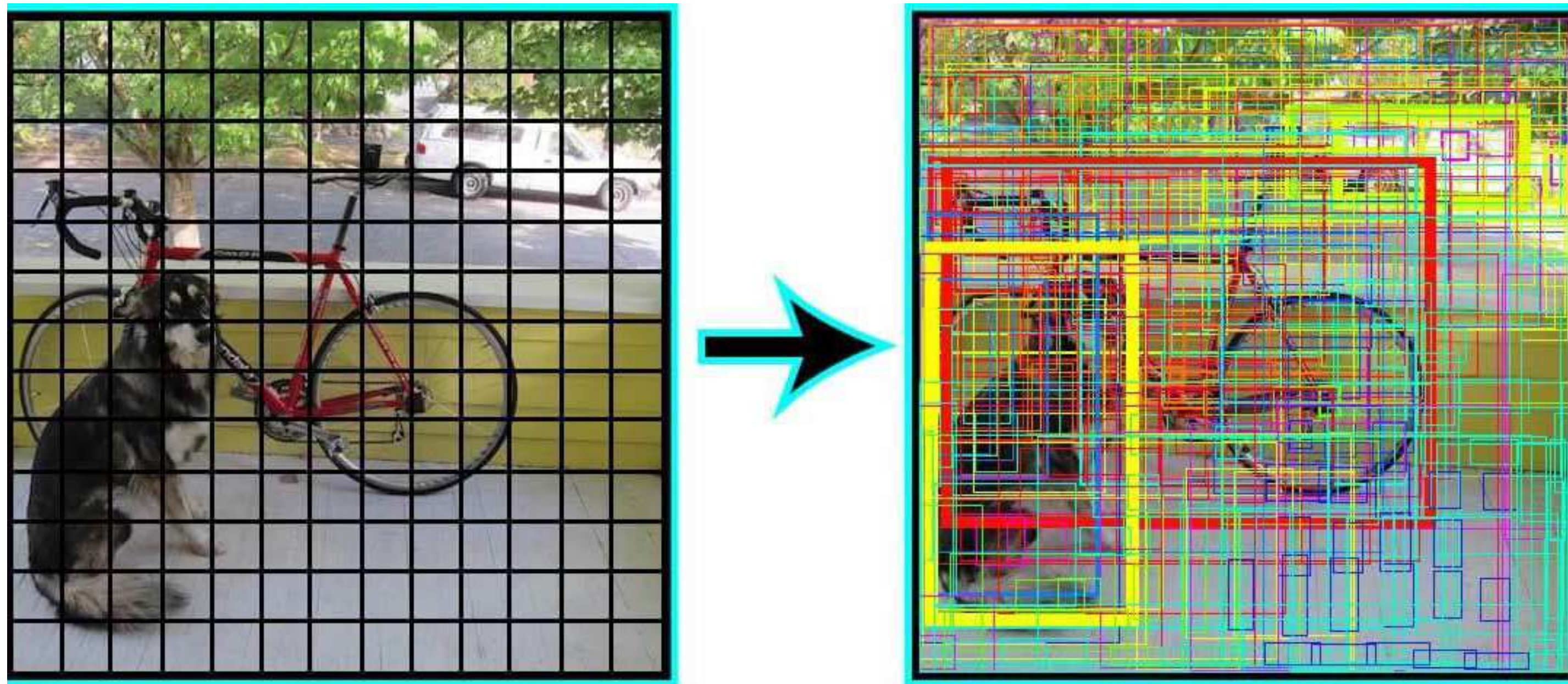
SxS Grid

For each box output:

- $P(\text{object})$: probability that the box contains an object
- B bounding boxes (x, y, h, w)
- $P(\text{class})$: probability of belonging to a class

$B=2$

YOLO

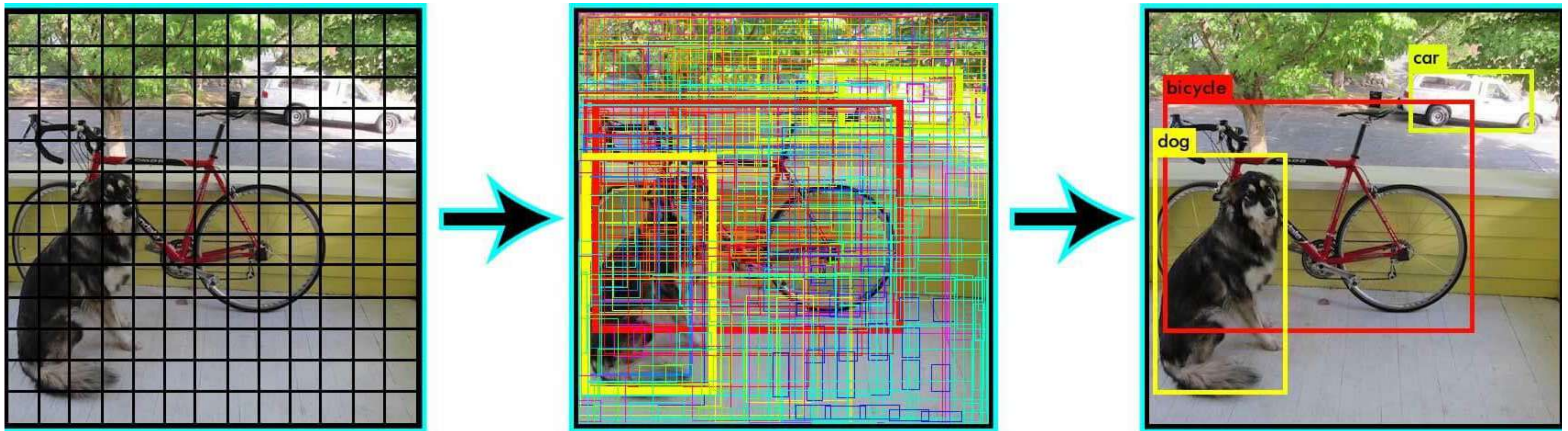


Many bounding boxes with different object probabilities

SxS Grid

Redmon et al. "You only look once: unified, real-time object detection (2015)."

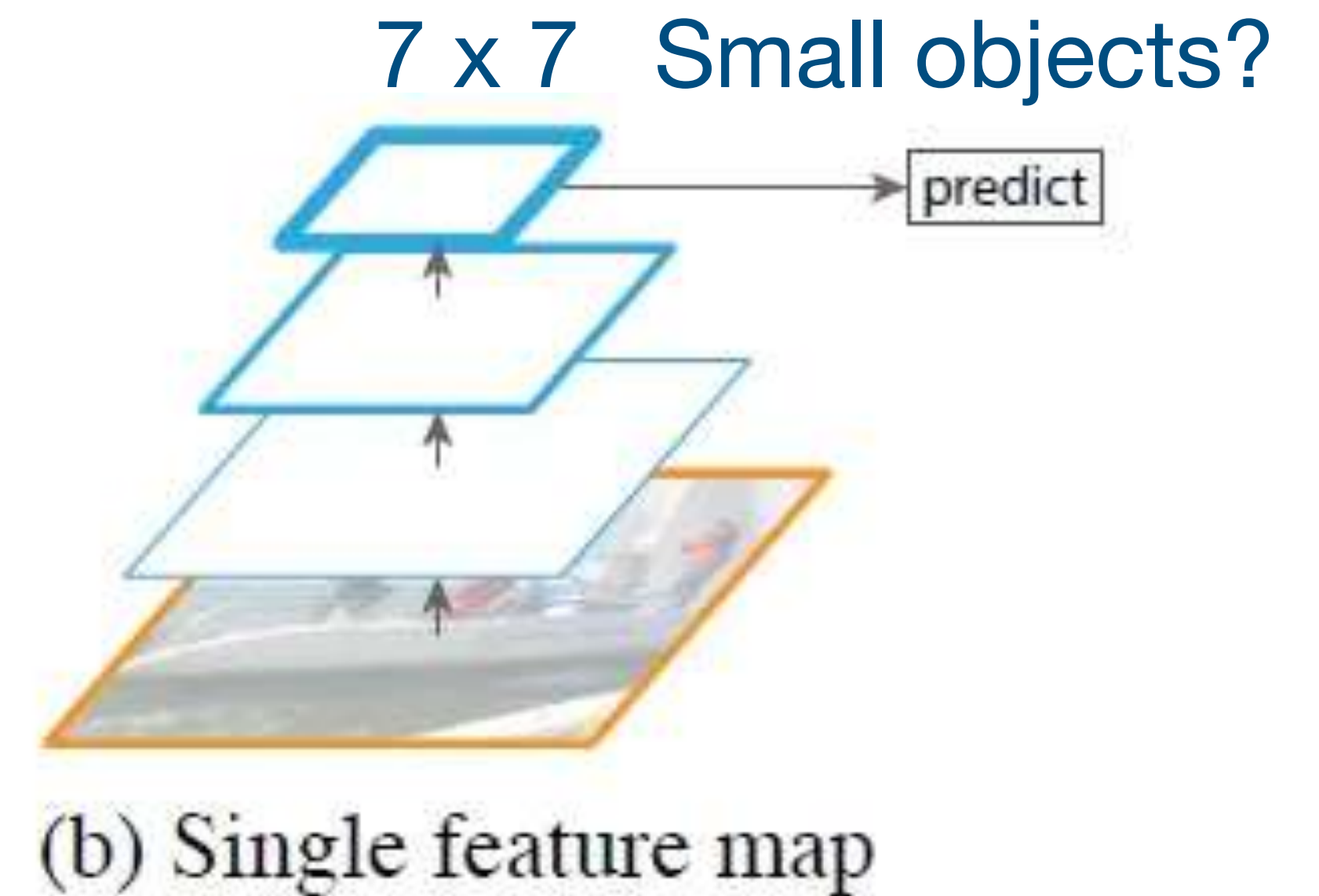
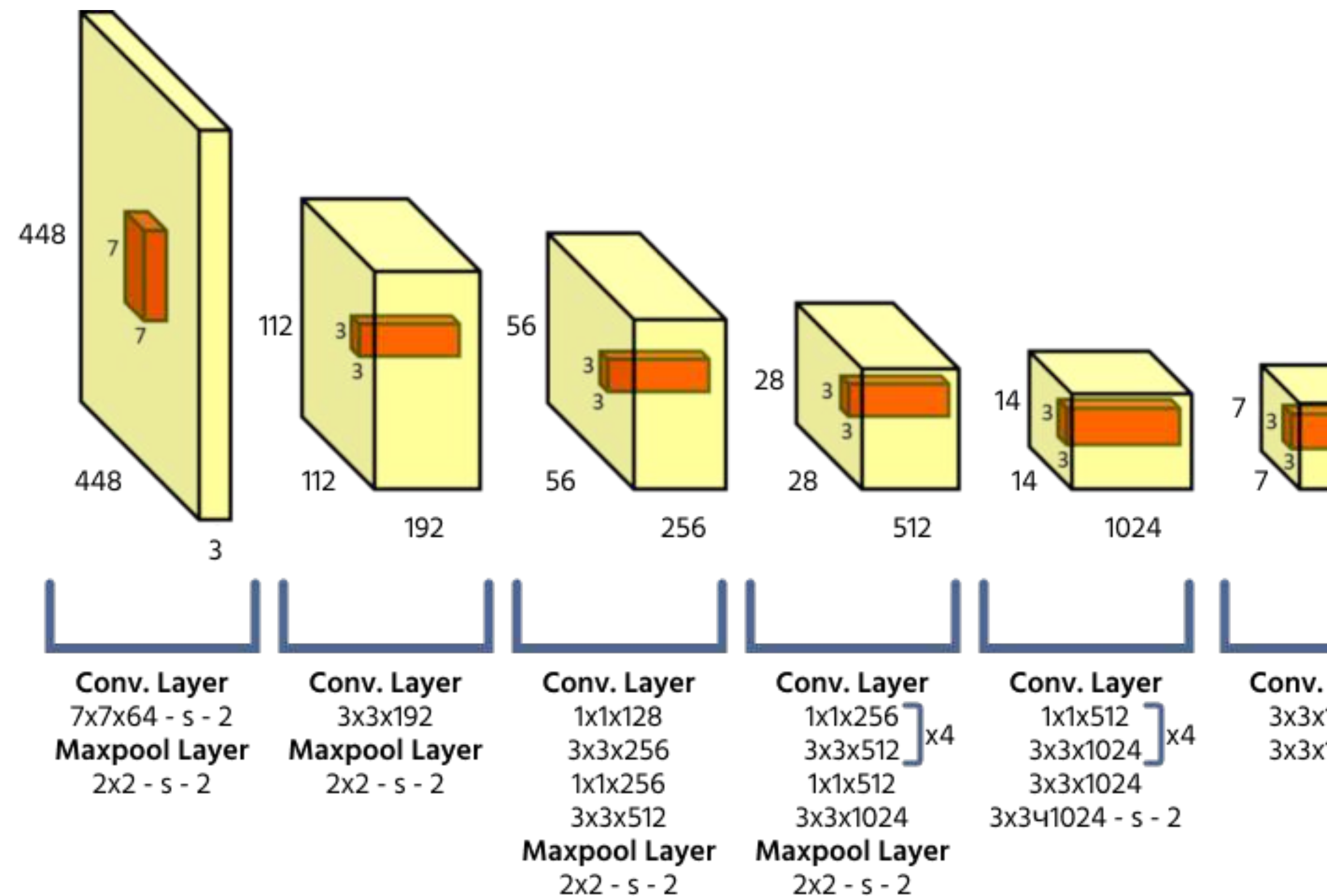
YOLO



$S \times S$ Grid

Redmon et al. "You only look once: unified, real-time object detection (2015)."

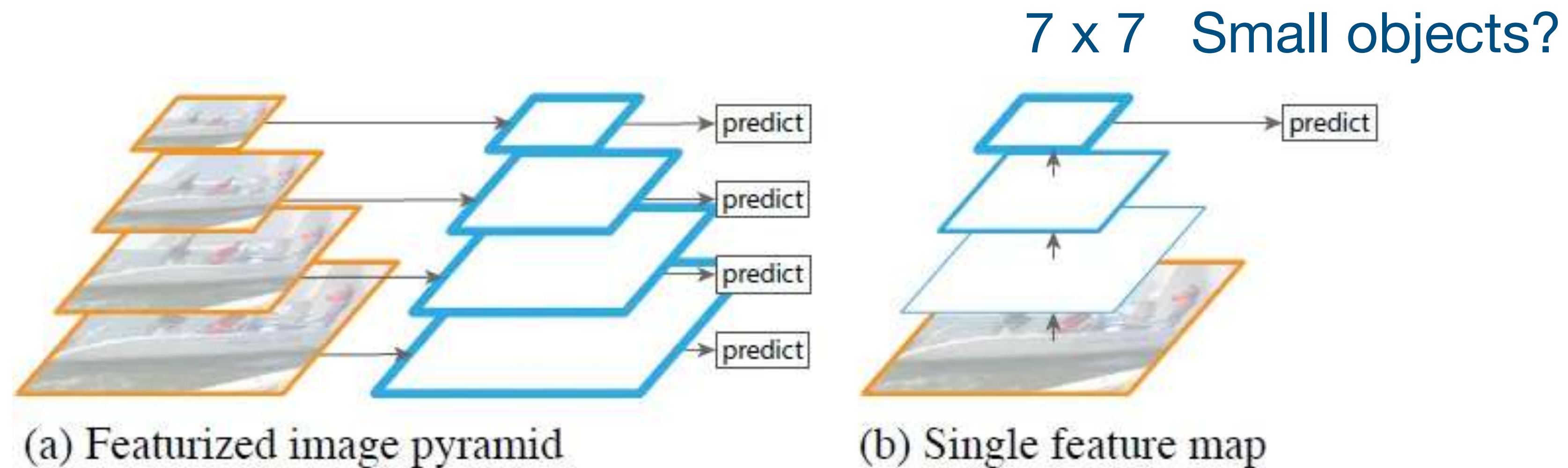
One-stage Object Detector: **YOLO**



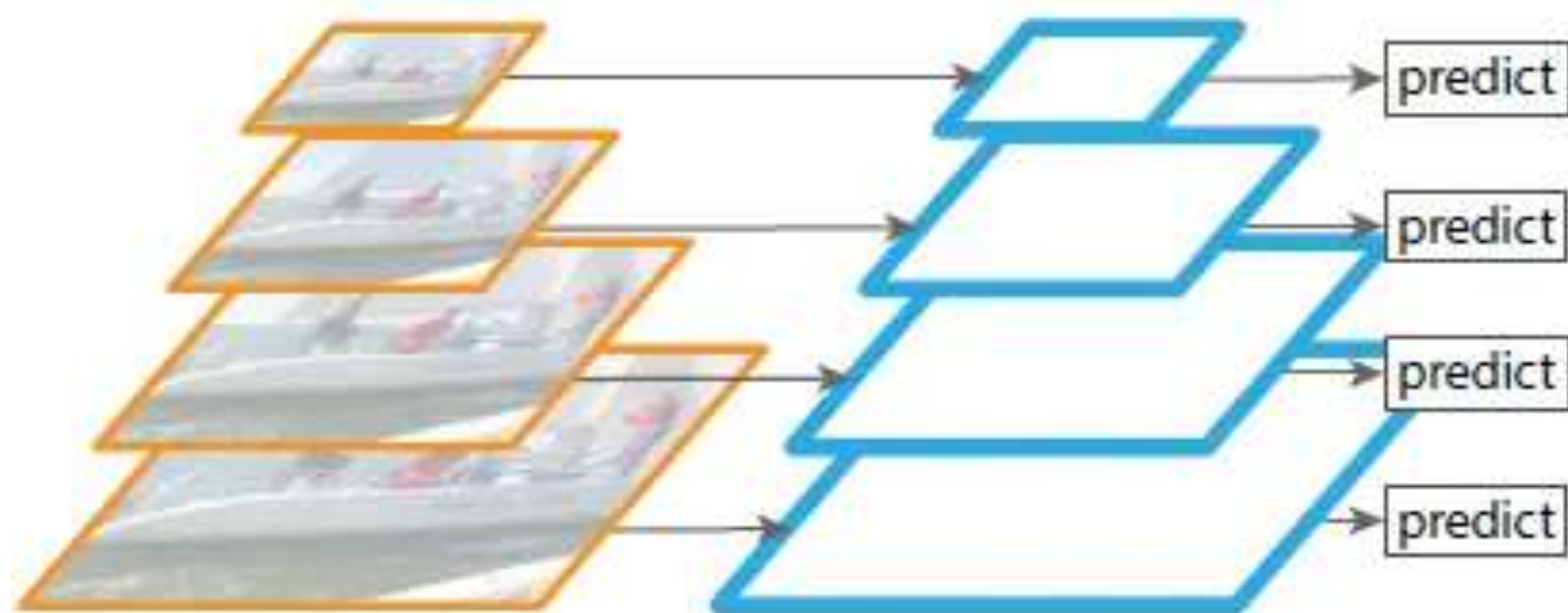
Redmon, Joseph, et al. "You only look once: Unified, real-time object detection." *CVPR* 2016.

<https://github.com/ultralytics/ultralytics>

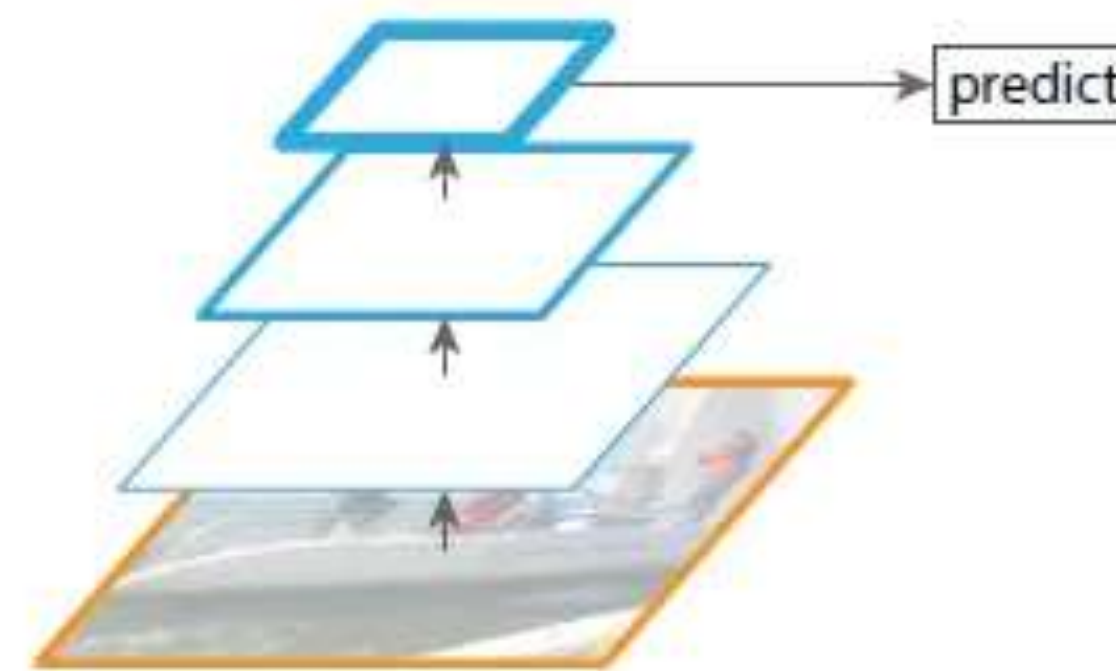
One-stage Object Detector: **YOLO** v.s. **FPN**



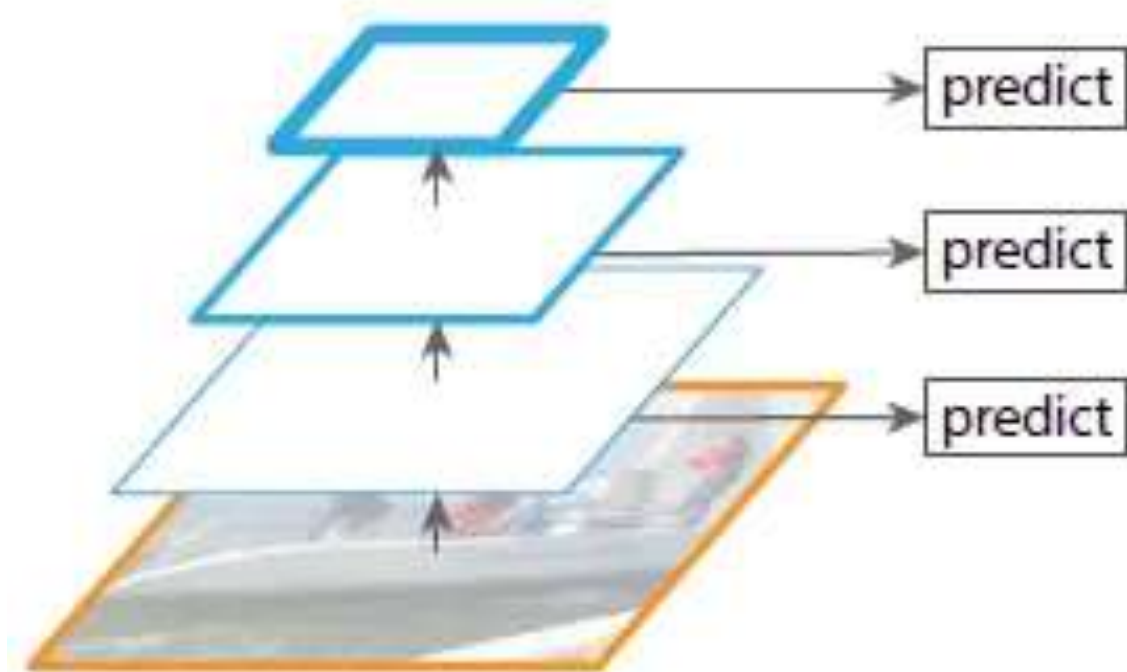
One-stage Object Detector: YOLO v.s. FPN



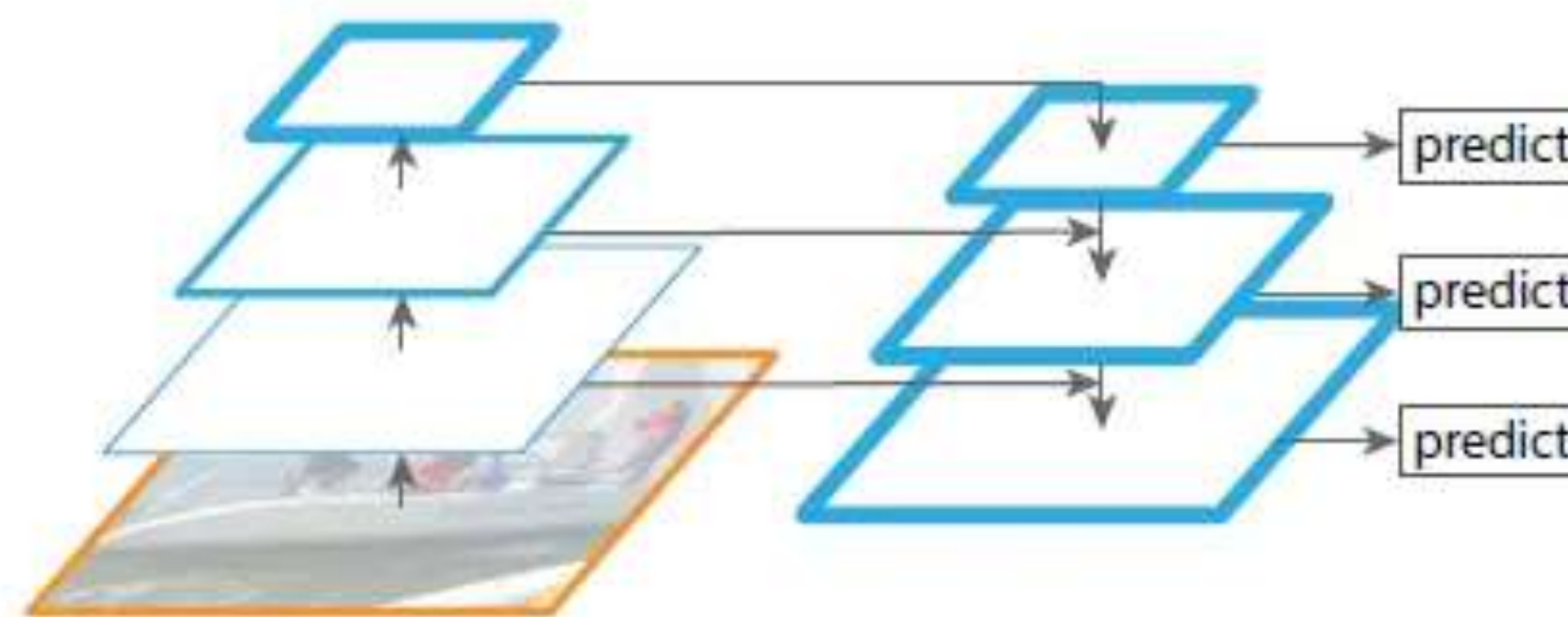
(a) Featurized image pyramid



(b) Single feature map



(c) Pyramidal feature hierarchy

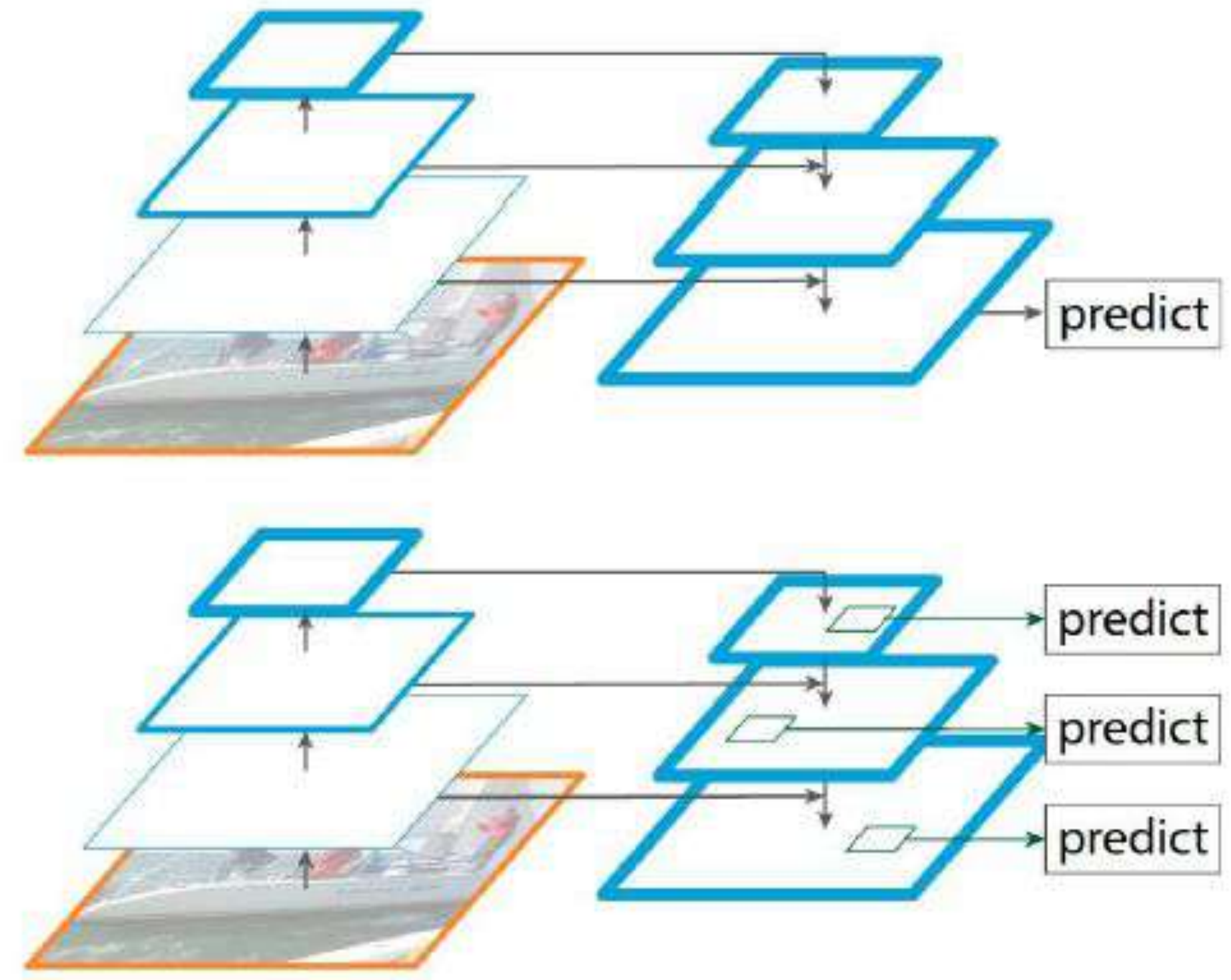


(d) Feature Pyramid Network

One-stage Object Detector: FPN

Feature Pyramid Network as the feature extractor

- Traditional backbone
 - Small feature maps have larger receptive field and contain better-extracted overall semantic information
 - Want this semantic information in larger feature maps for prediction
- Feature Pyramid Network
 - Richer representation
 - Enables multi-scale predictions



One-stage Object Detector: FPN

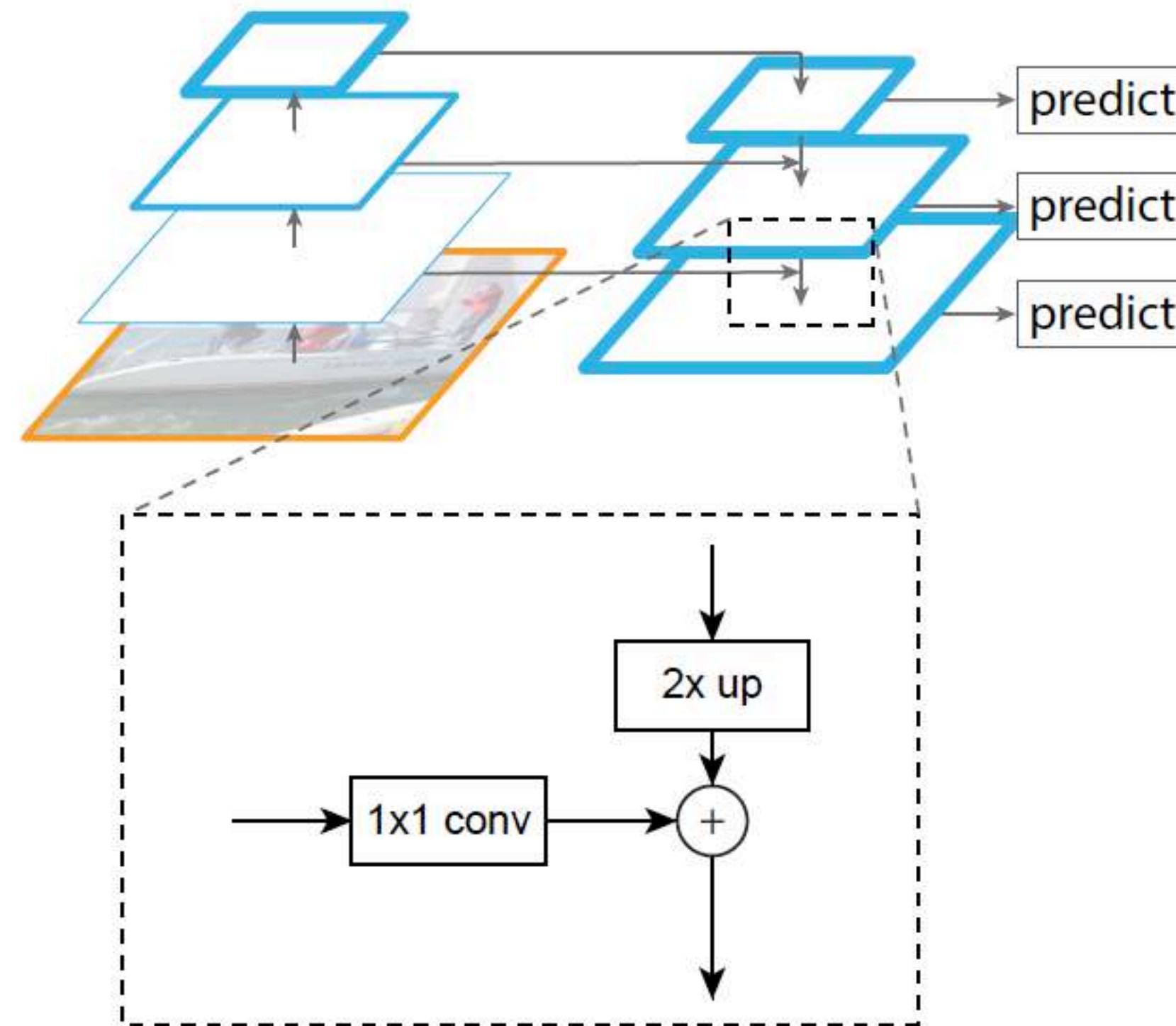
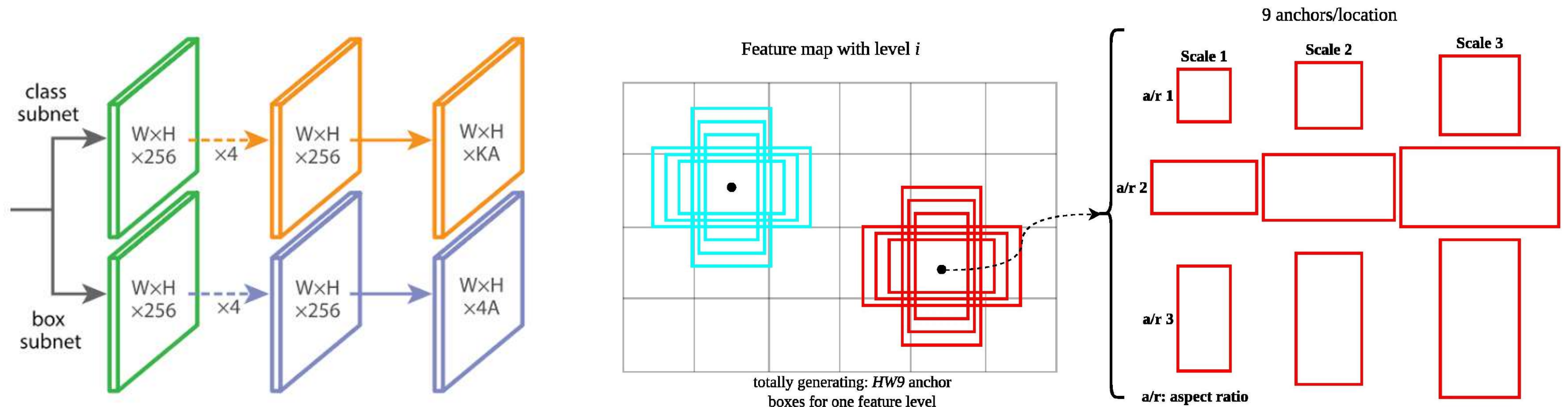


Figure 3. A building block illustrating the lateral connection and the top-down pathway, merged by addition.

One-stage Object Detector: FPN



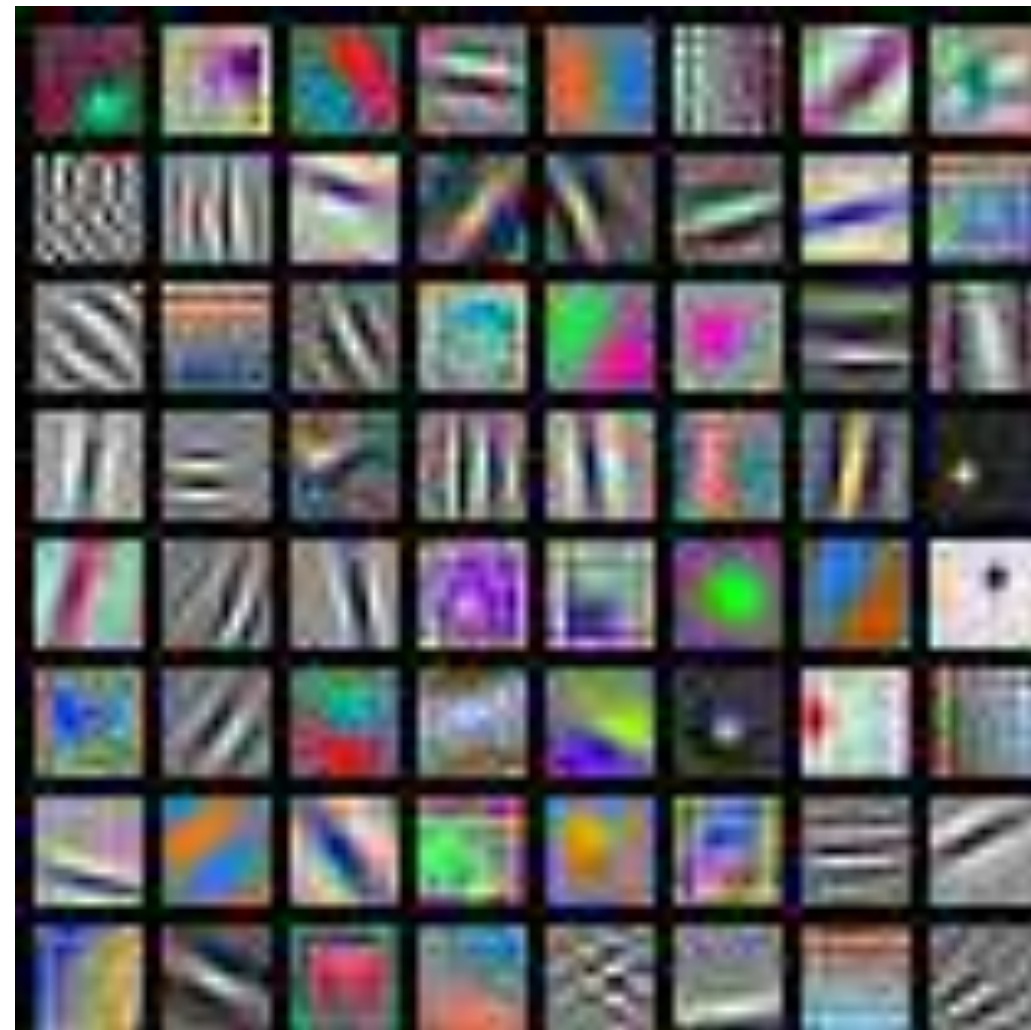
A: number of anchors

Visualization & Understanding

- Model Layers Visualization
- Saliency Maps
- CAM & Grad-CAM

Recap

First Layer: Visualize Filters



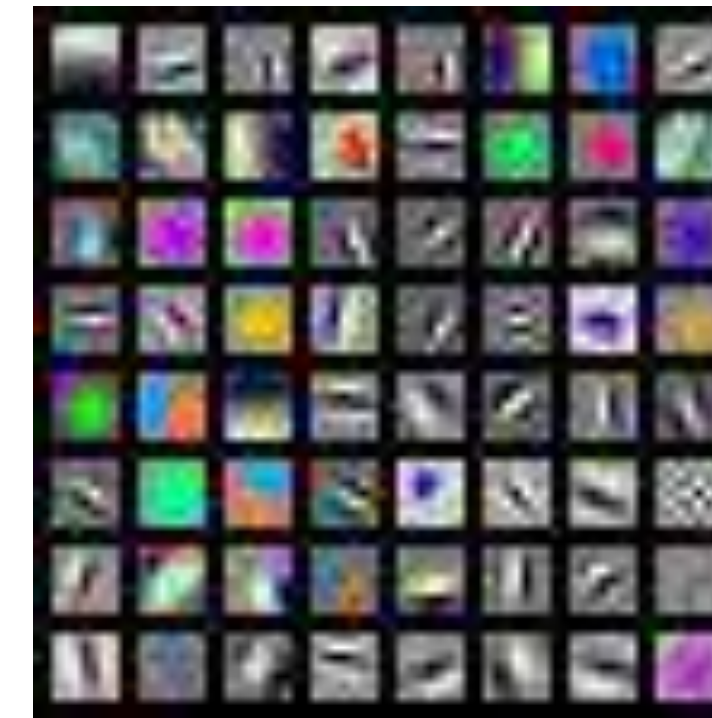
AlexNet:
 $64 \times 3 \times 11 \times 11$



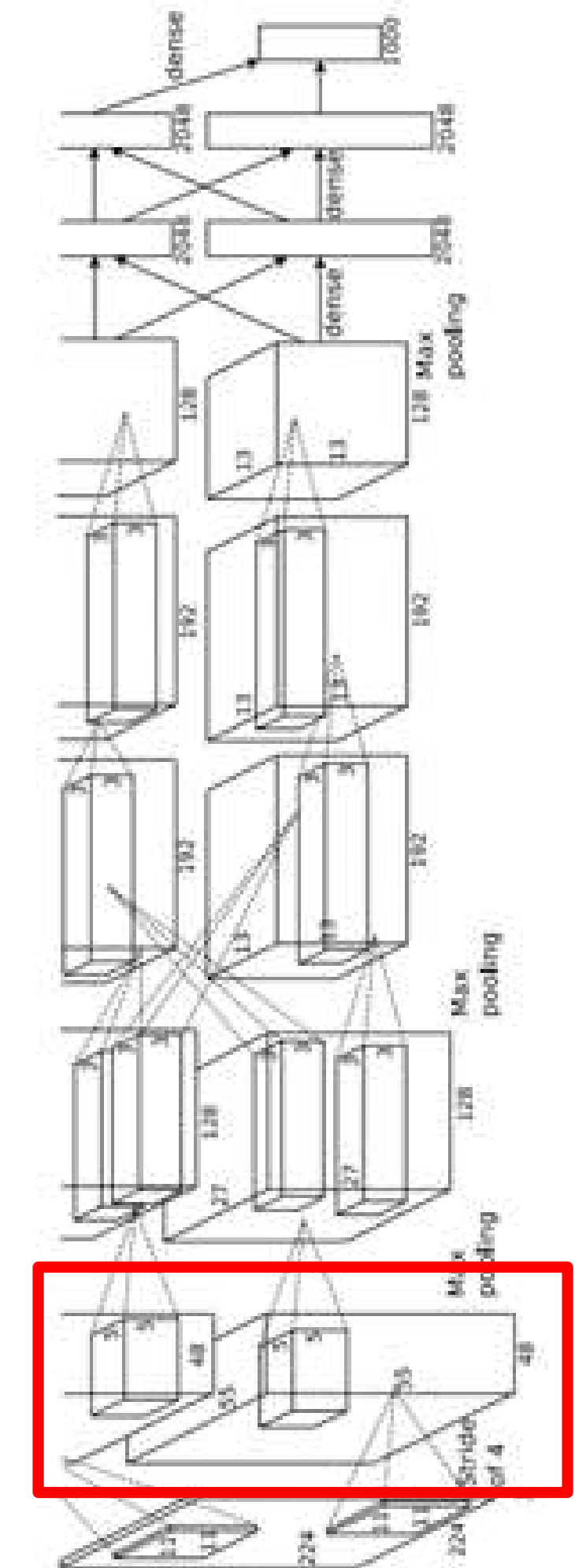
ResNet-18:
 $64 \times 3 \times 7 \times 7$



ResNet-101:
 $64 \times 3 \times 7 \times 7$



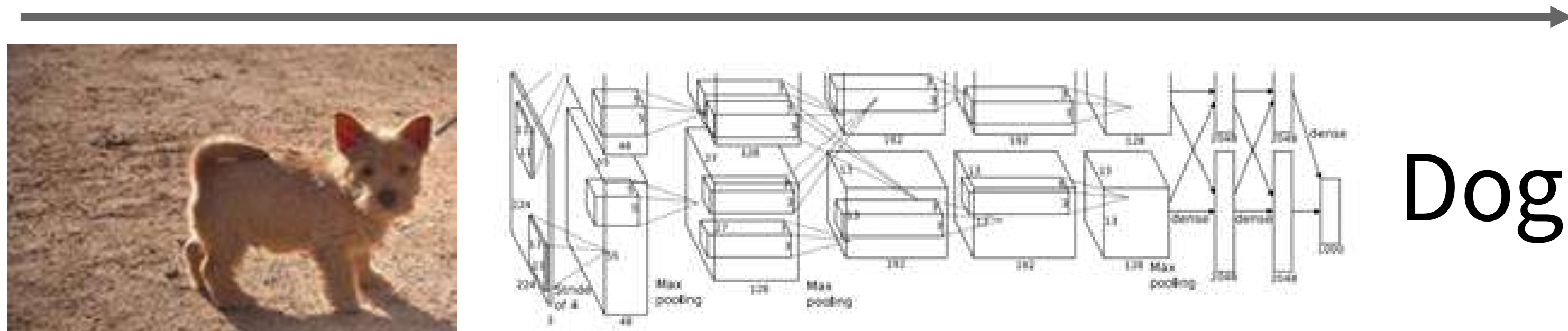
DenseNet-121:
 $64 \times 3 \times 7 \times 7$



Krizhevsky, "One weird trick for parallelizing convolutional neural networks", arXiv 2014
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
Huang et al, "Densely Connected Convolutional Networks", CVPR 2017

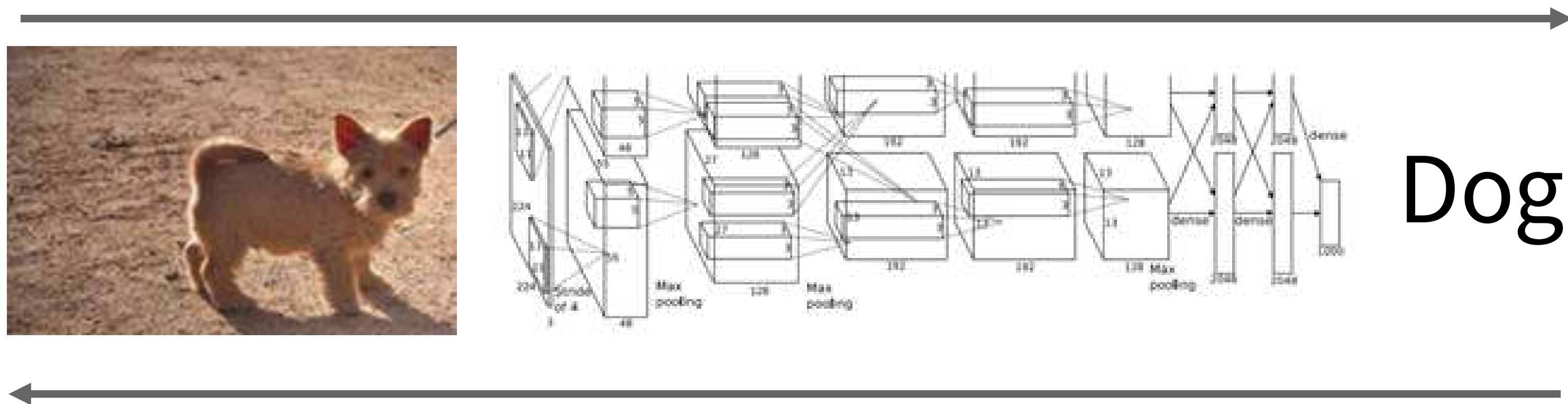
Which pixels matter: Saliency via Backprop

Forward pass: Compute probabilities

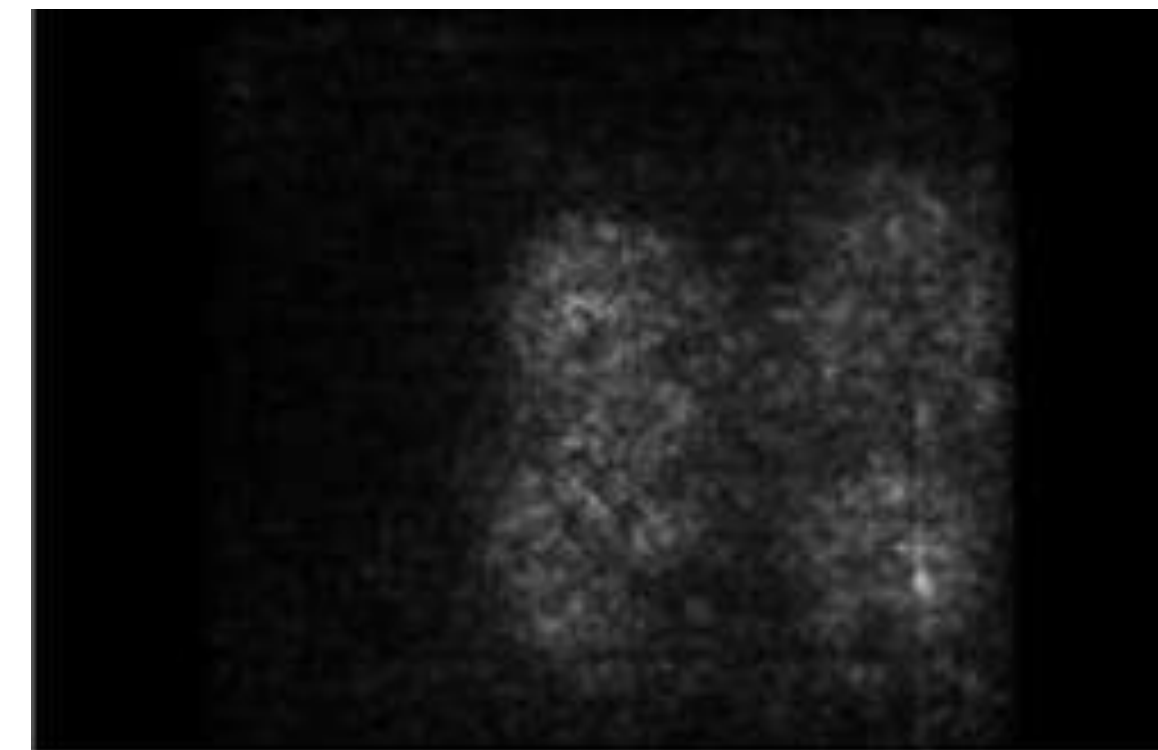


Which pixels matter: Saliency via Backprop

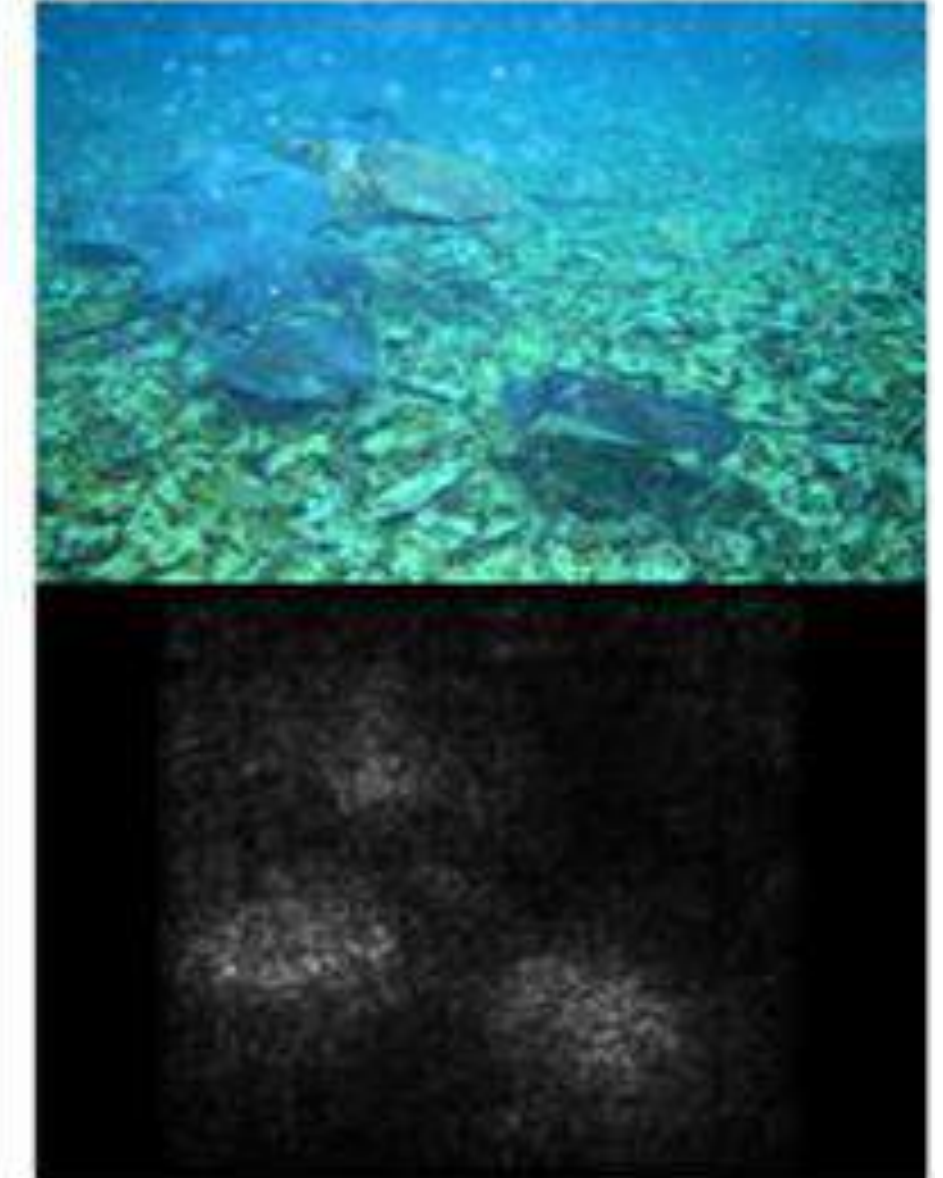
Forward pass: Compute probabilities



Compute gradient of (unnormalized) class score with respect to image pixels, take absolute value and max over RGB channels

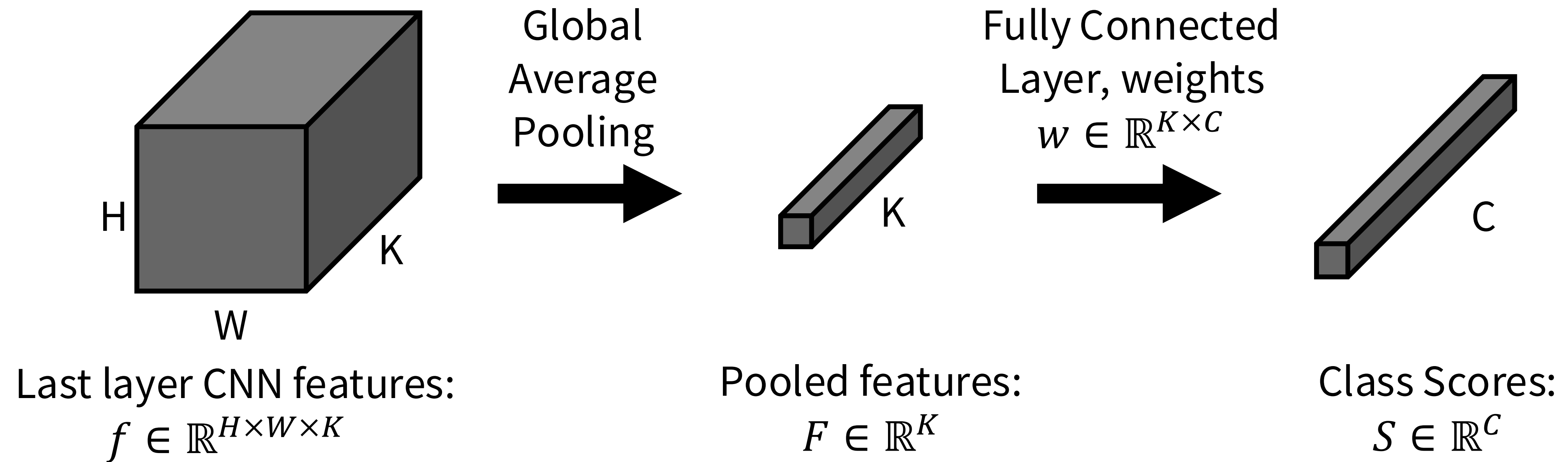


Saliency Maps

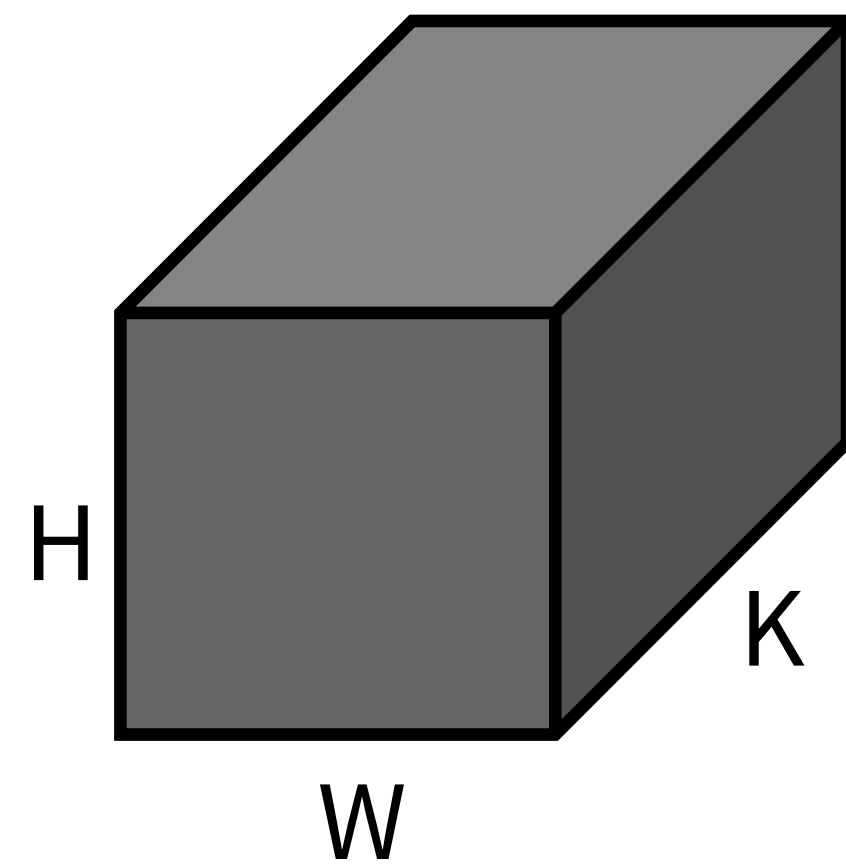


Simonyan, Vedaldi, and Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps", ICLR Workshop 2014.
Figures copyright Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman, 2014; reproduced with permission.

Class Activation Mapping (CAM)

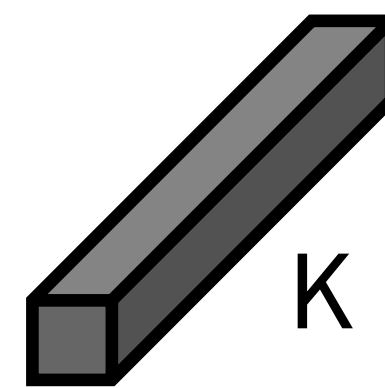


Class Activation Mapping (CAM)



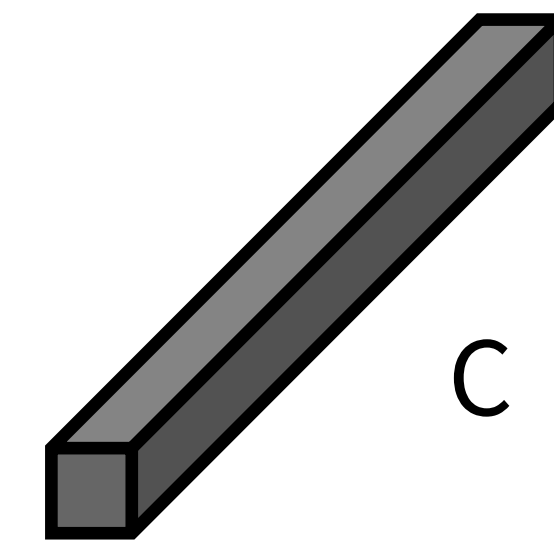
Last layer CNN features:
 $f \in \mathbb{R}^{H \times W \times K}$

Global
Average
Pooling
→



Pooled features:
 $F \in \mathbb{R}^K$

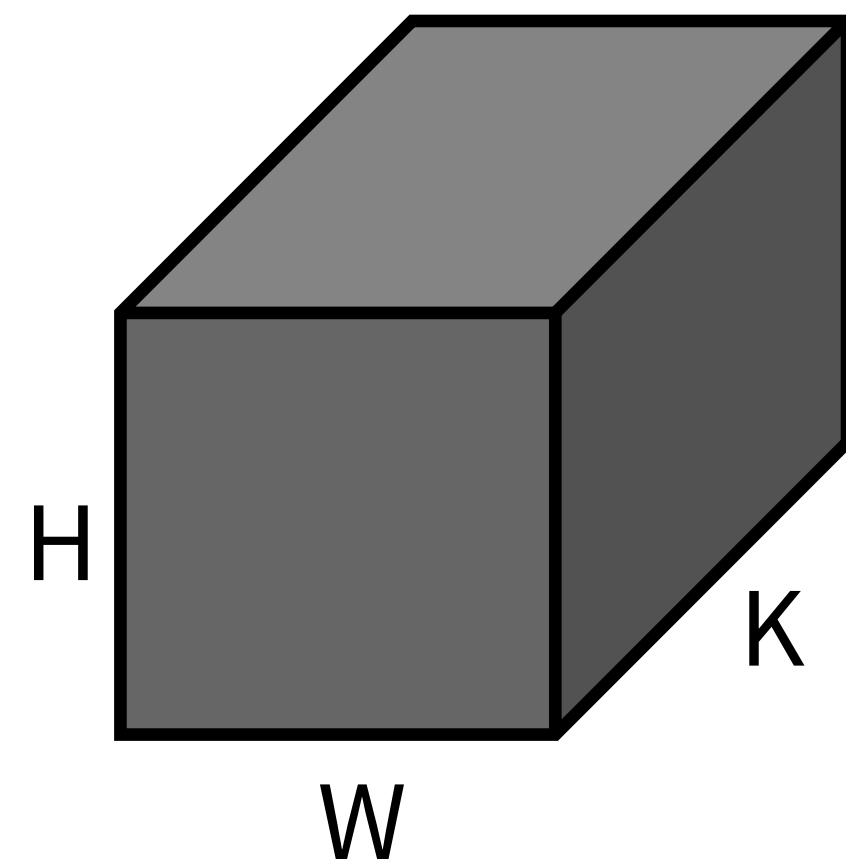
Fully Connected
Layer, weights
 $w \in \mathbb{R}^{K \times C}$
→



Class Scores:
 $S \in \mathbb{R}^C$

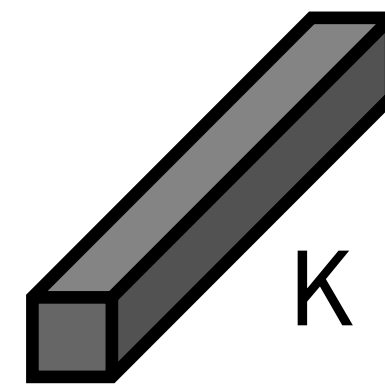
$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k}$$

Class Activation Mapping (CAM)



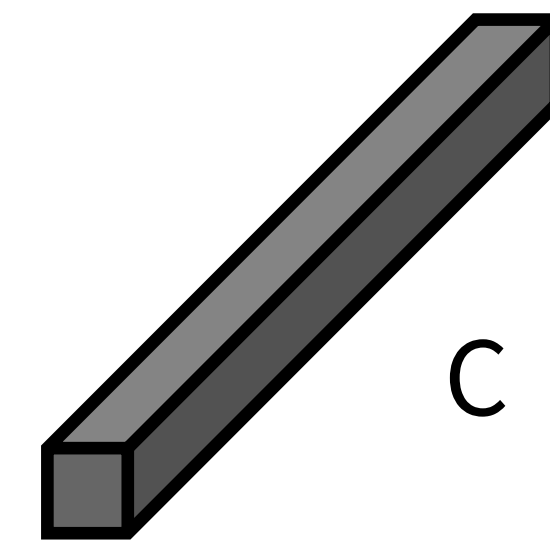
Last layer CNN features:
 $f \in \mathbb{R}^{H \times W \times K}$

Global
Average
Pooling
→



Pooled features:
 $F \in \mathbb{R}^K$

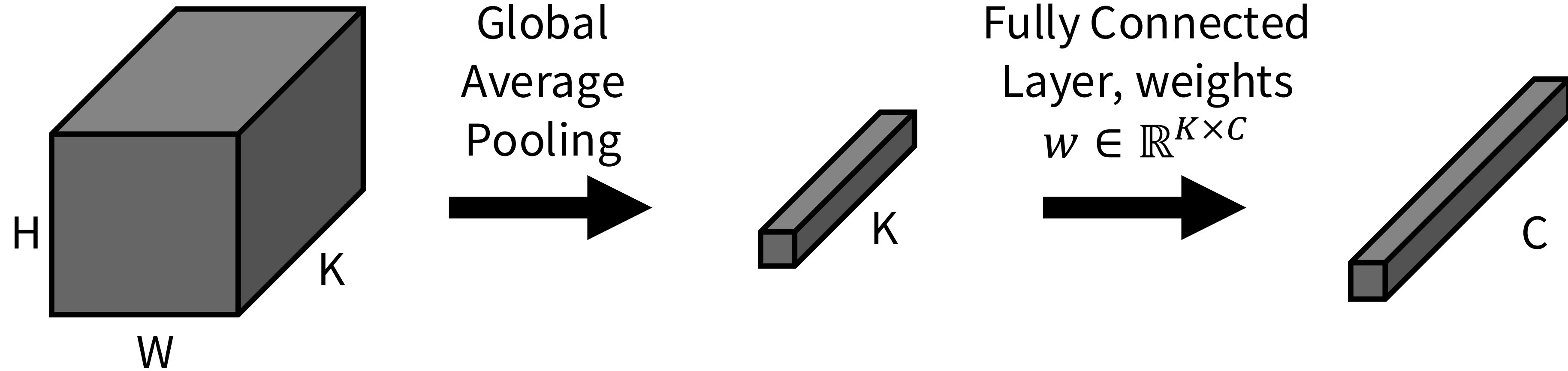
Fully Connected
Layer, weights
 $w \in \mathbb{R}^{K \times C}$
→



Class Scores:
 $S \in \mathbb{R}^C$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \quad S_c = \sum_k w_{k,c} F_k$$

Class Activation Mapping (CAM)



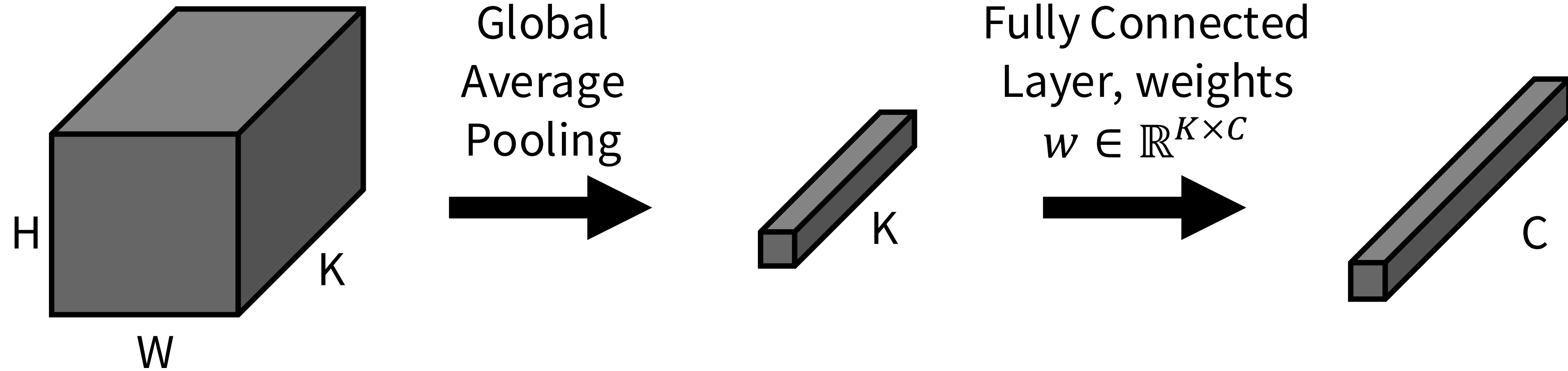
Last layer CNN features:
 $f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
 $F \in \mathbb{R}^K$

Class Scores:
 $S \in \mathbb{R}^C$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \quad S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$

Class Activation Mapping (CAM)



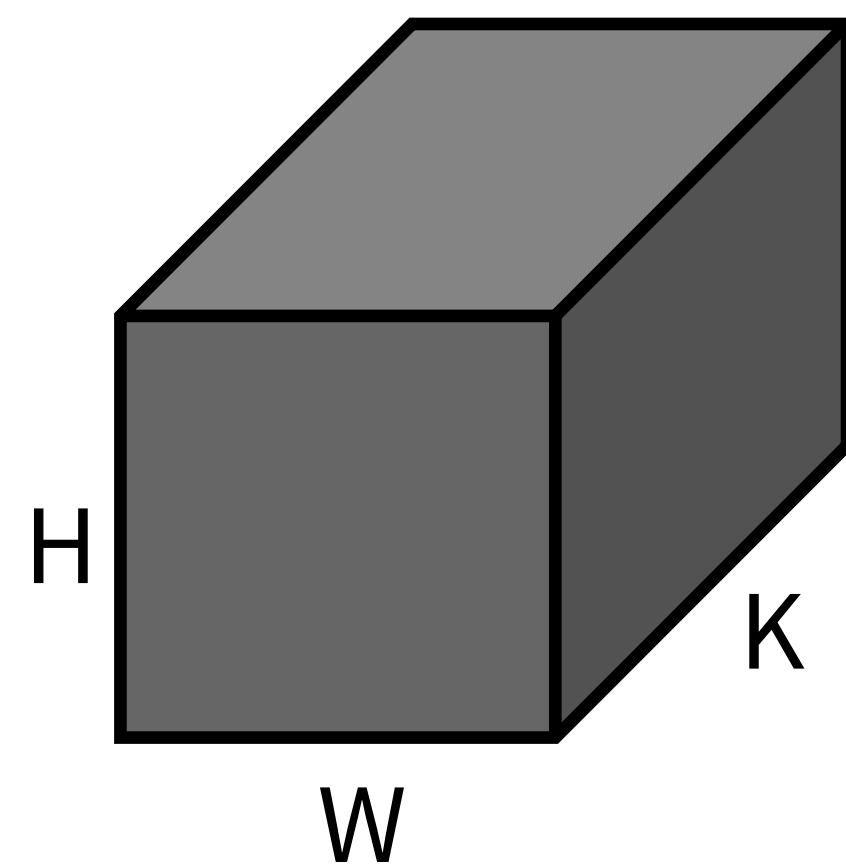
Last layer CNN features:
 $f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
 $F \in \mathbb{R}^K$

Class Scores:
 $S \in \mathbb{R}^C$

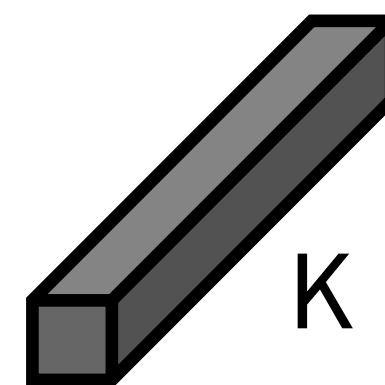
$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \quad S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k} \\ = \frac{1}{HW} \sum_{h,w} \sum_k w_{k,c} f_{h,w,k}$$

Class Activation Mapping (CAM)



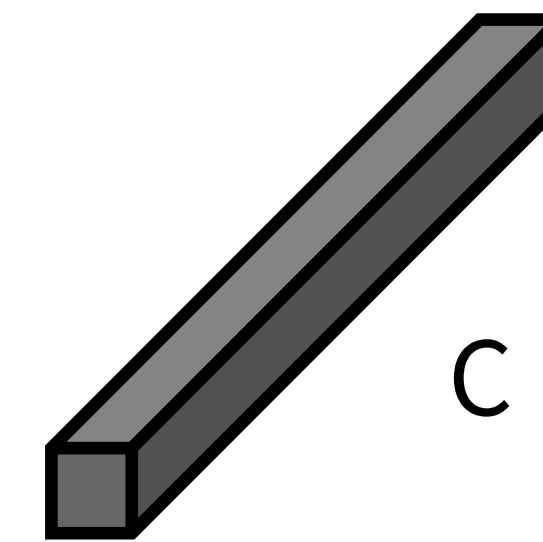
Last layer CNN features:
 $f \in \mathbb{R}^{H \times W \times K}$

Global
Average
Pooling

Pooled features:
 $F \in \mathbb{R}^K$

Fully Connected
Layer, weights
 $w \in \mathbb{R}^{K \times C}$

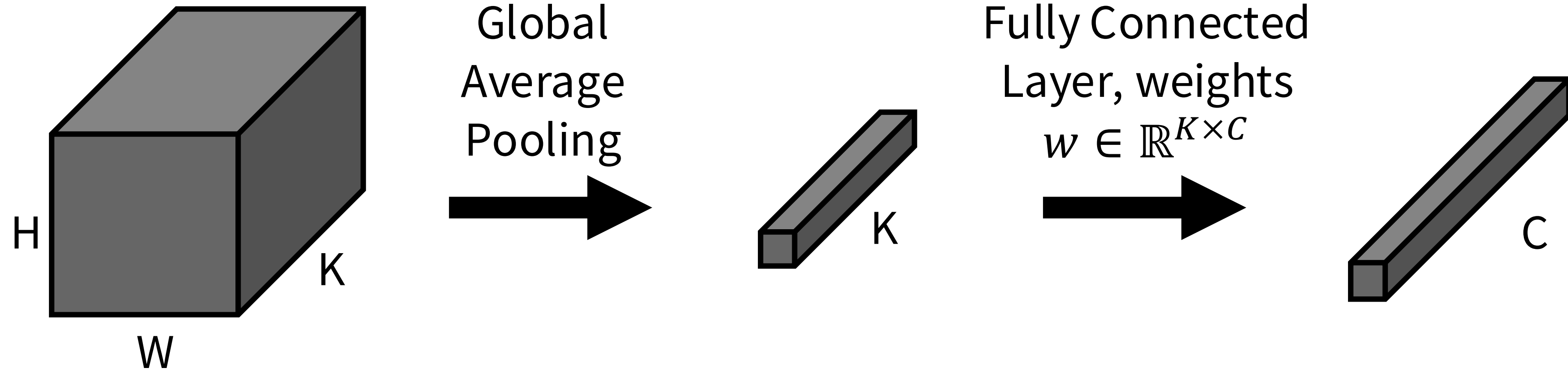



Class Scores:
 $S \in \mathbb{R}^C$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \quad S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$

$$= \frac{1}{HW} \sum_{h,w} \sum_k w_{k,c} f_{h,w,k}$$

Class Activation Mapping (CAM)



Last layer CNN features:
 $f \in \mathbb{R}^{H \times W \times K}$

Pooled features:
 $F \in \mathbb{R}^K$

Class Scores:
 $S \in \mathbb{R}^C$

$$F_k = \frac{1}{HW} \sum_{h,w} f_{h,w,k} \quad S_c = \sum_k w_{k,c} F_k = \frac{1}{HW} \sum_k w_{k,c} \sum_{h,w} f_{h,w,k}$$

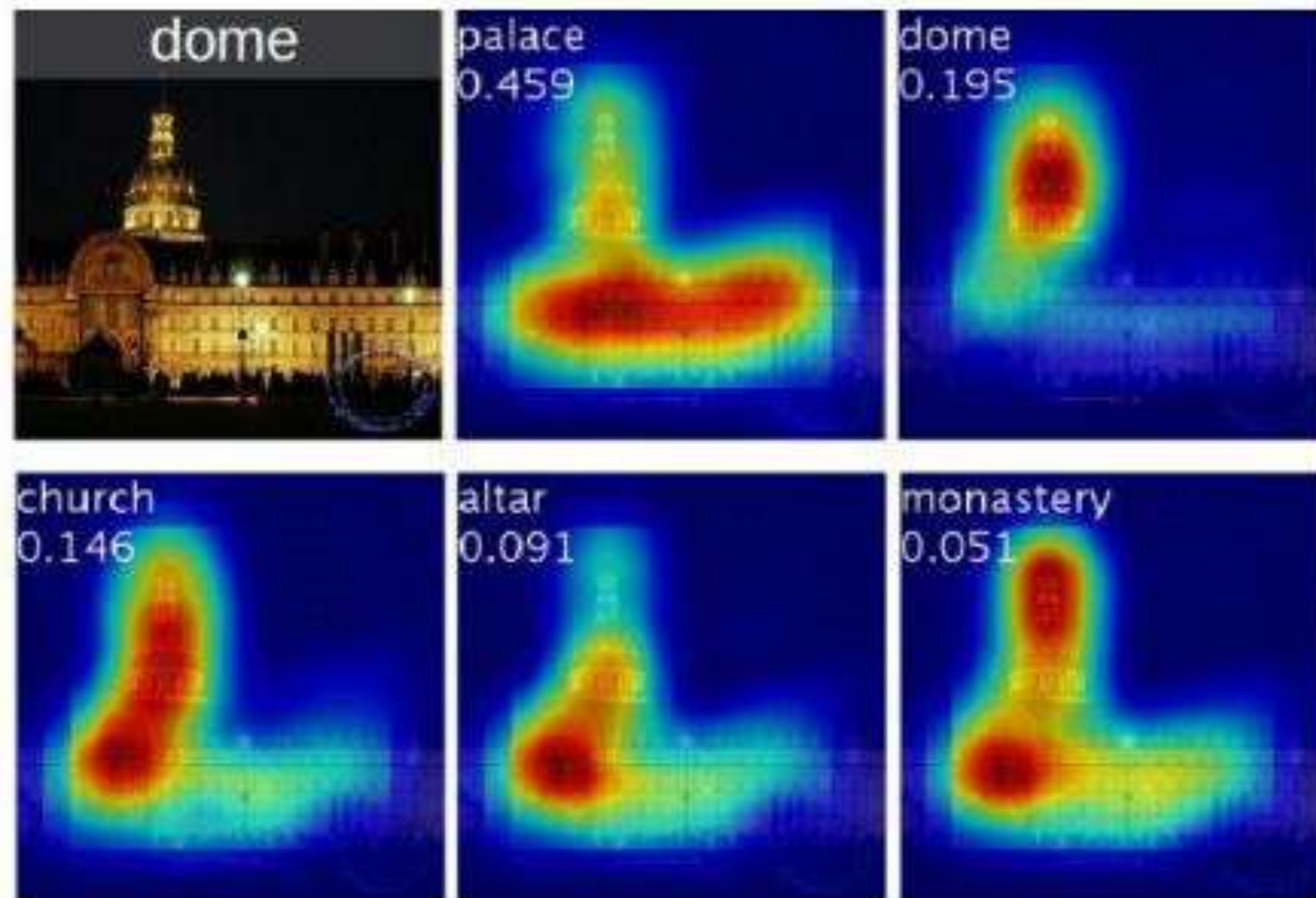
$$= \frac{1}{HW} \sum_{h,w} \sum_k w_{k,c} f_{h,w,k}$$

Class Activation Maps:

$$M \in \mathbb{R}^{C, H, W}$$

$$M_{c,h,w} = \sum_k w_{k,c} f_{h,w,k}$$

Class Activation Mapping (CAM)



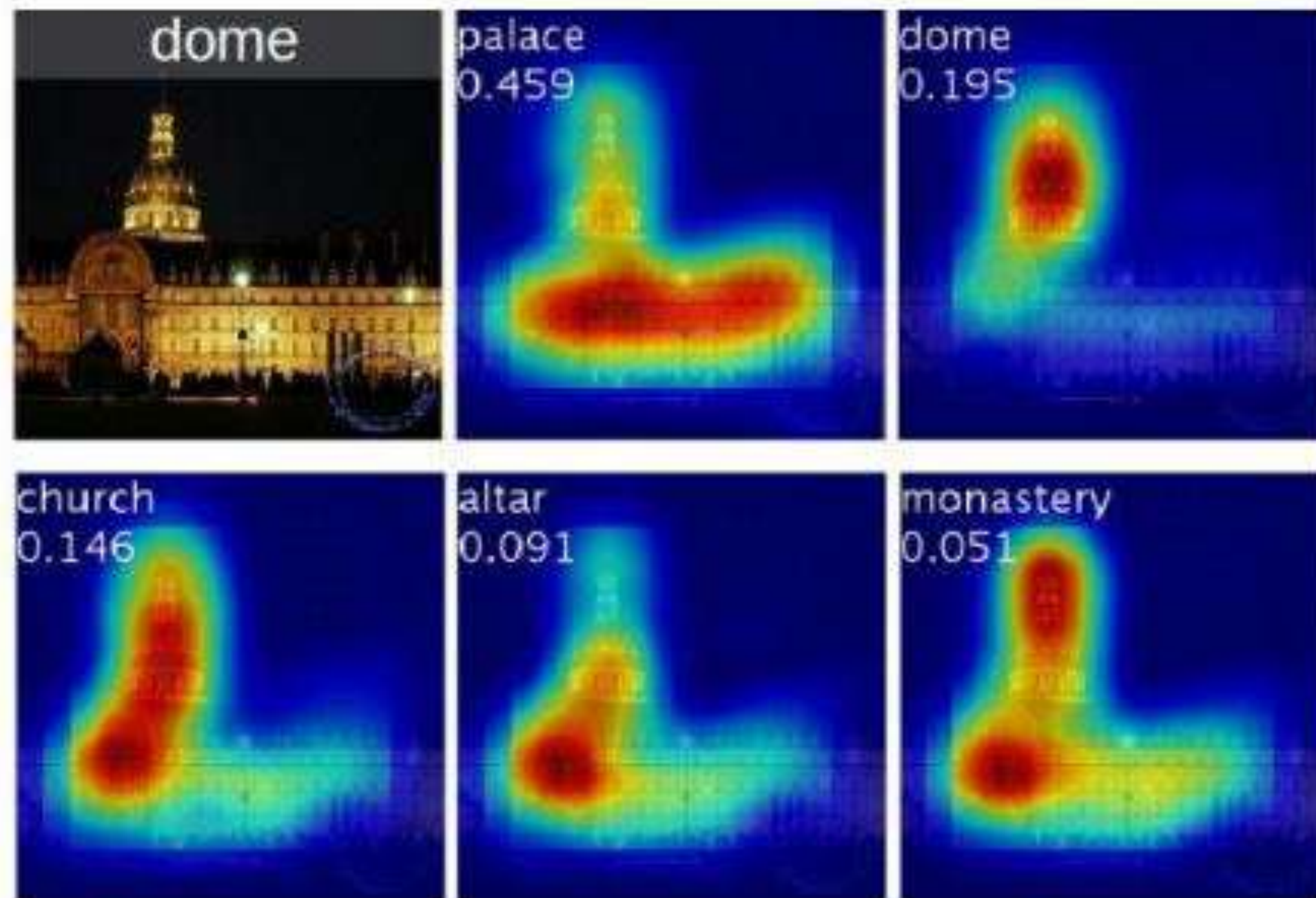
Class activation maps of top 5 predictions



Class activation maps for one object class

Class Activation Mapping (CAM)

Problem: Can only
apply to last conv layer



Class activation maps of top 5 predictions



Class activation maps for one object class

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times K}$

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times K}$
2. Compute gradient of class score S_c with respect to A :

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times K}$$

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times K}$
2. Compute gradient of class score S_c with respect to A:

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times K}$$

3. Global Average Pool the gradients to get weights $\alpha \in \mathbb{R}^K$:

$$\alpha_k = \frac{1}{HW} \sum_{h,w} \frac{\partial S_c}{\partial A_{h,w,k}}$$

Gradient-Weighted Class Activation Mapping (Grad-CAM)

1. Pick any layer, with activations $A \in \mathbb{R}^{H \times W \times K}$
2. Compute gradient of class score S_c with respect to A :

$$\frac{\partial S_c}{\partial A} \in \mathbb{R}^{H \times W \times K}$$

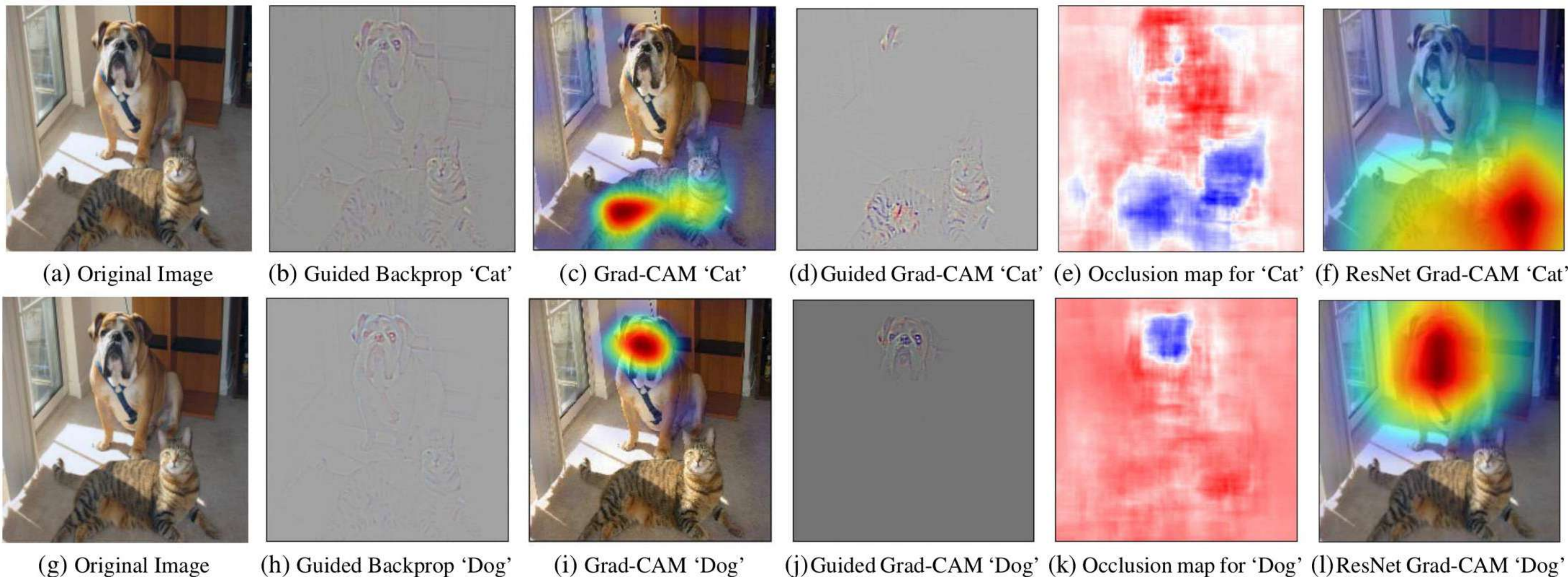
3. Global Average Pool the gradients to get weights $\alpha \in \mathbb{R}^K$:

$$\alpha_k = \frac{1}{HW} \sum_{h,w} \frac{\partial S_c}{\partial A_{h,w,k}}$$

4. Compute activation map $M^c \in \mathbb{R}^{H,W}$:

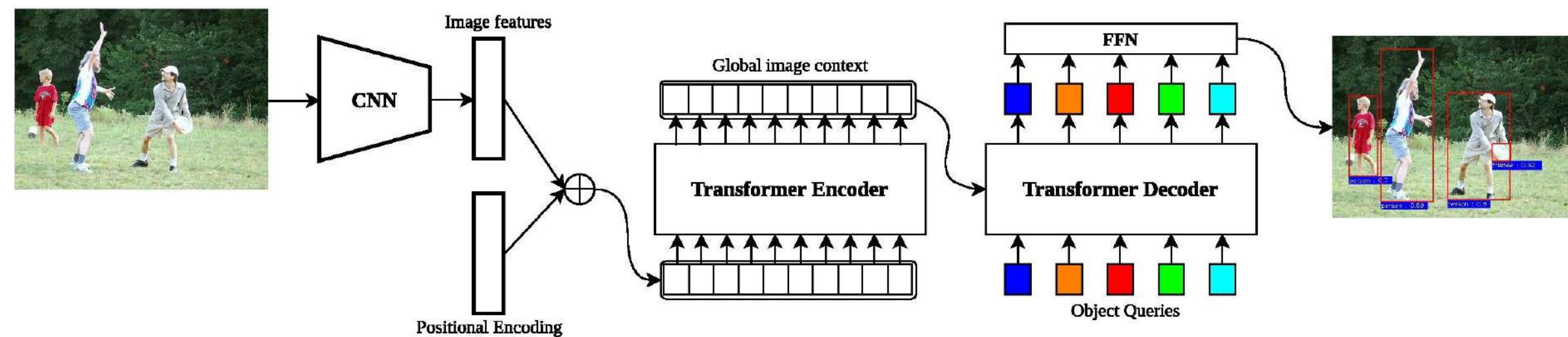
$$M_{h,w}^c = \text{ReLU} \left(\sum_k \alpha_k A_{h,w,k} \right)$$

Gradient-Weighted Class Activation Mapping (Grad-CAM)

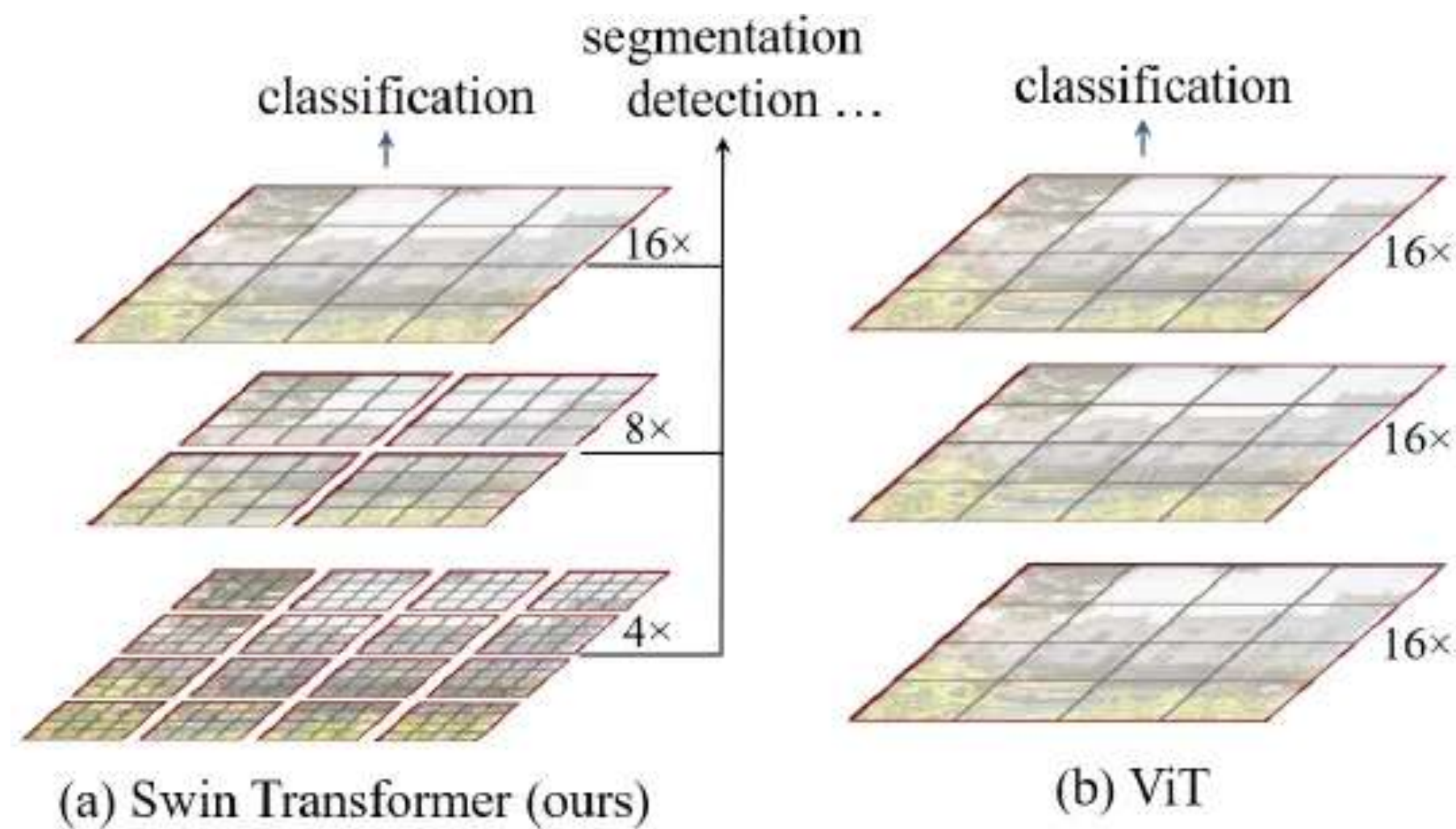


Next Lecture:

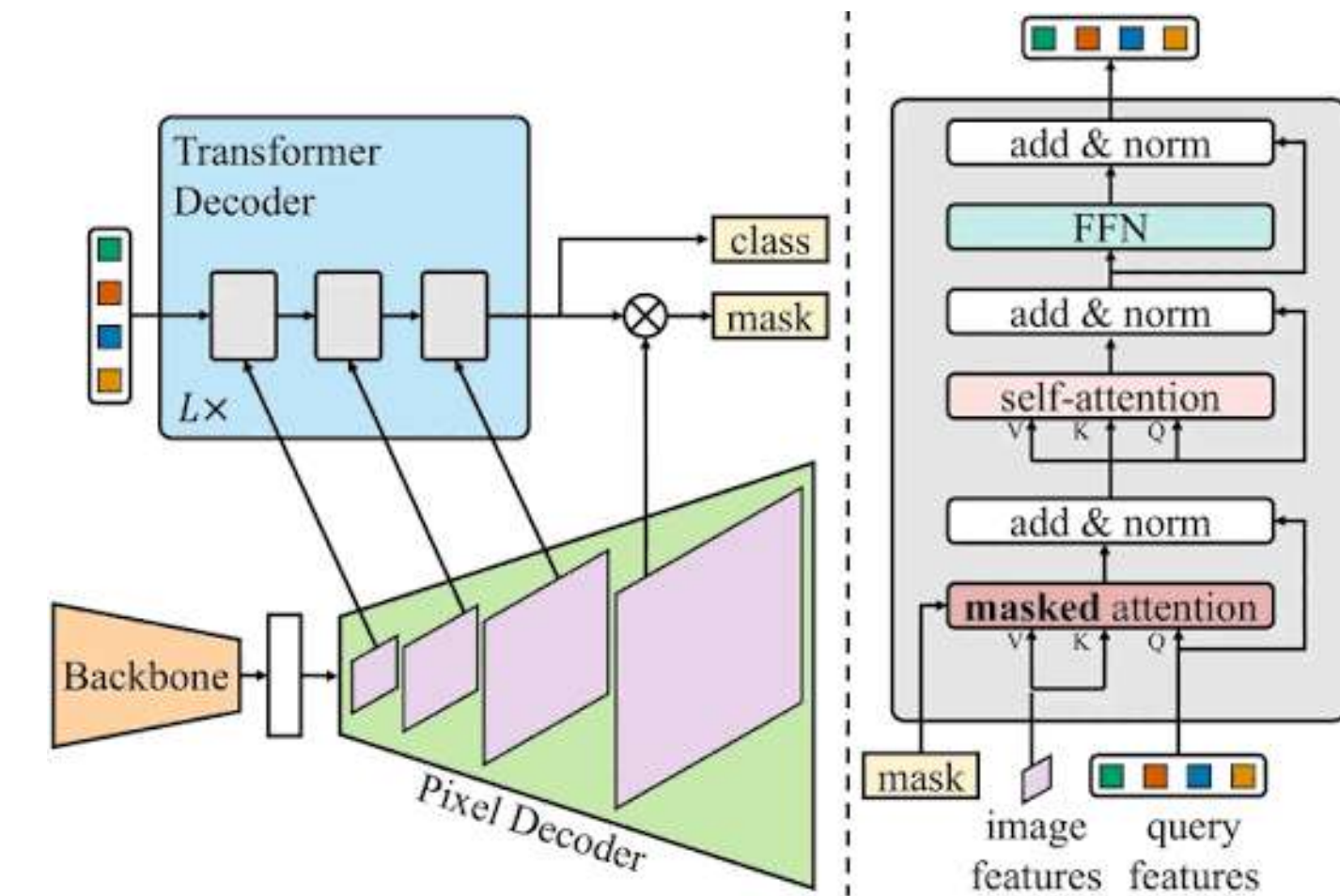
Transformer-based Detection & Segmentation



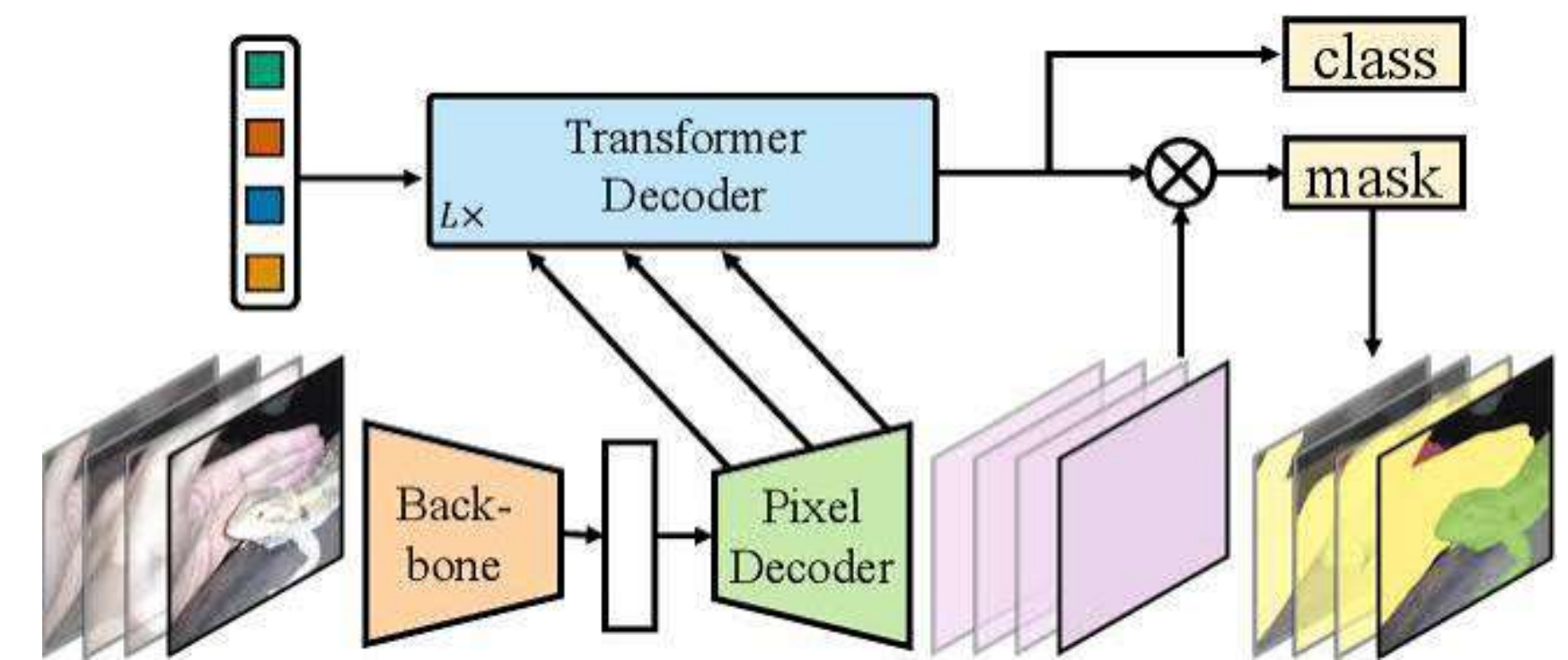
DETR



Swin Transformer



Mask2Former



Mask2Former-Video